A PROJECT REPORT ON

RAP twin

A Digital Twin and Planning Sandbox for Distributed Systems

Submitted towards the
Partial Fulfilment of the Requirements of

BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

Riaan Masoud Attar	Examination Seat No: B232311002
Avanti Nitin Patil	Examination Seat No: B231311085
Preet Prashant Boralkar	Examination Seat No: B231311013

Under the guidance of

Prof. K. K. Deshmukh

Department of Artificial Intelligence and Data Science
K. K. WAGH EDUCATION SOCIETY'S
K. K. WAGH INSTITUTE OF ENGINEERING EDUCATION AND RESEARCH
Hirabai Haridas Vidyanagari, Amrutdham, Panchavati, Nashik, Maharashtra 422003
(An Autonomous Institute since 2022, affiliated to Savitribai Phule Pune University)

A. Y. 2026-2027

CERTIFICATE

This is to certify that the project report entitled

RAP twin
A Digital Twin and Planning Sandbox for Distributed Systems

being submitted by

Riaan Masoud Attar	Examination Seat No: B232311002
Avanti Nitin Patil	Examination Seat No: B231311085
Preet Prashant Boralkar	Examination Seat No: B231311013

is a bonafide work carried out by them under the supervision of Prof. K. K. Deshmukh and it is approved for the partial fulfilment of the requirement of Bachelor of Technology (Artificial Intelligence and Data Science) during academic Year 2025-2026.

This project report has not been earlier submitted to any other Institute or University for the award of any degree or diploma.

Prof. K. K. Deshmukh
Internal Guide

Prof. Dr. D. V. Medhane
Head of Department,
Department of AI&DS

External Examiner

Date: 17-10-2026
Place: Nashik

Abstract

Researchers and engineers managing modern distributed computing fabrics – Edge, Multi-Cloud, and Hybrid environments – routinely face complex job scheduling, siloed telemetry, and unpredictable hardware failures. Evaluating a new, intelligent scheduling policy directly on production infrastructure is risky and expensive, and is prone to catastrophic failure during link outages or thermal events. **RAP twin** addresses this by providing a high-fidelity digital-twin and planning sandbox that automates and optimizes job placement, scheduling, and performance analytics for distributed systems without ever touching physical infrastructure. The platform unifies static hardware descriptors (YAML) with streaming live telemetry through a bidirectional REST API, maintaining a single synchronized state machine (`dt.state.DTState`). Workloads are routed through a pluggable planning engine offering six strategies – greedy, cheapest-energy, cheapest-cost, greenest, resilient/federated, and an RL-Markov policy – each supporting a dry-run mode that predicts Job Completion Time, energy, and SLA risk without committing real resource locks. A deterministic Chaos Engine injects link outages, packet noise, and thermal derating into the simulated topology, while a `SelfHealingController` and `ResourceGuardian` automatically reroute at-risk jobs and reclaim abandoned reservations. Beyond the original five-module design, the platform now includes cost and carbon accounting for every plan, a chaos-as-CI resilience gate that fails a build on SLA or latency regression, a blast-radius finder that searches for the smallest fault set that breaks a job, exportable incident bundles, a cluster importer that derives a twin directly from a live Kubernetes cluster, and a what-if capacity planner. All of this is exposed through a Flask REST API (API-key gated on mutating routes) and a real-time React/TypeScript dashboard. Measured benchmarks on the project's own fabric show a concrete trade-off rather than a uniform win: the resilient planner buys fallback coverage at a latency cost (p95 1.72 s vs. greedy's 1.26 s, 83.3% vs. 100% SLA hit rate under a tightened deadline), while five of the strategies make identical placements on the over-provisioned reference fabric and the cost behaviour of the RL-Markov and Resilient planners is documented as an open question, together with a list of known limitations. A suite of 166 automated tests and a live deployment validate that the platform is usable end to end. By offering an automated, scalable, and honestly-benchmarked emulation sandbox, RAP twin lets organizations statistically evaluate workload resilience and cost before committing a scheduling policy to expensive physical infrastructure.

Keywords: Digital Twin, Distributed Systems, Task Scheduling, Chaos Engineering, Predictive Analytics, Monte-Carlo Simulation, Edge Computing, Reinforcement Learning, Cost and Carbon Accounting, Resilience Testing, Flask, React.

Acknowledgement

We would like to thank our project guide, Prof. K. K. Deshmukh, for his guidance and support, and we will forever remain grateful for his constant encouragement extended in making this project report. Through our many discussions, he helped us form and solidify ideas, and the invaluable inputs from him and our team colleagues, along with the penetrating questions and motivation, have all contributed to the successful development of the concepts presented herein. We also extend our sincere gratitude to our Departmental Co-ordinator of Artificial Intelligence and Data Science, Dr. D. V. Medhane, and our esteemed Director, Dr. K. N. Nandurkar, for their continued support and leadership.

We would also like to thank our wonderful colleagues, parents and friends for listening to our ideas, asking questions, and providing feedback and suggestions for improving our ideas.

Riaan Masoud Attar
Avanti Nitin Patil
Preet Prashant Boralkar

INDEX

1 INTRODUCTION **1**
- 1.1 Project Idea . . . 2
- 1.2 Motivation of the Project . . . 3
- 1.3 Background . . . 4
  - 1.3.1 Digital Twin . . . 4
  - 1.3.2 Chaos Engineering . . . 4
  - 1.3.3 Job Placement and Scheduling . . . 4
  - 1.3.4 Federations, Fallbacks and Self-Healing . . . 4
- 1.4 Organisation of the Report . . . 4

2 LITERATURE SURVEY **6**
- 2.1 Digital Twins: Foundations and Reference Architectures . . . 7
- 2.2 Simulation-Based and Data-Driven Twins . . . 7
- 2.3 Network Digital Twins and the Edge-Cloud Continuum . . . 7
- 2.4 Intelligent Task Offloading with Reinforcement Learning . . . 8
- 2.5 Chaos Engineering . . . 8
- 2.6 Summary of the Surveyed Work . . . 8
- 2.7 Research Gap and Positioning . . . 9

3 PROBLEM DEFINITION AND SCOPE **11**
- 3.1 Problem Statement . . . 12
  - 3.1.1 Goals and Objectives . . . 12
  - 3.1.2 Assumptions . . . 14
  - 3.1.3 Scope . . . 15
- 3.2 Methodology . . . 15
  - 3.2.1 Development Approach . . . 17
- 3.3 Outcome . . . 17
  - 3.3.1 Type of Project . . . 17

4 PROJECT PLAN **19**
- 4.1 Project Timeline . . . 20
  - 4.1.1 Development History . . . 21
- 4.2 Risk Management . . . 22
- 4.3 Mentor Interaction . . . 23
- 4.4 Team Organization . . . 24

4.4.1 Team Structure 24

5 SOFTWARE REQUIREMENT SPECIFICATION 26

5.1 Product Perspective 27
5.1.1 User Classes 27
5.1.2 Use Cases 28
5.2 Functional Requirements 28
5.3 Non-Functional Requirements 30
5.4 Constraints 30
5.5 Hardware Requirements 31
5.6 Software Requirements 31
5.7 Interfaces 31
5.8 Requirements Traceability 32

6 DETAILED DESIGN 34

6.1 Architectural Design 35
6.1.1 Layered View 35
6.1.2 Execution and Concurrency Model 36
6.1.3 Deployment Architecture 37
6.1.4 Key Design Decisions 38
6.2 Overview of Project Modules 38
6.2.1 State Engine 39
6.2.2 Predictive Analytics 40
6.2.3 Cost Model 41
6.2.4 Economics 42
6.2.5 Planner Factory and Strategies 42
6.2.6 Chaos Engine 43
6.2.7 Self-Healing and Resource Guardian 44
6.2.8 CI Resilience Gate 45
6.2.9 Blast-Radius Finder 45
6.2.10 What-If Planner 46
6.2.11 Monte-Carlo Orchestrator 46
6.2.12 Interoperability, Bundles and MQTT 47
6.2.13 API, Events and Dashboard 47
6.3 User Interface 48
6.3.1 Cross-Cutting Behaviour 49
6.3.2 Dashboard Tab: Overview Screen 49
6.3.3 Topology and Federations Screens 50
6.3.4 Job Composer Screen 50
6.3.5 Recent Plans and Plan Detail Screens 51
6.3.6 Nodes and Links Screens 51
6.3.7 Jobs Screen 51
6.3.8 Chaos and Faults Screens 51
6.3.9 What-If and Reservations Screens 51
6.3.10 Documentation Screen 52

6.3.11 Screen-to-Interface Mapping . . . 52
6.4 Data Design . . . 52
6.4.1 Data Structure . . . 52
6.4.2 Database Description . . . 56
6.5 UML Diagrams . . . 57
6.5.1 Use-Case Diagram . . . 57
6.5.2 Class Diagram . . . 57
6.5.3 Sequence Diagrams . . . 60
6.5.4 State Diagrams . . . 61
6.6 Tools and Technology Used . . . 62
6.7 Algorithm . . . 63
6.7.1 Problem Formulation and Notation . . . 64
6.7.2 Greedy Family: Greedy, Cheapest-Energy, Cheapest-Cost, Greenest, Bandit . 64
6.7.3 Resilient (Federated) Planner for Fault-Tolerant Placement . . . 66
6.7.4 RL-Markov Planner: Markov Decision Process . . . 69
6.7.5 Cost, Energy and Carbon Objectives . . . 72
6.7.6 Predictive Reliability Update . . . 72
6.7.7 Self-Healing Decision Rule . . . 72
6.7.8 Minimal Fault-Set Search (Blast Radius) . . . 73
6.7.9 Comparison of the Strategies . . . 74

7 RESULTS AND DISCUSSION 75
7.1 Experimental Setup . . . 76
7.1.1 Dataset . . . 76
7.1.2 Performance Parameters . . . 76
7.2 Additional Experiments . . . 77
7.2.1 Strategy Comparison in Dry-Run Mode . . . 77
7.2.2 Committed Planning with Contention . . . 78
7.2.3 Chaos-as-CI Gate . . . 79
7.2.4 Monte-Carlo Sweep . . . 80
7.2.5 Blast-Radius and What-If Examples . . . 80
7.3 Efficiency Issues . . . 81
7.4 Software Testing . . . 81
7.4.1 Test Cases and Test Results . . . 81
7.5 Known Limitations and Threats to Validity . . . 83
7.6 Summary . . . 84
7.6.1 Result Analysis and Discussion . . . 84

8 CONCLUSION AND FUTURE WORK 85
8.1 Conclusions . . . 86
8.2 Future Work . . . 86
8.3 Application . . . 86

Annexure A Program Outcome and Program Specific Outcome Mapping
A.1 Program Outcome (PO) Mapping . . .

A.2 Program Specific Outcome (PSO) Mapping . . . . . .

Annexure B References

Annexure C Plagiarism Report

Annexure D Paper Published (if any)

Annexure E Sponsorship detail (if any)

List of Tables

2.1 Surveyed work and its relation to RAP twin . . . 9
3.1 Objectives, acceptance criteria and evidence . . . 14
4.1 Project Timeline . . . 20
4.2 Project phases and deliverables . . . 21
4.3 Selected milestones from the Git history . . . 22
4.4 Risk register . . . 23
4.5 Mentor suggestions and their status . . . 24
5.1 User classes . . . 27
5.2 Use-case summary (full reports are in the SRS) . . . 28
5.3 Functional requirements, implementation and status . . . 28
5.4 REST and SSE interface . . . 32
5.5 Traceability of requirement groups to tests . . . 33
6.1 Layers of the implementation . . . 36
6.2 Background threads in the API process . . . 36
6.3 Design decisions and their rationale . . . 38
6.4 Module inventory . . . 39
6.5 Strategies exposed by the planner factory . . . 43
6.6 Chaos event kinds . . . 44
6.7 Environment variables that configure the twin . . . 48
6.8 Dashboard tabs and the interfaces they use . . . 52
6.9 Principal data stores . . . 53
6.10 Node descriptor blocks and their consumers . . . 54
6.11 Statistics of the reference fabric (`nodes/`, `sim/topology.yaml`) . . . 55
6.12 Technologies and their role . . . 63
6.13 Notation used in the algorithm descriptions . . . 64
6.14 Parameters of the greedy family (`DEFAULT_CFG` and the factory overrides) . . . 65
6.15 Presets of the resilient planner (`DEFAULT_MODES`) . . . 67
6.16 Criteria and weights of the Markov planner . . . 70
6.17 Comparison of the planning strategies . . . 74
7.1 Measured Benchmark Table (commit `2e7962d`, seed 7, job_limit 6) . . . 76
7.2 Dry-run planning of the 11-job catalogue on the 100-node fabric (one pass; planning time over 20 repetitions) . . . 77
7.3 Sequential committed planning of the 11-job catalogue (each job sees the capacity reserved by earlier jobs) . . . 78
7.4 Resilience gate results (committed baseline, seed 7, 6 jobs, deadline scale 0.28) . . . 79
7.5 What-if analyses of the 11-job catalogue (Greedy, committed planning in both forks) 80

7.6 Test cases by group (166 tests, all passed) 81

List of Figures

3.1 RAP twin – High-Level Closed-Loop Pipeline 16
3.2 RAP twin – Simulation and Planning Sub-Pipeline 17

1. System Architecture

6.1 RAP twin – Current Architecture (full module set) 35
6.2 Deployment architecture of the public instance 37

2. User Interface Screens

6.3 Dashboard navigation: eight tabs and their contents 49
6.4 Composition of the Dashboard tab (schematic) 50
6.5 Logical entity-relationship diagram of the twin's data 56

3. UML Diagrams

6.6 Use Case Diagram 57
6.7 Class diagram of the digital-twin core (`dt/`) 59
6.8 Sequence diagram: planning a job with `POST /plan` 60
6.9 Sequence diagram: a node fault and the self-healing reaction 61
6.10 State diagrams: (a) reservation lifecycle, (b) node health 62
6.11 Flowchart of the resilient planner 68
6.12 Flowchart of the Markov planner's recursion 71

4. Datasets

7.1 RAP twin – Measured Benchmark Dataset (commit `2e7962d`, seed 7) 76
7.2 Planning time and predicted latency per strategy 77
7.3 Catalogue cost and carbon per strategy (dry run) 78
7.4 Committed sequential planning: latency, cost and reserved CPU 79
7.5 p95 predicted latency at each gate checkpoint 80

CHAPTER 1
INTRODUCTION

1.1 Project Idea

The world today is rapidly advancing into the era of 5G/6G and mobile edge computing, where distributed computing fabrics are becoming highly dynamic and complex. A single organisation may now operate gaming-class workstations, single-board computers, phones, GPU servers and an HPC partition as one logical pool of compute, joined by links that range from 400 Gbps InfiniBand to a congested campus Wi-Fi access point. Network engineers and researchers face massive challenges dealing with this heterogeneous hardware, siloed telemetry, and varying link reliability. Testing a new, intelligent job scheduling policy directly on production infrastructure is expensive and risky, because a faulty policy may overload a node, strand a job behind a failed link, or silently trade away reliability for speed. That is where RAP twin comes in.

RAP twin is a high-fidelity digital-twin and planning sandbox designed to automate, optimise, and simulate job placement and scheduling across distributed systems. Instead of risking live infrastructure, the tool provides a safe, self-contained emulation ecosystem that mirrors a physical fabric. By continuously merging static hardware descriptors with streaming live telemetry through a bidirectional REST API, RAP twin allows researchers to experiment with offloading strategies in a closed-loop environment: a job is planned against the twin, faults are injected into the twin, and the platform's controllers react to those faults, all without a single production node being touched.

The main technologies and features that power RAP twin include:

- **Multi-Strategy Planning Engine:** a pluggable planning engine supports six headline strategies – Greedy, Cheapest-Energy, Cheapest-Cost, Greenest, Resilient/Federated, and an RL-Markov policy – together with a Bandit variant that learns the best execution format for each stage. Researchers can therefore target strict latency, cost, energy, carbon, or fault-tolerance goals with the same job descriptor.
- **Chaos Engineering, CI Gating, and Monte-Carlo Tooling:** the platform injects deterministic faults (fourteen event kinds covering link outages, node kills, thermal derating, power caps and whole-zone blackouts), runs Monte-Carlo trial sweeps to evaluate policies statistically, and provides a chaos-as-CI gate that fails a build when a scheduling change regresses SLA, latency, cost, or carbon against a committed baseline.
- **Predictive Analytics and Unified State Management:** background controllers continuously ingest live telemetry (through the `/observe` endpoint or an MQTT sidecar) to forecast node utilisation, thermal derating, reliability and failure windows, and automatically reroute work or reclaim resources when a node degrades.
- **Economics, Blast-Radius, and What-If Analysis:** every plan is priced in money (INR) and carbon (CO_2); a blast-radius finder searches for the smallest fault set that breaks a job; and a what-if planner forks the twin to answer capacity questions ("would two more nodes fix our deadline misses?") without touching the live state.

The optimised placement plans and performance analytics produced by RAP twin can be explored in real time through a web dashboard, or exported as statistical reports (CSV, JSON) and industry-standard interoperability formats (Azure DTDL, Kubernetes CRDs). A cluster importer can also work in the opposite direction, deriving a twin directly from a live Kubernetes cluster. With RAP twin, organisations and researchers can statistically evaluate workload resilience and cost before deploying a scheduling policy to physical infrastructure, whether in Industry 4.0 smart manufacturing, the Internet of Vehicles, or next-generation multi-cloud telecommunications.

1.2 Motivation of the Project

In the rapidly advancing era of 5G/6G and mobile edge computing, managing distributed computing fabrics has become a highly dynamic and complex challenge. While next-generation networks promise unprecedented connectivity, engineers struggle with heterogeneous hardware, siloed telemetry, and varying link reliability. Traditional, static computation-offloading strategies fail to adapt to unpredictable conditions, and testing new scheduling policies directly on live production infrastructure is expensive, risky, and prone to catastrophic failure.

The motivation behind RAP twin is to solve these problems in distributed edge orchestration by providing a safe, high-fidelity digital-twin and planning sandbox. The key motivating factors are:

1. **Bridging the Gap for Intelligent Scheduling:** academic literature proposes valuable but theoretical Multi-Agent Deep Reinforcement Learning (MADRL) and federated models for task offloading, yet practical tools to evaluate them safely are scarce. RAP twin's multi-strategy planning engine lets such algorithms be compared against strict latency, cost, and energy goals without risk.
2. **Risk Mitigation and Cost Savings:** evaluating offloading strategies directly on physical infrastructure is labour-intensive and risky. Automating closed-loop simulation locally removes that risk and lets engineers run safe, offline policy sweeps.
3. **Resilience and Chaos Engineering:** edge environments are riddled with unpredictable errors – node failures, link jitter, dynamic thermal derating. RAP twin's chaos engine systematically injects deterministic faults, and its chaos-as-CI gate turns that into an automated regression check on every change.
4. **Unified State and Predictive Analytics:** live streaming telemetry is continuously merged with static hardware descriptors through a bidirectional REST API (`/observe`), and a predictive analytics engine computes utilisation forecasts and failure windows to help meet strict SLAs.
5. **Honest, Data-Driven Policy Benchmarking:** RAP twin's Monte-Carlo tooling and measured benchmark reports (Chapter 7) are built to surface real trade-offs and open questions – such as the resilient planner's latency and egress cost for fallback coverage, and the fact that several strategies produce identical placements on an over-provisioned fabric – rather than present only favourable numbers.
6. **Scalability and Interoperability:** RAP twin is designed to handle large, heterogeneous fabrics (the reference fabric has 100 nodes) and to interoperate with real infrastructure: it can import a live Kubernetes cluster as a twin, and export a twin's state as Azure DTDL or Kubernetes CRDs for an external control plane.
7. **Time-Sensitive Task Offloading:** in domains such as Industry 4.0 and autonomous vehicles, where tasks must be processed within strict deadlines, RAP twin's dry-run evaluation mode lets organisations confidently deploy algorithms capable of meeting those constraints.

Ultimately, the goal is to help organisations in sectors such as Industry 4.0 smart manufacturing, the Internet of Vehicles, and next-generation 5G/6G telecommunications unlock the potential of AI, federated, and Markov-based scheduling policies, while reducing the operational

costs, deployment risks, and hardware inefficiencies that have historically plagued edge computing environments.

1.3 Background

This section introduces the four concepts on which the rest of the report depends.

1.3.1 Digital Twin

A digital twin is a virtual counterpart of a physical system that is kept synchronised with it through a continuous flow of data, so that experiments run on the virtual copy predict how the physical system would behave [1], [5]. In the context of this project the "physical system" is a computing fabric: a set of nodes, each with a CPU, memory, optional accelerators, a power profile and a health record, connected by links with a speed, a round-trip time, jitter and a loss rate. The twin holds the *static* description of that fabric (YAML descriptors) and a *dynamic* overlay of live observations (current load, injected faults, forecast reliability); planners always see the merged view.

1.3.2 Chaos Engineering

Chaos engineering is the discipline of deliberately injecting failures into a system in order to build confidence in its resilience [13]. Existing tools mostly act on running application containers. RAP twin applies the idea one level down, to the *scheduler*: the faults are injected into the twin, and the question being asked is whether a scheduling policy still meets its deadlines while the fabric is degraded.

1.3.3 Job Placement and Scheduling

A job in RAP twin is a directed pipeline of *stages*. Each stage declares the data volume it consumes, the CPU cores, memory and accelerator memory it needs, and the execution formats it accepts (native, WebAssembly, CUDA, NPU and so on). *Placement* is the decision of which node runs each stage; *planning* is the process that makes that decision and predicts its consequences (latency, energy, risk, cost, carbon). A planner is therefore a function from a job and the current state of the twin to a placement and a prediction, and the platform's central design choice is to make that function pluggable.

1.3.4 Federations, Fallbacks and Self-Healing

Nodes are grouped into *federations* that are derived from their labels (federation, zone, site, rack or region, in that order of precedence). A fault-tolerant planner can spread a job across federations so that the loss of one zone does not stop the job, and can reserve one or more *fallback* nodes for each stage. A background controller watches the primary node of each committed stage and, when the node degrades, promotes a fallback; this is the self-healing loop described in Section 6.2.7.

1.4 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 surveys the literature on digital twins, chaos engineering and intelligent task offloading, and identifies the gap that RAP twin addresses. Chapter 3 states the problem, the goals, assumptions and scope, and describes the methodology. Chapter 4 presents the project plan and the team organisation. Chapter 5 summarises the software requirements. Chapter 6 describes the detailed design: the architecture, each module, the dashboard, the data design, the UML diagrams, the technologies used and the algorithms. Chapter 7 reports the experiments, the test suite and the limitations that apply to the results.

Chapter 8 concludes and lists future work. The annexures map the project to the programme outcomes and give the references.

CHAPTER 2
LITERATURE SURVEY

Over the past decade, significant progress has been made in the development of Digital Twins (DTs) and in the management of distributed computing systems. However, most existing systems are either domain-specific or lack the intelligent, integrated mechanisms needed to handle the scheduling decisions, unpredictable failures and performance variability inherent in modern distributed computing fabrics. This chapter reviews thirteen representative works in five themes – digital-twin foundations, data-driven twins, network and edge twins, intelligent task offloading, and chaos engineering – and then states the gap that RAP twin addresses. Reference numbers in square brackets point to the list in Annexure B.

2.1 Digital Twins: Foundations and Reference Architectures

Qi et al. [1] and Fuller et al. [2] established the foundational case for digital twins in industry, reviewing the enabling technologies and the open research challenges. Both stop short of a concrete, pluggable scheduling engine that a researcher can actually run against a simulated fabric. Lv et al. [4] surveyed digital-twin reference architectures more broadly and emphasised the need for continuous, bidirectional data synchronisation between a physical entity and its virtual counterpart, so that the twin can support real-time monitoring and prediction. Grieves and Vickers [5] supply the original conceptual grounding of the digital twin across engineering fields and review how the idea has evolved, while Geyer and Bondorf [3] study a proximity-based service-discovery mechanism for distributed twin systems. Taken together these papers establish three design principles that RAP twin follows: the twin must be *synchronised* with its physical counterpart, it must be *modular* and loosely coupled so that components can be exchanged, and it must be usable by a researcher without proprietary infrastructure.

RAP twin operationalises these principles in its core state engine, `dt.state.DTState`. The engine merges static topology descriptors with live telemetry received through the `/observe` endpoint (and, for broker-based deployments, through an MQTT sidecar), so the planning engine always reflects current constraints. Its planner factory (`dt/planners.py`) makes the scheduling policy a replaceable component, which is the modularity the surveys call for.

2.2 Simulation-Based and Data-Driven Twins

Wang et al. [6] present a simulation-based digital-twin model for data-driven decision optimisation, and Kim et al. [7] combine a twin with machine learning for real-time monitoring and predictive maintenance. Both emphasise that a twin is only as useful as its predictions, and that a unified predictive view built from siloed telemetry sources remains an operational challenge. RAP twin's `PredictiveAnalyzer` (Section 6.2.2) addresses this on a modest scale: it processes CPU load, battery, reliability and link measurements into one set of per-node and per-link forecasts – utilisation, reliability, availability window, projected thermal derating – and these forecasts feed directly into the cost model and the planners.

2.3 Network Digital Twins and the Edge-Cloud Continuum

Sai et al. [10] and Raza et al. [11] survey Digital Twin Networks and Network Digital Twin architectures for the edge-cloud continuum, and Tran-Dang and Kim [12] survey digital-twin-empowered computation offloading for 5G and beyond. The three works converge on one conclusion: a complete twin system needs multi-layered state management that merges static descriptors, master data and real-time streaming telemetry, and it needs standard interfaces for interoperability. Yet

these are catalogues of architectural requirements, not benchmarked implementations. RAP twin's interoperability layer answers the standardisation requirement concretely: it exports the twin as Azure DTDL models and Kubernetes custom resources, and it imports a live Kubernetes cluster as a twin (`tools/import_cluster.py`).

2.4 Intelligent Task Offloading with Reinforcement Learning

Xu et al. [8] propose digital-twin-driven collaborative scheduling of heterogeneous tasks and edge-end resources with multi-agent deep reinforcement learning, and Dai et al. [9] apply federated deep reinforcement learning to task offloading in digital-twin edge networks. Both treat the deviation between the twin's estimate of a device's compute capacity and its real capacity as a first-class quantity. Such models offer sophisticated autonomous decision making, but their computational cost and limited interpretability make them difficult to compare fairly with traditional heuristics, and the papers do not ship a platform in which a reader can run the learned policy next to simple baselines on the same fabric.

RAP twin takes the complementary route. Its RL-Markov planner (Section 6.7.4) is a transparent finite-horizon Markov decision process solved by dynamic programming, optionally biased by a tabular Q-learning component; it sits behind the same interface as the Greedy, Cheapest-Energy and Resilient planners, so a reader can substitute one for another and compare the predicted latency, energy, risk, cost and carbon. The project does *not* claim to reproduce the multi-agent or federated learning systems of [8] and [9]; it provides the evaluation harness in which such a policy could be tested.

2.5 Chaos Engineering

Esen et al. [13] review chaos experiments in microservice architectures and catalogue the tools used to inject failures and build confidence against turbulent conditions. These tools are effective, but running them in production carries inherent risk, and they mostly target application-level failures such as destroyed containers rather than physical-layer disruption. RAP twin bridges the gap with an integrated chaos engine that injects deterministic physical-layer failures – link outages, node failures, thermal derates, zone blackouts – into a simulated sandbox, and goes one step further with a chaos-as-CI resilience gate that replays a fault schedule against every proposed scheduling change.

2.6 Summary of the Surveyed Work

Table 2.1 summarises the focus of each reference and the way RAP twin relates to it.

Table 2.1: Surveyed work and its relation to RAP twin

Ref.	Type / theme	Main focus	Relation to RAP twin
[1], [2]	Surveys; DT foundations	Enabling technologies, challenges and open research in industrial twins	Motivates a runnable twin with a concrete scheduling engine
[4]	Survey; DT architecture	Reference architectures, bidirectional synchronisation	`DTState` merges static and live data
[5]	Concept paper	Origin and evolution of the digital-twin concept	Conceptual basis of the twin/physical split
[3]	Method; DT systems	Proximity-based service discovery for distributed twins	Reinforces modular, loosely coupled design
[6]	Method; simulation	Simulation-based twin for decision optimisation	Dry-run planning and what-if forks
[7]	Method; ML twin	Real-time monitoring and predictive maintenance	`PredictiveAnalyzer` forecasts
[10], [11]	Surveys; network twins	Architecture, capabilities and requirements of network twins and the edge-cloud continuum	Open APIs, DTDL/CRD export, cluster import
[12]	Survey; offloading	Twin-empowered computation offloading for 5G and beyond	Pluggable offloading strategies
[8], [9]	Methods; DRL	Multi-agent and federated deep RL for twin-driven offloading	RL-Markov policy as a comparable baseline harness
[13]	Survey; chaos	Chaos experiments in microservices	Physical-layer chaos engine and CI gate

2.7 Research Gap and Positioning

The surveyed literature supports four observations. First, digital-twin research has produced reference architectures and surveys [1], [2], [4], [10], [11], but few of them are accompanied by an implementation that a researcher can run. Second, twin-driven offloading work [8], [9] concentrates on learned policies and typically reports results for the learned policy alone. Third, chaos engineering [13] targets running application infrastructure rather than the scheduling policies that decide where work runs. Fourth, none of the surveyed works treats money and carbon as first-class planning objectives alongside latency, or provides a regression gate that stops a scheduling change

from merging when it degrades resilience.

RAP twin addresses these gaps by integrating, in one open and tested platform, (i) a synchronised twin of a heterogeneous fabric, (ii) a pluggable multi-strategy planning engine with a dry-run mode, (iii) a chaos engine and a CI gate that test the *policy* rather than the infrastructure, (iv) per-plan cost and carbon accounting, and (v) import and export paths to real infrastructure. It also commits to reporting its own limitations (Section 7.5) so that its results can be interpreted correctly.

CHAPTER 3
PROBLEM DEFINITION AND SCOPE

3.1 Problem Statement

To develop a high-fidelity digital twin and planning sandbox that can intelligently schedule and orchestrate heterogeneous workloads across complex distributed computing fabrics – such as Edge, Multi-Cloud, and Hybrid environments, including those with siloed telemetry, dynamic thermal conditions, and unpredictable link outages – without risking costly disruptions to live production infrastructure. The platform must improve the statistical evaluation of advanced task-offloading policies and enable safe, large-scale benchmarking and resilient architectural decision-making, while honestly reporting the trade-offs and open issues that the benchmarking itself surfaces.

The problem can be decomposed into five sub-problems, each of which maps onto a part of the system:

1. **Representation:** how to describe a heterogeneous fabric (compute, memory, accelerators, power, health, links) and keep that description synchronised with live observations.
2. **Decision:** how to choose, for every stage of a job, a node that satisfies resource and format constraints while balancing latency, energy, cost, carbon and risk.
3. **Resilience:** how to keep a job running when nodes or links fail, by reserving fallbacks and re-routing automatically.
4. **Evaluation:** how to test a policy repeatedly and reproducibly, under faults, without touching production, and how to turn that test into an automated regression gate.
5. **Presentation and integration:** how to expose the twin to operators (dashboard, API) and to existing systems (Kubernetes, Azure Digital Twins).

3.1.1 Goals and Objectives

Goals

1. Establish RAP twin as a high-fidelity sandbox and emulation tool for distributed computing fabrics in sectors such as Industry 4.0 manufacturing, 5G/6G telecommunications, and the Internet of Vehicles.
2. Demonstrate measurable, honestly reported trade-offs between job completion time, cost, energy and SLA compliance across competing scheduling strategies, rather than claiming a uniform win for any one policy.
3. Contribute to the safe transition of theoretical AI models (multi-agent and federated deep reinforcement learning) into practical engineering by removing the risk of failure on physical infrastructure.
4. Develop a scalable, closed-loop platform capable of ingesting live streaming telemetry (REST or MQTT) while maintaining accurate twin-to-physical parity.

Objectives

1. **Enable Intelligent Multi-Strategy Planning:** dynamically schedule heterogeneous workloads using a pluggable engine (Greedy, Cheapest-Energy, Cheapest-Cost, Greenest, Resilient/Federated, RL-Markov) tailored to latency, cost, energy and carbon targets.
2. **Achieve Real-Time State Synchronisation:** continuously merge static hardware descriptors (YAML) with live streaming telemetry through a bidirectional REST API, maintaining an accurate, unified state machine of the fabric.

3. **Automate Predictive Analytics and Self-Healing:** forecast thermal derating, node utilisation and network loss using background controllers (`SelfHealingController`, `ResourceGuardian`) that manage resources dynamically and clean up expired reservations.
4. **Facilitate Deterministic Chaos Engineering and CI Gating:** inject simulated network noise, link outages and thermal throttling, and automatically fail a build when a scheduling change regresses SLA, latency, cost or carbon against a committed baseline.
5. **Execute Large-Scale Statistical Benchmarking:** orchestrate many scheduling trials through Monte-Carlo simulation, generating statistical reports of workload resilience, cost and SLA hit rate.
6. **Support Capacity and Impact Analysis:** let engineers evaluate a proposed hardware change (what-if) or find the smallest fault set that breaks a job (blast radius) without touching the live twin.
7. **Provide Interoperability with Real Infrastructure:** import a live Kubernetes cluster as a twin, and export twin state as Azure DTDL or Kubernetes CRDs.

Table 3.1 turns each objective into a measurable acceptance criterion and records how the criterion is checked and where the evidence appears in this report.

Table 3.1: Objectives, acceptance criteria and evidence

Obj.	Acceptance criterion	How it is checked	Evidence
1	Seven strategy names resolve to a planner; every strategy plans all 11 catalogue jobs without an infeasible result	`tools/policy_benchmark.py`; experiment of Section 7.2.1	Ch. 7
2	A telemetry change posted to `/observe` is visible in the next snapshot; descriptor edits reload in about 0.5 s	`test_api.py`, `test_management.py`; watcher interval in `dt/state.py`	Ch. 7
3	A degraded primary is replaced by a fallback after two consecutive polls; expired reservations are reclaimed	`dt/self_heal.py` thresholds; event types `fabric.selfheal.*`	Sec. 6.2.7
4	The CI gate returns the same verdict for the same commit and seed, and fails when a threshold is breached	`test_ci_gate.py` (determinism and failure cases)	Sec. 6.2.8
5	A seeded Monte-Carlo sweep writes per-trial CSV and prints mean, p95, p99 and SLO hit rate	`sim/montecarlo.py`	Sec. 7.2.4
6	What-if and blast-radius runs never modify live state or files on disk	`test_whatif.py`, `test_blast_radius.py`	Sec. 7.2.5
7	A saved Kubernetes node list imports to valid descriptors; DTDL and CRD documents are served	`test_import_cluster.py`; `/standards/*`	Sec. 6.2.12

3.1.2 Assumptions

- **Telemetry Quality:** static hardware descriptors and live telemetry updates are assumed to reflect the real-world state of edge nodes and network links reasonably well.
- **Data Availability:** organisations can provide workload descriptors and streaming telemetry through the designated REST (or MQTT) channels.
- **Technical Infrastructure:** the target user has a local workstation or a Docker-enabled environment to host the Flask API, the dashboard and the simulation ecosystem.
- **Algorithm Suitability:** learned and Markov-based scheduling algorithms can optimise task offloading when given an accurate virtual representation of the physical network.
- **Job Model:** a job is a sequential pipeline of stages; the cost model charges the transfer of each stage's data from the node that ran the previous stage.

- **Prediction Fidelity:** latency, energy and risk are *model predictions*, not measurements from executed workloads; their absolute values are meaningful only relative to one another (Section 7.5).

3.1.3 Scope

- Developing a high-fidelity digital-twin and planning sandbox for simulating, scheduling and orchestrating heterogeneous tasks across distributed computing fabrics.
- Converting static hardware constraints and live telemetry into optimised job placement plans, cost and carbon estimates, and statistical performance metrics.
- Supporting domains that rely on edge computing and next-generation architectures: Industry 4.0 smart manufacturing, the Internet of Vehicles, and 5G/6G telecommunications.
- Incorporating deterministic fault injection (the chaos engine), a chaos-as-CI resilience gate and Monte-Carlo tooling to validate scheduling resilience and SLAs under network uncertainty.
- Emphasising real-time state synchronisation by continuously merging static node descriptors with live telemetry.
- Focusing on task offloading and simulated policy evaluation (Greedy, RL-Markov, Resilient/Federated, and so on) rather than the physical execution of containerised workloads, although a containerised fabric launcher is provided for local emulation.
- Developing both an administrative backend (Flask API) and an interactive web dashboard for topology health, predictive analytics, cost and carbon, and job history.
- Providing interoperability tooling (cluster import, DTDL/CRD export) rather than acting as a production orchestrator in its own right.

Out of scope. The project does not execute the workloads it plans, does not provide a production scheduler for Kubernetes or any other orchestrator, does not train deep reinforcement-learning agents, and does not include live integration with a physical 5G/6G test-bed (requirement FR-30 is recorded as planned work).

By focusing on these assumptions and clearly defining the scope, RAP twin offers a robust solution to the problems of distributed task scheduling and policy evaluation, while minimising the challenges and risks associated with testing directly on live production infrastructure.

3.2 Methodology

Fig. 3.1 shows the complete, closed-loop process flow of RAP twin. The pipeline has seven steps; the module that implements each step is named so that the steps can be traced to Chapter 6.

1. **Physical Fabric and Telemetry Ingestion:** edge servers, IoT devices and static hardware descriptors supply live telemetry and static configuration through the REST gateway (`/observe`) or the MQTT sidecar (`sim/mqtt_bridge.py`).
2. **State Merge and Sandbox Synchronisation:** incoming updates are merged by `DTState` into a per-node and per-link *dynamic* overlay on top of the *base* descriptors, producing the *effective* view that planners read.
3. **Chaos Engineering and Background Controllers:** the `ChaosRunner`, `SelfHealingController` and `ResourceGuardian` run continuously to inject faults, keep primary placements healthy, and release stale reservations.

4. **Unified State Management and Predictive Analytics:** the `PredictiveAnalyzer` turns utilisation, battery and link measurements into forecasts that feed the cost model and the planners.
5. **Multi-Strategy Planning and Dry-Run:** a job descriptor with an SLA deadline is evaluated by the selected policy; in dry-run mode the plan and its predicted latency, energy, risk, cost and carbon are returned without committing any reservation.
6. **Monte-Carlo, What-If and Blast-Radius Analysis:** synthetic workload profiles are simulated across many trials; the what-if planner forks the twin to test a proposed hardware change; the blast-radius search finds the smallest fault set that breaks a job.
7. **Visualisation, Export and Gating:** results are shown on the dashboard, exported as Azure DTDL or Kubernetes CRDs, and checked by the CI gate before a scheduling change merges.

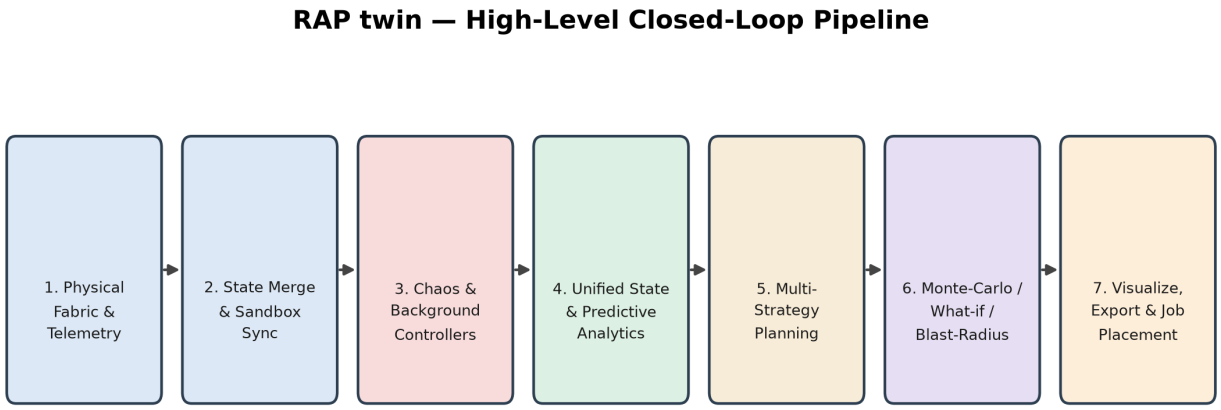


Figure 3.1: RAP twin – High-Level Closed-Loop Pipeline

The simulation and planning sub-pipeline (Fig. 3.2) refines steps 4 to 7 and ensures high-fidelity, closed-loop emulation from raw telemetry ingestion to statistical performance evaluation:

1. **State Management and Telemetry Ingestion:** static descriptors and job descriptors (SLAs, multi-stage workflows, resource hints) are parsed and validated; live telemetry is merged continuously; the controllers monitor node reliability and reclaim expired reservations.
2. **Multi-Strategy Workload Planning:** the pluggable engine assigns stages to nodes using the selected strategy; the dry-run evaluator predicts the outcome so that users can perform "what-if" analysis without committing resources.
3. **Chaos Engineering and Fault Injection:** the chaos engine injects deterministic, physical-layer failures so that policies can be observed adapting and re-routing under realistic unpredictability.

4. **Cost, Carbon, What-If and Blast-Radius Analysis:** every plan is priced in money and carbon; proposed capacity changes are evaluated on a forked twin; the smallest fault set that breaks a given job is found automatically.
5. **Dashboard Preview, CI Gate and Export:** users get a real-time view of topology, predictive health and job history; the CI gate can fail a build on regression; the final twin state can be exported to open standards.

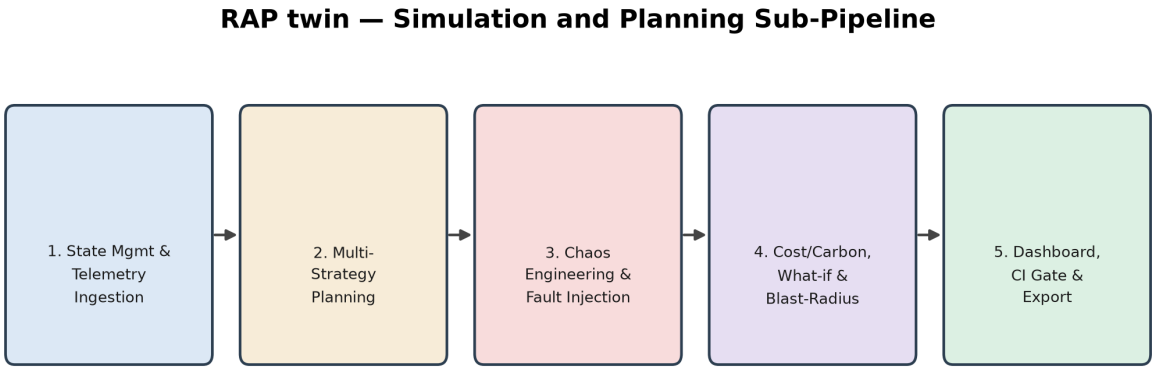


Figure 3.2: RAP twin – Simulation and Planning Sub-Pipeline

3.2.1 Development Approach

The platform was built incrementally in the order of the dependencies in Figure 3.1: first the state engine and the baseline planner, then the cost model and predictor, then the chaos engine, the controllers and the dashboard, and finally the advanced modules (economics, CI gate, blast radius, bundles, cluster import, what-if, authentication and MQTT). Each module was accompanied by automated tests (Section 7.4), and every number reported in Chapter 7 is produced by a script that is committed with the project so that the experiment can be repeated.

3.3 Outcome

The direct outcome of this project is a working, tested and deployed platform: a Flask REST API exposing 29 routes (24 distinct paths), seven pluggable scheduling strategy names, a chaos engine wired into an automated CI resilience gate, cost and carbon accounting, blast-radius and what-if analysis tools, incident bundle export and import, a Kubernetes cluster importer, an MQTT telemetry sidecar, and a real-time React/TypeScript dashboard with an in-app documentation section. The platform is covered by 166 automated tests (all passing) and has been deployed to a public server. Measured benchmarks (Chapter 7) show concrete, honestly reported trade-offs between the available strategies rather than a single policy declared the winner on every metric.

3.3.1 Type of Project

RAP twin is primarily application-oriented, with a focus on practical implementation and real-world problem solving within the domain of distributed computing fabrics and edge orchestration.

Application-Oriented: the core goal is a functional, deployable sandbox usable by researchers and engineers in Industry 4.0 smart manufacturing, 5G/6G telecommunications and the Internet of Vehicles, to automate and optimise job placement, scheduling and performance analytics. The platform is designed for immediate practical use, offering a dashboard, command-line tools and a REST API that safely simulate closed-loop edge emulation without risking live production infrastructure.

Domain-Specific:

- *Edge Computing and 5G/6G Networks:* RAP twin safely manages and orchestrates heterogeneous workloads, addressing siloed telemetry, resource constraints and time-sensitive task offloading.
- *Chaos Engineering and CI:* deterministic fault injection, Monte-Carlo tooling and a CI resilience gate validate policy resilience and SLA hit rates under simulated uncertainty.
- *Artificial Intelligence:* RAP twin applies Markov decision processes, bandit learning and tabular reinforcement learning to resource allocation, and turns raw telemetry into predictive insight.

Research-Oriented: the project also contains research elements in translating theoretical scheduling models into a practical engineering environment, and in characterising honestly where a sophisticated policy under-performs a simpler one on a non-latency metric. Examples reported in Chapter 7 are the Resilient planner's much higher egress bill and the observation that five strategies make identical placements on the over-provisioned reference fabric.

CHAPTER 4
PROJECT PLAN

4.1 Project Timeline

The project was planned over the academic year 2025–2026 as seven overlapping phases, shown in Figure 4.1. Table 4.2 states what each phase produced.

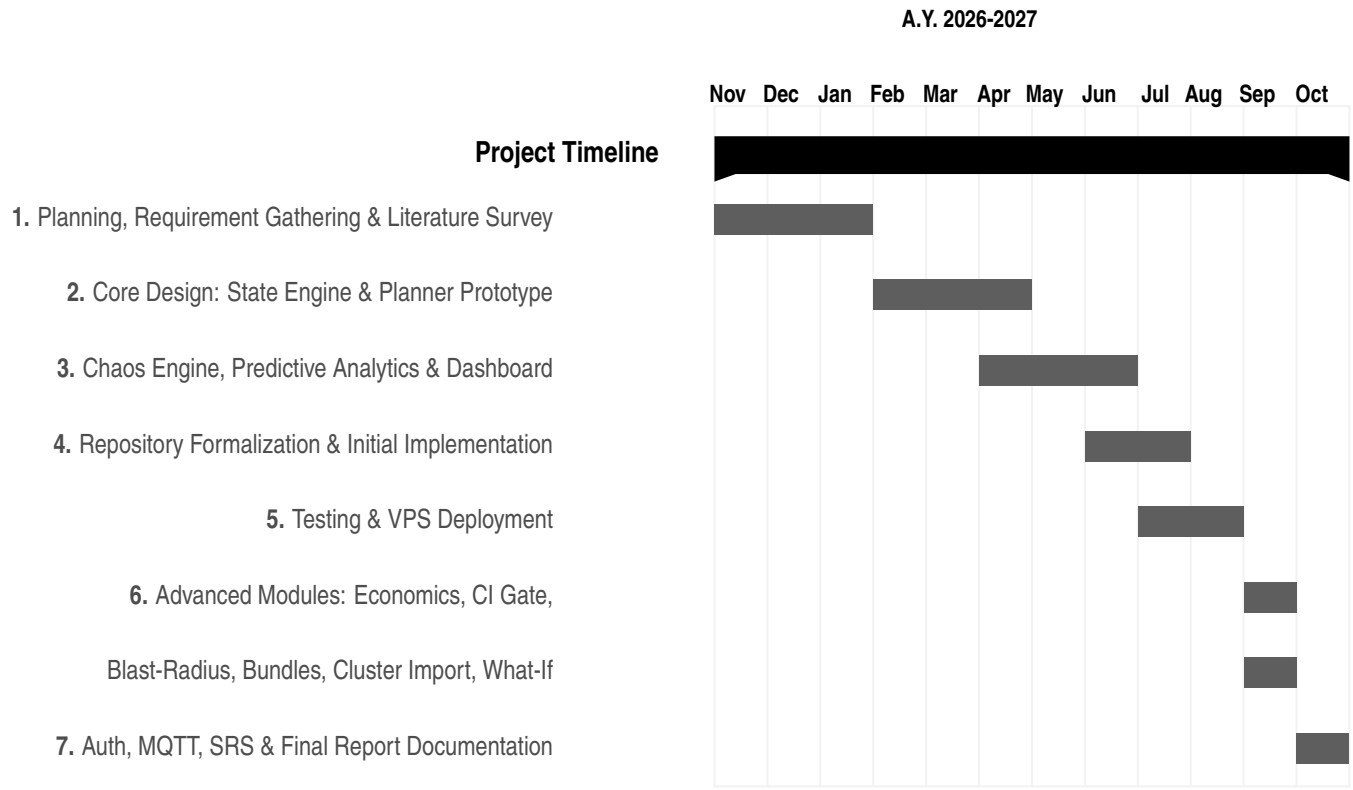


Table 4.1: Project Timeline

Table 4.2: Project phases and deliverables

Phase	Months	Activities	Deliverables
1	Nov–Jan	Problem selection, requirement gathering, literature survey, choice of technology stack	Problem statement, survey of 13 references, first SRS outline
2	Feb–Apr	State engine, cost model, baseline (Greedy) planner prototype	`dt/state.py`, `dt/cost_model.py`, `dt/policy/greedy.py`
3	Apr–Jun	Chaos engine, predictive analytics, first dashboard	`sim/chaos.py`, `dt/predict.py`, React dashboard
4	Jun–Jul	Repository formalisation, schemas, planners (Resilient, RL-Markov, Bandit)	Tests, schemas, planner factory
5	Jul–Aug	Testing and deployment to a public server	Test suite, Caddy and systemd deployment
6	Sep	Advanced modules: economics, CI gate, blast radius, bundles, cluster import, what-if	`dt/economics.py`, `tools/ci_gate.py`, `sim/blast_radius.py`, `tools/bundle.py`, `tools/import_cluster.py`, `dt/whatif.py`
7	Oct	Authentication, MQTT ingestion, GPU launcher support, SRS, documentation, final report	FR-28, FR-29, FR-31, SRS v1.2, Docs tab, this report

4.1.1 Development History

The version-control history of the project is an independent record of how the work progressed. The repository contains 89 commits, all by the project lead, in four bursts: 58 commits between 7 and 10 November 2025 (the first simulator, the Docker launcher, the federation-aware and bandit planners and the first dashboard), 2 commits in July and August 2026 (repository consolidation), 13 commits in September 2026 (React front end, deployment tooling, management screens, economics, CI gate, blast-radius finder, cluster import, bundles and what-if planning) and 16 commits in October 2026 (authentication, MQTT and GPU support, measured benchmark table, theme support, in-app documentation and report). Table 4.3 lists the milestones that matter most to the design.

Table 4.3: Selected milestones from the Git history

Date	**Milestone**
2025-11-08	Federation-aware planner; bandit strategy and policy benchmarking tool; topology visualisation
2025-11-09	Node hot-reload; pseudo-autonomous controllers and synthetic workloads
2026-09-16	React front end; chaos state-leak fixes; updates streamed over SSE; deployment tooling for the public server
2026-09-18	Cost and carbon for every plan; chaos-as-CI gate; blast-radius finder and predictive-state fix; cluster import and incident bundles; what-if planning
2026-10-08	API-key authentication (FR-31); GPU device requests in the launcher (FR-28); MQTT sidecar (FR-29); measured benchmark table replaces an unsupported one
2026-10-11	Dark/light theme; per-stage CPU request shown in plan detail
2026-10-14	17-page documentation tab inside the dashboard

4.2 Risk Management

Table 4.4 records the project risks that were identified and the mitigation that was applied.

Table 4.4: Risk register

Risk	Likelihood	Mitigation
Reported results not reproducible from the code	High (found)	The unsupported table was replaced by a script-generated one (`tools/benchmark_srs_table.py`); every figure in Chapter 7 comes from a committed script
A scheduling change silently degrades resilience	Medium	Chaos-as-CI gate with a committed baseline (`ci/baseline.json`) run by GitHub Actions
Chaos state leaks between experiments	Medium (found)	Predictor checkpoint and restore around blast-radius trials; fresh `DTState` per gate run
Unauthorised changes to a public deployment	Medium	Optional `X-API-Key` on the ten mutating routes; single loopback-bound worker behind a TLS proxy
Scope growth beyond the semester	High	Strict ordering of phases; advanced modules added only after the core was tested
Single point of knowledge	Medium	In-app documentation, README files, worklog and this report
Fidelity gap between twin and real hardware	High	Declared as an assumption and as a limitation (Section 7.5); validation against real containers listed as future work

4.3 Mentor Interaction

An interaction with the industry/alumni mentor, Sahil Badgujar, was held online on 11 October 2026 and recorded in the Industry/Alumni Mentor Interaction Record. The mentor reviewed the dashboard and made three suggestions: (i) minor user-interface changes, (ii) expanded documentation, and (iii) a recommendation feature that helps a user choose the most suitable scheduling algorithm for a workload. Table 4.5 shows how each suggestion has been handled.

Table 4.5: Mentor suggestions and their status

Suggestion	Action taken	Status
Minor UI changes	Dark/light theme with a toggle, responsive phone navigation (menu button and drop-down), a hint that explains why dry-run plans do not change CPU usage, and per-stage CPU shown in plan detail	Done
Expanded documentation	17-page in-app Docs tab (architecture, API reference, configuration, deployment, troubleshooting), Markdown documentation, and this report	Done
Algorithm recommendation system	Not yet implemented; the design is outlined in Section 8.2	Planned

4.4 Team Organization

The project is carried out by a group of three team members, each assigned a specific role to ensure effective collaboration and clear reporting.

4.4.1 Team Structure

1. Riaan Masoud Attar – Digital Twin API & Platform Infrastructure. Responsible for the core digital-twin infrastructure, state management and platform-level concerns that every other module depends on.

- Digital Twin API (Flask) backend development across all 29 routes.
- Unified state machine (`dt.state.DTState`) implementation and hot reloading.
- Live telemetry ingestion (`/observe` endpoint) and the MQTT telemetry sidecar.
- Background self-healing controllers (`SelfHealingController` and `ResourceGuardian`).
- REST API security (`FABRIC_API_KEY` on mutating routes), interoperability exporters (DTDL, Kubernetes CRDs) and the importer.
- Incident bundle export and import, the dashboard's data layer, and deployment.

2. Avanti Nitin Patil – Intelligent Planning & AI Scheduling. Responsible for the multi-strategy planning engine, workload constraint parsing and the advanced AI scheduling policies.

- Multi-strategy planning engine and the shared planner factory (`dt/planners.py`).
- Federated/Resilient planner and RL-Markov (MDP) policy implementation.
- Cost- and carbon-aware strategies (Cheapest-Cost, Greenest) built on `dt/economics.py`.
- Planner CLI development and the dry-run and batch-planning evaluator.
- Job descriptor parsing, SLA constraints and schema validation (YAML/JSON).
- What-if capacity planning (`dt/whatif.py`).

3. Preet Prashant Boralkar – Simulation, Chaos Engineering & Analytics. Responsible for evaluating the system under real-world uncertainty, large-scale benchmarking and resilience validation.

- Chaos engine development (deterministic fault and noise injection).
- Chaos-as-CI resilience gate (`tools/ci_gate.py`) and baseline regression checks.
- Blast-radius finder (`sim/blast_radius.py`) for minimal-fault-set search.
- Monte-Carlo simulation tooling and orchestration.
- Predictive analytics and failure forecasting (`PredictiveAnalyzer`).
- Statistical policy evaluation and benchmark reporting (Section 7.1.2).

CHAPTER 5

SOFTWARE REQUIREMENT SPECIFICATION

This chapter summarises the requirements captured in full detail in the project's standalone Software Requirements Specification (SRS, file `RAPtwin_SRS_v1.2.docx`). The SRS enumerates 31 individually status-tagged functional requirements (FR-01 to FR-31), ten use cases, the non-functional requirements and a data model. The status given in the SRS is repeated below and, where a code review for this report found that the implementation differs from the SRS wording, the difference is stated in a remarks column rather than hidden.

5.1 Product Perspective

RAP twin is a self-contained software product. It does not replace an orchestrator; it sits beside one. It receives a description of a fabric (node and topology descriptors, optionally imported from Kubernetes), receives live observations (REST or MQTT), accepts job descriptors from a person or a script, and returns placements together with predictions. Its external interfaces are: a REST and Server-Sent-Events API, a web dashboard, a set of command-line tools, and file formats (YAML descriptors, JSON reports, ZIP incident bundles, DTDL and CRD documents).

5.1.1 User Classes

Table 5.1: User classes

User	**Characteristics**	**Typical use**
Fabric operator / researcher (primary)	Understands scheduling, SLAs and fault tolerance; comfortable with YAML and REST	Submits jobs, starts chaos scenarios, reviews the dashboard, runs sweeps
SRE / platform engineer	Operates real fabrics and wants to validate a policy change before rollout	Dry-run planning, what-if analysis, CI gate
Student / research user	Basic ML and distributed-systems knowledge; wants to compare policies	Benchmarks RL-Markov against heuristic planners
Developer / maintainer	Knows Python, REST and the pluggable planner interface	Adds strategies, extends the chaos engine, runs the test suite

5.1.2 Use Cases

Table 5.2: Use-case summary (full reports are in the SRS)

ID	Use case	Description
UC1	Load / hot-reload topology	Operator edits node or topology YAML; the system detects and ingests the change
UC2	Dry-run planning	User submits a job with `dry_run=true` and receives predicted latency, energy and SLA risk without a reservation
UC3	Committed batch planning	User submits several jobs through `/plan_batch` for committed placement
UC4	Observe live telemetry	Telemetry is posted to `/observe` and merged into the twin
UC5	Inject chaos scenario	Operator starts a deterministic fault schedule through `/chaos/start`
UC6	Self-healing re-route	The controller promotes a fallback when a primary degrades (system-triggered)
UC7	Reclaim expired reservations	The guardian releases stale reservations (system-triggered)
UC8	Monte-Carlo sweep	Operator runs many randomised trials to estimate a policy's SLA compliance
UC9	What-if capacity change	Operator forks the twin, applies a hardware change and compares outcomes
UC10	Export to DTDL / CRD	An external control plane consumes an exported twin description

5.2 Functional Requirements

The functional requirements are grouped in Table 5.3. "Realised in" names the module that implements the requirement; "Status" is the SRS status; "Remarks" records the result of the code review carried out for this report.

Table 5.3: Functional requirements, implementation and status

ID	Requirement (abridged)	Realised in	Status	Remarks
FR-01	Parse node and topology YAML at start-up	`dt/state.py`	Implemented	
FR-02	Hot-reload descriptors by background poll (∼0.5 s)	`DTState._watch_loop`	Implemented	
FR-03	Merge telemetry through `POST/observe`	`apply_observation`	Implemented	

ID	Requirement (abridged)	Realised in	Status	Remarks
FR-04	Validate node, topology and job payloads against JSON Schema	`dt/validators.py`, `tools/validate_nodes.py`	Implemented	Partial: not wired into the API (Sec. 7.5)
FR-05	Track estimation deviation Δf between twin and physical capacity	—	Implemented	Not found in code; see Sec. 7.5
FR-06	Forecast utilisation, thermal derating and link-loss variability	`dt/predict.py`	Implemented	Forecast accuracy not scored
FR-07	Pluggable planner interface	`dt/planners.py`, `dt/policy/`	Implemented	
FR-08	Greedy baseline planner	`policy/greedy.py`	Implemented	
FR-09	Cheapest-Energy planner	`GreedyPlanner` (energy weight)	Implemented	
FR-10	Resilient/Federated planner with fallbacks	`policy/resilient.py`	Implemented	
FR-11	RL-Markov planner (MDP)	`policy/mdp.py`	Implemented	
FR-12	ε-greedy / TD(0) RL policy and UCB1 bandit	`rl_stub.py`, `bandit.py`	Implemented	`choose_node()` unused by planners
FR-13	Dry-run evaluation mode	all planners	Implemented	
FR-14	Batch planning (`/plan_batch`)	`dt/api.py`	Implemented	
FR-15	Deterministic, time-compressed chaos scenarios	`sim/chaos.py`	Implemented	14 event kinds
FR-16	Start/stop chaos through REST	`dt/chaos_runner.py`	Implemented	
FR-17	Detect degraded nodes and reroute reservations	`SelfHealingController`	Implemented	
FR-18	Reclaim expired reservations	`ResourceGuardian`	Implemented	
FR-19	Monte-Carlo sweeps with CSV/JSON reports	`sim/montecarlo.py`	Implemented	CSV and console summary
FR-20	What-if analysis without mutating the live twin	`dt/whatif.py`	Implemented	
FR-21	Blast radius of a node/link failure	`sim/blast_radius.py`	Implemented	
FR-22	Export to Azure DTDL	`dt/exporters.py`	Implemented	
FR-23	Export to Kubernetes CRDs	`dt/exporters.py`	Implemented	
FR-24	CloudEvents on an internal event bus	`dt/events.py`	Implemented	
FR-25	Real-time dashboard updated over SSE	`web/src`, `/stream`	Implemented	
FR-26	Dynamic topology editing through REST	`/add_node`, `/links`	Implemented	
FR-27	Containerised nodes that mirror `nodes/*.yaml`	`fabric_docker/`	Implemented	
FR-28	GPU resource requests in the launcher	`launch_fabric.py`	Implemented	CPU/RAM limits not applied
FR-29	Event-driven telemetry (MQTT)	`sim/mqtt_bridge.py`	Implemented	

ID	Requirement (abridged)	Realised in	Status	Remarks
FR-30	Live integration with a 5G/6G hardware test-bed	—	Planned	Future work
FR-31	API authentication	`dt/api.py`	Implemented	Optional, via `FABRIC_API_KEY`

Of the 31 requirements, 30 are recorded as implemented and one (FR-30) as planned. The code review identified two requirements whose implementation is incomplete relative to the SRS wording (FR-04 and FR-05) and three with qualifications (FR-12, FR-19, FR-28). These are carried to the limitations in Section 7.5 and to the future work in Section 8.2.

5.3 Non-Functional Requirements

- **Performance:** the SRS requires that baseline heuristic planners return a placement decision in under 5 ms per job, that a 1,000-trial Monte-Carlo sweep completes within a practical benchmarking session, that descriptor changes are picked up within one poll interval (∼0.5 s), and that the dashboard reflects a chaos-triggered re-route without a manual refresh. The measured planning time is reported in Section 7.2.1; it exceeds the 5 ms target.
- **Reliability:** a single degraded node or link must not cause an unhandled exception; the self-healing controller re-routes before the SLA deadline where a fallback exists; a failed Monte-Carlo trial must not abort the sweep.
- **Security:** mutating REST routes require an `X-API-Key` header matching `FABRIC_API_KEY` when the variable is set; node and job names are validated to reject path-traversal and injection characters; bundle import rejects archives that would write outside the target directory.
- **Observability:** state changes are published as CloudEvents on an internal bus and streamed to the dashboard over Server-Sent Events (`/stream`, `/events`).
- **Maintainability:** new strategies are added by implementing the planner interface under `dt/policy/` without modifying the state engine or the API; code quality is enforced by black, isort and ruff, and correctness by the pytest suite.
- **Usability:** the dashboard presents topology, node health and job history without requiring the operator to read raw JSON; API errors use suitable HTTP status codes with a clear message.
- **Portability:** the back end is pure Python and the front end a standard Vite/React build; both run on Linux, macOS and Windows without platform-specific dependencies beyond optional Docker support.

5.4 Constraints

- RAP twin runs as a single process on one workstation or server; it simulates a distributed fabric but is not itself horizontally distributed. The deployed service therefore uses exactly one worker process, because the twin state (reservations, overrides, event bus) lives in that process.
- The file-watcher hot reload introduces a bounded latency (∼0.5 s) between an on-disk change and the state engine picking it up.

- Persistent state is file-based (YAML and JSON descriptors, an in-memory state object); there is no relational database.
- Background controllers run as threads, so Python's global interpreter lock bounds their concurrency.
- A job is a sequential pipeline of stages with CPU, memory and accelerator-memory requirements; arbitrary non-sequential workload graphs are not supported by the cost model.

5.5 Hardware Requirements

Client side: any workstation, tablet or phone with a modern web browser (Chrome, Firefox, Edge) and network access to the server.

Server side: a standard x86_64 workstation or server with at least 4 CPU cores and 4 GB RAM. As recorded in the SRS, development and benchmarking were carried out on an Intel Core i7-13700K workstation with an NVIDIA RTX 4090 GPU; the GPU is tracked as a schedulable resource and is not required for the core simulation, chaos or dashboard functionality, which is CPU-bound. The additional measurements added for this report (Chapter 7) were taken on a laptop with an Intel Core i5-1240P (16 hardware threads) and 15 GB of RAM, and that configuration is stated next to every result that depends on it.

Deployment: the public instance runs on an Ubuntu virtual server behind a Caddy reverse proxy (Figure 6.2).

5.6 Software Requirements

- Python 3 with Flask ($\geq$3.0), PyYAML, jsonschema, requests, NumPy, Pandas, Matplotlib and rich. The SRS specifies Python 3.12 and the CI workflows run 3.12; the measurements in this report ran on Python 3.14.7.
- gunicorn for the deployed service; pytest, black, isort and ruff for testing and code quality.
- React 19, TypeScript, Vite, Tailwind CSS 4, D3.js and TanStack Query for the dashboard.
- Docker (optional, for the containerised fabric launcher) and an MQTT broker with `paho-mqtt` (optional, for the telemetry sidecar; `requirements-mqtt.txt`).
- Git and GitHub Actions for version control and continuous integration.

5.7 Interfaces

RAP twin exposes 29 route-and-method combinations on 24 distinct paths. All requests and responses use JSON (except bundle download and the SSE stream). Table 5.4 groups them by function; the ten routes marked with an asterisk are the *mutating* routes that require the API key when authentication is enabled.

Table 5.4: REST and SSE interface

Function	Routes	Purpose
State and telemetry	GET `/health`, `/snapshot`, `/topology`, `/nodes/<name>`; POST `/observe`	Liveness, unified twin state, declared links, node descriptor, telemetry merge
Planning	POST `/plan`, `/plan_batch`, `/release`; GET `/plans`, `/jobs`, `/economics`	Plan one or many jobs, release reservations, plan history, job catalogue, pricing in force
Catalogue editing	POST `/add_node`*, `/links`*, `/jobs`*; DELETE `/nodes/<name>`*, `/links`*, `/jobs`*, `/plans`*	Create, edit and delete nodes, links, jobs and plan history
Chaos and resilience	GET `/chaos`; POST `/chaos/start`*, `/chaos/stop`*, `/overrides/reset`*, `/whatif`, `/blast_radius`	Scenario catalogue and control, fault reset, capacity what-if, minimal-fault search
Interoperability	GET `/standards/dtdl`, `/standards/crds`, `/bundle`	DTDL and CRD export, incident bundle download
Live updates	GET `/stream`, `/events`	SSE feed of CloudEvents; recent events

The frontend communicates with the back end exclusively through this surface, and every route returns a JSON envelope of the form {`"ok": true, "data": ...`} or {`"ok": false, "error": ...`}.

5.8 Requirements Traceability

Table 5.5 links the main requirement groups to the automated tests that exercise them, so that a reader can see which requirements are protected from regression. The complete list of tests is given in Section 7.4.

Table 5.5: Traceability of requirement groups to tests

Requirements	**Test file**	**What the tests assert**
FR-03, FR-14, FR-13	`test_api.py`	Plan responses carry rich per-stage metrics; batch planning reuses plan logic and propagates errors
FR-31	`test_api_auth.py`	Every gated route returns 401 without the key; read-only and evaluation routes stay open; the SSE stream is not gated
FR-26, FR-02	`test_management.py`	Node, link and job CRUD; path-traversal names rejected; a removed node is not resurrected by the watcher; fault reset
FR-21	`test_blast_radius.py`	Reported fault sets are minimal; trials are independent; the fabric is restored after a search
FR-20	`test_whatif.py`	Fork-and-diff correctness; nothing persisted; bad change sets rejected clearly
Economics	`test_economics.py`	Hand-calculated cost and carbon; egress only across sites; zone overrides
Gate (Obj. 4)	`test_ci_gate.py`	Determinism; threshold and baseline regression failures; shipped config matches the baseline
FR-22, FR-23 (inverse)	`test_import_cluster.py`	Kubernetes quantity parsing; capacity, GPU and readiness mapping; written descriptors load into the twin
Bundles	`test_bundle.py`	Round trip; refuses to clobber a non-empty directory; rejects path traversal
FR-27, FR-28	`test_launch_fabric.py`	GPU device requests only for GPU nodes; no side effects
FR-29	`test_mqtt_bridge.py`	Topic and payload parsing; malformed input never raises or posts
Benchmarks	`test_benchmark_*.py`	Unit conversion and rendering of the benchmark table; no fabricated baselines

CHAPTER 6
DETAILED DESIGN

6.1 Architectural Design

Fig. 6.1 shows the architecture of RAP twin. Static descriptors and streaming telemetry are unified by the state engine (`dt.state.DTState`), which exposes a state-query interface to the predictive-analytics component and to the planner factory (seven strategy names backed by three planner classes and two learning components). The chaos and guardrail layer and the economics module sit alongside the planners, injecting faults and pricing every plan respectively. Beneath them, four tool layers – analysis tools (Monte-Carlo, what-if, blast radius), the CI resilience gate, the interoperability layer (DTDL/CRD export, cluster import) and incident bundles – consume and produce twin state without ever changing the live fabric except through an explicit plan commit. An MQTT sidecar and the REST API form the ingestion and egress boundary, and the React dashboard is the primary human interface.

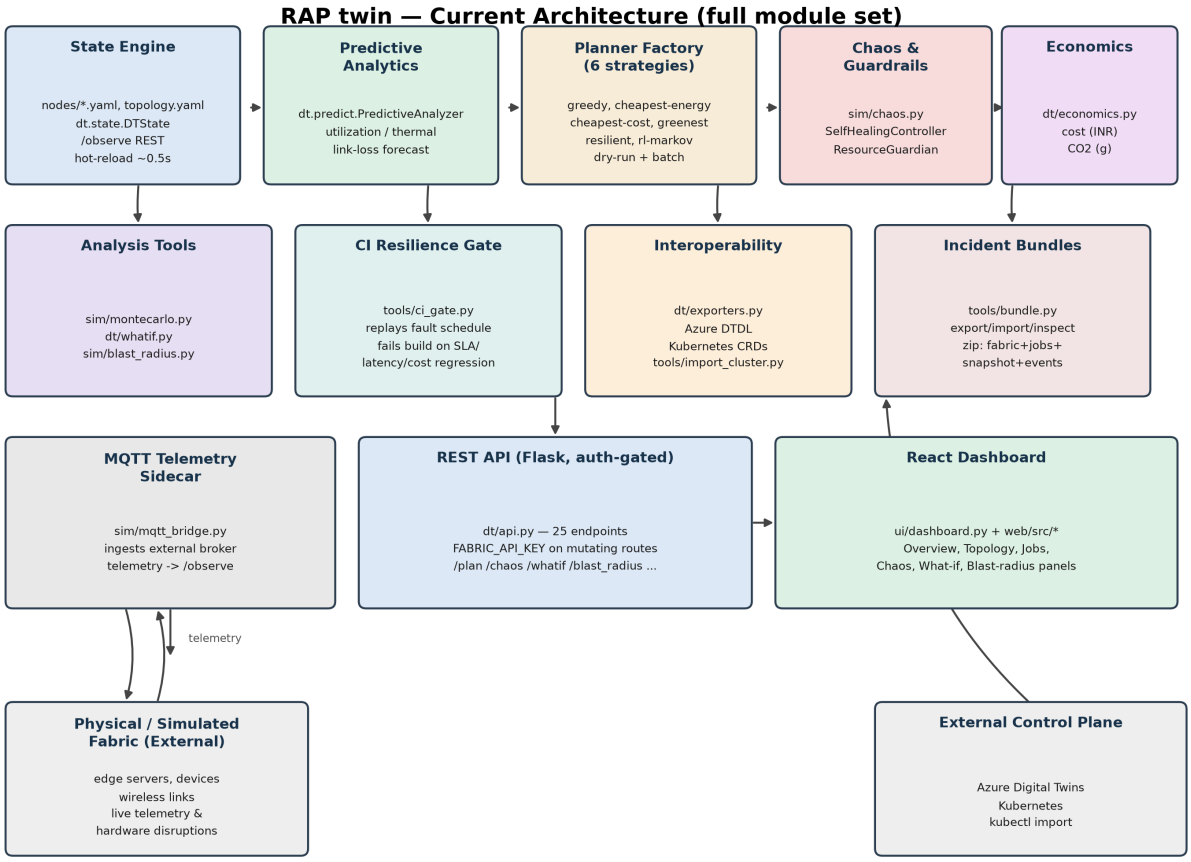


Figure 6.1: RAP twin – Current Architecture (full module set)

6.1.1 Layered View

The software is organised in five layers. Each layer depends only on the layers below it, which is what allows the CI gate, the blast-radius search and the what-if planner to reuse the state engine

and the planners without going through the HTTP API.

Table 6.1: Layers of the implementation

Layer	**Responsibility**	**Main files**
Presentation	REST/SSE API, web dashboard, command-line planners	`dt/api.py`, `web/src/`, `planner/`, `ui/`
Orchestration	Planner factory, chaos runner, self-healing and guardian controllers	`dt/planners.py`, `dt/chaos_runner.py`, `dt/self_heal.py`
Decision	Planners, cost model, economics, predictor	`dt/policy/*.py`, `dt/cost_model.py`, `dt/economics.py`, `dt/predict.py`
State	Unified twin state, event bus, validators	`dt/state.py`, `dt/events.py`, `dt/validators.py`
Data	Descriptors, topology, overrides, job library, reports	`nodes/`, `sim/topology.yaml`, `sim/overrides.json`, `jobs/`

6.1.2 Execution and Concurrency Model

The whole twin runs inside one Python process. The Flask application constructs the singletons at import time – the state engine, the cost model, the planners, the controllers and the chaos runner – and shares them between request-handling threads. Table 6.2 lists the background threads. All state mutation is guarded by a re-entrant lock in `DTState`; planners read through `nodes_for_planner()`, which returns deep copies, so a plan in progress never sees a half-applied update.

Table 6.2: Background threads in the API process

Thread	**Period**	**Work**
`DTStateWatch`	0.5 s	Re-reads `overrides.json`, `topology.yaml` and changed node files; merges them without losing runtime state
`SelfHealing`	2.5 s	Checks the primary node of every committed stage and promotes a fallback when it degrades
`ResourceGuardian`	5 s	Releases the oldest expired reservation on any node that is nearly full
`ChaosRunner`	per run	Applies a scenario's events at accelerated virtual time
`NoiseInjector`	7.5 s	Optional (off by default): random small perturbations of one node and one link
HTTP workers	on demand	Request handling; each open SSE stream occupies one thread for its lifetime

6.1.3 Deployment Architecture

The public instance (Figure 6.2) runs on an Ubuntu virtual server. Caddy terminates TLS with an automatically issued certificate, serves the compiled single-page application from `web/dist`, and proxies `/api/*` to a gunicorn process bound to the loopback interface. The proxy strips the `/api` prefix, which is why the same front-end build works against the development server, and disables response buffering on that path so that Server-Sent Events arrive immediately. The service runs a *single* gunicorn worker with 24 threads, because the twin state lives in process memory: a second worker would hold a divergent copy. A shell script (`deploy/deploy.sh`) synchronises the code, rebuilds the front end and restarts the systemd unit. For local development, `run.sh` starts the API and the Vite development server together.

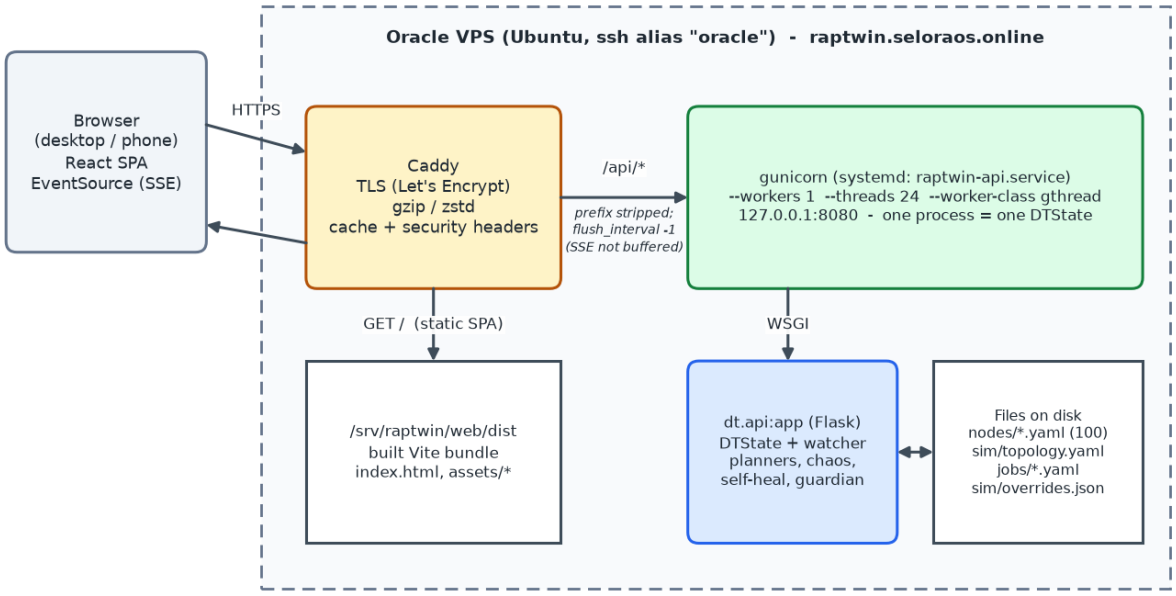


Figure 6.2: Deployment architecture of the public instance

6.1.4 Key Design Decisions

Table 6.3: Design decisions and their rationale

Decision	**Alternative**	**Rationale**
Files instead of a database	Relational or document store	Descriptors are human-edited and version-controlled; no server to deploy; the twin is rebuilt from files in milliseconds
In-process state, one worker	Shared store, many workers	Reservations and overrides must be consistent; a single process makes that trivially true
Dry-run as the default in the UI	Commit by default	A planner experiment should not consume capacity unless the user asks for it
Pluggable planner interface	Single hard-wired policy	The research question is the comparison of policies on the same fabric
Additive millisecond score in the resilient planner	Multiplicative score	Every term is in a common unit, so each weight reads as "how many milliseconds is this worth" and can be tuned independently
CloudEvents on an internal bus	Polling only	Clients refresh only the data an event affects; a standard envelope eases integration
Deterministic chaos with seeds	Random fault injection	A policy change can be accepted or rejected in CI only if the experiment is repeatable

6.2 Overview of Project Modules

The system consists of thirteen modules. Table 6.4 lists them with their source files; the subsections that follow describe each one: its purpose, its inputs and outputs, the parameters that govern it, and how it interacts with the others.

Table 6.4: Module inventory

#	Module	Source files
1	State engine	`dt/state.py`, `dt/validators.py`
2	Predictive analytics	`dt/predict.py`
3	Cost model	`dt/cost_model.py`
4	Economics	`dt/economics.py`
5	Planner factory and strategies	`dt/planners.py`, `dt/policy/greedy.py`, `resilient.py`, `mdp.py`, `bandit.py`, `rl_stub.py`
6	Chaos engine	`sim/chaos.py`, `dt/chaos_runner.py`, `dt/noise.py`
7	Self-healing and guardian	`dt/self_heal.py`
8	CI resilience gate	`tools/ci_gate.py`, `ci/gate.yaml`, `ci/baseline.json`
9	Blast-radius finder	`sim/blast_radius.py`
10	What-if planner	`dt/whatif.py`
11	Monte-Carlo orchestrator	`sim/montecarlo.py`
12	Interoperability, bundles and MQTT	`dt/exporters.py`, `tools/import_cluster.py`, `tools/bundle.py`, `sim/mqtt_bridge.py`
13	API, events and dashboard	`dt/api.py`, `dt/events.py`, `web/src/`

6.2.1 State Engine

Purpose. `DTState` is the single source of truth about the fabric. It combines three layers of information for every node and link:

- *base* – the static descriptor loaded from `nodes/<name>.yaml` or from the link entry in `topology.yaml`, plus capacities derived from it (`caps`: CPU units = cores × base GHz, core count, RAM, VRAM);
- *dyn* – the mutable runtime overlay (`NodeDyn` or `LinkDyn`): down flag, thermal derate, power cap, clock skew, packet duplication and reordering, used CPU/memory/VRAM, reliability, availability window, battery, forecasts and the dictionary of active reservations;
- *effective* – the view that planners read, computed on demand: free CPU is $\max(0, \text{cores} \times (1 - \text{derate}) - \text{used})$, free memory and VRAM are capacity minus use, and a link's metrics are taken from its dyn override, else its base value, else the topology default.

Inputs and loading. At start-up the engine loads the 100 node descriptors, the topology (defaults, links, pricing, energy) and any fault overrides. A watcher thread polls every 0.5 s: it reloads overrides when the file's modification time changes, reloads the topology the same way, and reloads node files when their set of modification times changes, preserving each node's runtime state across the reload. Overrides are applied as a *difference* against the previously applied set, so a field that disappears from the file is reset to its default instead of staying degraded.

Telemetry. `apply_observation` accepts `apply` or `revert` actions for a node or a link. For a node, a known field is overwritten; a revert resets the named fields to their defaults. For a link, an unknown key creates an ad-hoc ("synthetic") link so that chaos events between two zones can carry metrics, and such a link is deleted once everything on it has been reverted. After each node update the predictor is called and the node's forecast fields are refreshed; after each link update the link forecast is refreshed.

Reservations. `reserve(request)` succeeds only if the node exists, is not down and has enough free CPU, memory and VRAM; it increments the `used_*` counters, stores the reservation (identifier `res-0000001` and so on, with a timestamp) in the node's `dyn.reservations`, refreshes the forecast and emits a CloudEvent. `release` reverses it.

Federations. A federation is derived from node labels in the order `federation`, `zone`, `site`, `rack`, `region`, defaulting to `global`. For each federation the engine aggregates total and free CPU, memory and VRAM, the number of down nodes and of "hot" nodes (thermal derate ≥ 0.25), the reservation count, mean trust and mean loss, and three ratios used by the resilient planner: *load factor* (used CPU share), *down fraction* and *hot fraction*. Links between nodes of different federations are bucketed into federation links with the minimum speed, maximum loss and mean RTT.

Outputs. `snapshot()` returns a cached, thread-safe JSON-ready view (nodes, links, federations, predictive overview, economics defaults); the cache is invalidated whenever the state changes. `nodes_for_planner()` returns deep copies of every node with the effective and predictive blocks attached.

Validation. `dt/validators.py` offers JSON-Schema checking of nodes, jobs and topologies with non-throwing (`lint_*`) and throwing (`assert_*`) variants. The API itself applies lighter, hand-written checks (valid names, positive CPU and memory, well-formed formats) because a quarter of the generated node files do not satisfy the node schema; this is recorded in Section 7.5.

6.2.2 Predictive Analytics

Purpose. `PredictiveAnalyzer` turns raw measurements into forecasts that the cost model and the planners consume. It is deliberately lightweight (standard library only) so that it runs inside unit tests.

Estimators. Three small primitives are used. An *exponentially weighted moving average* (EWMA) with $\alpha = 0.35$ smooths utilisation, thermal derating, link latency, jitter and loss. A *rolling least-squares trend* over a window of 12 samples estimates the slope of utilisation or latency. A *reliability model* adjusts a node's base reliability for its mean time between failures and uptime and for its battery.

Node forecast. For a node with utilisation sample u (used cores divided by total cores), the analyser computes the smoothed utilisation $\bar{u}$, its slope s, and then

$$\begin{aligned} \text{util_forecast} &= \text{clamp}\left(\bar{u} + 30\,s,\ 0,\ 1\right), \\ \text{projected_derate} &= \text{EWMA}\left(\text{derate} + \max(0, s)\right), \\ \text{reliability} &= r_0\Big(1 - \tfrac{1}{2}\min\left(1, \tfrac{\text{uptime}}{\text{MTBF}}\right)\Big) \cdot b, \end{aligned}$$

where r_0 is the descriptor's reliability (default 0.95) and b is 0.5 when the battery will last less than one hour, 0.7 when less than two hours, and 1 otherwise. The *availability window* is either declared in the descriptor or, for battery-powered nodes, the remaining battery divided by the drain rate.

Link forecast. A link keeps EWMAs of latency, jitter and loss and an approximate 95th-percentile latency $\hat{p}_{95} = \bar{\ell} + 4\,|s_\ell| + 2\,\bar{j}$.

Checkpoint and restore. Forecasts are deliberately sticky: a node that ran hot keeps an elevated projected derate after the fault clears, as a real twin's history should. A search that needs independent trials cannot tolerate that, so the analyser can deep-copy all of its series (`checkpoint`) and put them back (`restore`); the blast-radius search wraps every trial in this pair. This fixed a defect found while the search was being built, in which a thermal derate in one trial depressed every later trial.

Known issues. Two implementation defects affect these forecasts and are described in Section 7.5: the link $\hat{p}_{95}$ read-out uses an accumulator instead of the slope, and the trend estimate is numerically degenerate when timestamps are epoch milliseconds.

6.2.3 Cost Model

Purpose. `CostModel` estimates, for a stage on a node, the compute time, the transfer time from the previous stage, the energy and a risk score, and aggregates them over a job. All planners use the same model, which is what makes their predictions comparable. The formulas are collected here; they are the basis of the algorithms in Section 6.7.

Compute time. The "work" of a stage is $W = 2\,s + 120\,c$ milliseconds-equivalent, where s is the data size in MB and c the requested CPU cores; it is multiplied by 0.85 for I/O-bound stages and floored at 15 ms. A node's capacity is $U = \text{cores} \times \text{GHz} \times (1 - d)$ for derate d (the larger of the descriptor, runtime and projected derate). Then

$$t_{\text{comp}} \;=\; \frac{W}{\max(1,\, U/10)\;\max(1,\, a)}\Big(1 + 0.35\,\text{util_forecast}\Big) \;+\; 6000\,(1 - r),$$

where a is the accelerator multiplier and r the node's reliability. The multiplier is $\min(6,\, 1 + \text{accel_score}/10)$ for a CUDA-capable node when CUDA is permitted, $\min(3,\, 1 + \text{TOPS}/10)$ for an NPU, and 1 otherwise. A node whose reliability is below 0.35, or whose usable availability window (the declared window minus a 15 s grace period, when positive) is shorter than the compute time, is treated as infeasible ($t = \infty$).

Transfer time. For a link with speed v Gbps, RTT, jitter and loss percentage λ, the effective throughput is $v \cdot 1000 \cdot 0.85 \cdot (1 - \min(0.3, \lambda/100))$ Mbps and

$$t_{\text{xfer}} \;=\; \frac{8\,s}{\text{throughput}} \cdot 1000 \;+\; \text{RTT} \;+\; \text{jitter}.$$

A down link, or a forecast loss of 30 % or more, gives $t = \infty$. When a node pair has no declared link, the engine estimates one from the nodes' own network hints (the slower of the two speeds, the larger of the two latencies and loss rates) and the topology defaults.

Energy. With thermal design power P (default 65 W), idle fraction 0.12, and utilisation proxy $u = \text{clamp}(c/\text{cores},\, 0.05,\, 1)$ inflated by the derate and the forecast, the power is $P\big(0.12 + 0.88\,u^{0.85}\big)$ and the energy of a stage is that power multiplied by its compute time, in kilojoules.

Risk. Risk is a weighted blend in $[0, 1]$:

$$\rho = 0.25\,(1-\text{trust}) + 0.15\,\text{ssd_wear} + 0.15\,\min\!\left(1, \tfrac{\text{crashes}}{5}\right) + 0.15\,\text{derate} + 0.10\,\min\!\left(1, \tfrac{\lambda}{5}\right) + 0.20\,(1-r) + 0.15\,a_w,$$

where $a_w = \max(0,\, 120 - \text{window})/120$ is the shortage of availability window.

Job cost and SLO. For a sequential job, latency is the sum over stages of compute time plus the transfer time from the previous stage's node; energy is the sum of stage energies; risk is their mean. The SLO penalty is zero when the latency is within the deadline D and otherwise $\big((L/D)^{1.2} - 1\big)/0.002$.

6.2.4 Economics

Purpose. Operators budget money and, increasingly, emissions, so every plan is priced. `Economics` reads the topology defaults (currency, per-hour rates, price per kWh, grid carbon intensity) and per-zone overrides.

Formulas. For a stage that runs for t hours on c cores and m GB with accelerator rates, and consumes E kWh:

$$\begin{aligned} \text{compute} &= t\,(c \cdot r_{\text{cpu}} + m \cdot r_{\text{mem}}) + t\,r_{\text{gpu or npu}}, \\ \text{electricity} &= E \cdot p_{\text{kWh}}(\text{zone}), \qquad \text{CO}_2 = E \cdot g(\text{zone}), \\ \text{egress} &= \tfrac{\text{size}_{\text{MB}}}{1024} \cdot r_{\text{egress}} \quad \text{if the previous stage ran on a different site (zone).} \end{aligned}$$

The reference topology uses INR with $r_{\text{cpu}} = 2$ per core-hour, $r_{\text{gpu}} = 30$ per GPU-hour, $r_{\text{npu}} = 6$, $r_{\text{mem}} = 0.8$ per GB-hour, $r_{\text{egress}} = 1$ per GB, $p_{\text{kWh}} = 8$ and $g = 720\,\text{g}\,\text{CO}_2/\text{kWh}$. `annotate_plan` adds `cost`, `cost_breakdown` and `co2_g` to every stage and `cost_total`, `co2_g` and `energy_kwh` to the plan, for *every* strategy.

Rates versus absolute cost. A stage that runs for 300 ms rents hardware for 300 ms, so its absolute cost is almost zero on every node and cannot discriminate between candidates at any weight. The cost-aware planners therefore score candidates on the *per-hour rate* of the placement (`stage_rate`) and add cross-site egress valued at 1000 ms per currency unit; the absolute cost is what is reported on the finished plan.

6.2.5 Planner Factory and Strategies

Purpose. `dt/planners.py` maps any accepted strategy name to a planner, so that the API, the CI gate, the blast-radius search and the what-if planner all describe the same policy by the same name. An alias table resolves synonyms (for example `energy`, `cost-aware`, `carbon`, `mdp`, `ft`, `balanced`) to seven canonical names. `plan_once` calls a planner with `dry_run` and, for the resilient planner, the preset name.

Table 6.5: Strategies exposed by the planner factory

Strategy	Class	Configuration and objective
`greedy`	`GreedyPlanner`	Minimise compute + transfer + 10·risk + reliability and churn penalties; locality bonus 0.5 ms
`cheapest-energy`	`GreedyPlanner`	As `greedy`, plus 0.1 ms per kJ
`cheapest-cost`	`GreedyPlanner`	As `greedy`, plus 25 ms per currency unit per hour (`FABRIC_COST_WEIGHT`)
`greenest`	`GreedyPlanner`	As `greedy`, plus 25 ms per g CO_2 per hour (`FABRIC_CARBON_WEIGHT`)
`bandit`	`GreedyPlanner` + `BanditPolicy`	Greedy with a UCB1 bandit that learns the execution format for each (stage signature, node)
`resilient`	`FederatedPlanner`	Additive millisecond score with load, spread, network, resilience, reliability and availability terms; fallbacks and shadow reservations; presets `resilient`, `network-aware`, `federated`
`rl-markov`	`MarkovPlanner`	Finite-horizon MDP solved by dynamic programming with a weighted Tchebycheff objective, $\gamma = 0.92$, failure penalty 10, redundancy 3

All planners honour the same hard constraints: a node must not be down, must have enough free CPU, memory and accelerator memory, and must support at least one of the stage's allowed formats. The algorithms are described in detail in Section 6.7. The dashboard's strategy menu offers ten entries because it lists the three presets of the resilient planner (*resilient*, *network aware*, *federated*) and the alias *balanced* separately.

6.2.6 Chaos Engine

Purpose. The chaos engine replays a schedule of faults against the twin, in accelerated virtual time. A schedule is read from `sim/topology.yaml`: a top-level `chaos:` list that always applies, plus the events of an optionally named scenario (`wifi_brownout`, `core_link_cut`). Each event has a start time `at_s` and, optionally, a `duration_s`; an event with a duration automatically receives a matching revert event.

Event kinds. Fourteen kinds are implemented (Table 6.6). Group events (`zone_blackout`, `federation_partition`) are expanded through a label index built from the node descriptors, so "zone = rackC" resolves to every node carrying that label.

Table 6.6: Chaos event kinds

Family	Kind	Effect on the twin
Link (4)	`link_degrade`	Lower speed; raise RTT, jitter or loss on one link
	`link_loss_spike`	Set the loss rate of one link
	`link_down`, `link_up`	Mark a link down; clear the down flag
Node (7)	`node_kill`, `node_recover`	Mark a node down; clear the flag
	`power_cap`	Limit a node's power in watts
	`thermal_derate`	Set a derating fraction in $[0, 1]$
	`clock_skew`	Set a clock offset in ms
	`packet_dup`, `packet_reorder`	Set a probability in $[0, 1]$
Group (3)	`zone_blackout`, `zone_recover`	Mark every node with a label value down; clear
	`federation_partition`	Degrade every link between the nodes of two label values and a federation-level link

Three delivery paths. The same engine can write `sim/overrides.json` (picked up by the watcher), post each change to `/observe`, or – inside the API process – apply the change directly to `DTState` through a store subclass (`ChaosRunner`), which avoids races between a stop request and the file watcher. The runner accepts a speed factor between 0.1 and 1000, runs one schedule at a time, keeps a log of the last 200 messages, and emits `fabric.chaos.started` and `fabric.chaos.finished` or `stopped` events. The CI gate uses a fourth, silent path in which events are applied in schedule order without waiting.

Noise. An optional `NoiseInjector` (disabled by default and in the deployed service) perturbs one random node (thermal derate, used cores, battery) and one random link (RTT, jitter, loss) every 7.5 s to keep a demonstration twin lively.

6.2.7 Self-Healing and Resource Guardian

Self-healing controller. When a plan is committed (not a dry run), the API registers each placed stage with the controller: the job, the stage, its resource request, the primary node and reservation, the fallbacks and any shadow reservations. Every 2.5 s the controller examines each registered primary node. The node counts as *degraded* if it no longer exists, is marked down, has a reliability below 0.75, or has a predicted failure window shorter than 90 s. A stage must be degraded for two consecutive polls (the *stability window*) before the controller acts, which prevents flapping. It then performs the first applicable action:

1. **Promote a shadow:** take the oldest shadow reservation that still exists, release the old primary's reservation, make the shadow the new primary and emit `fabric.selfheal.promote`;
2. **Fail over:** otherwise reserve capacity on one of the recorded fallback nodes, chosen at random, release the old primary and emit `fabric.selfheal.failover`;
3. while the stage is healthy, **top up shadows** until the job's redundancy minus one is reached,

emitting `fabric.selfheal.shadow`.

All four thresholds (interval, reliability, availability, stability) are configurable through environment variables (Table 6.7).

Resource guardian. Every 5 s the guardian inspects each node that holds reservations. If the node's CPU utilisation is at least 0.92 it releases the single oldest reservation whose age exceeds 300 s and emits `fabric.guardian.release`. The intent is to prevent a permanent deadlock in which abandoned reservations keep a nearly full node closed to new work. As noted in Section 7.5, the guardian does not tell the self-healing controller when it releases a primary's reservation.

6.2.8 CI Resilience Gate

Purpose. Chaos tools test running infrastructure; the gate tests a *scheduling change* before it merges, the way unit tests guard application code. `tools/ci_gate.py` replays named fault schedules against a fresh twin for every (scenario, strategy) pair, plans the job catalogue in dry-run mode at checkpoints along the timeline, and checks the worst case against thresholds.

Procedure. For each pair a new `DTState` is created. The planner measures a *baseline* sample before any fault. Events are then applied in schedule order without waiting; a sample is taken only after *damage* events (reverts are skipped, because sampling after a revert would measure recovery and hide the peak), limited to six checkpoints per scenario plus the final state. At each sample every job is planned, annotated with cost and carbon, and compared with its deadline. A sample records the SLA percentage, the p95 and mean latency, and the total cost and carbon of the catalogue.

Verdict. The gate fails (exit code 1) if, for any pair, the worst SLA is below the floor, the worst p95, peak cost or peak carbon exceeds an absolute ceiling, the worst SLA has dropped by more than the allowed number of points relative to a committed baseline, or the worst p95 has grown by more than a percentage. Bad usage returns exit code 2. The shipped configuration (`ci/gate.yaml`) uses the first six catalogue jobs, the strategies `greedy` and `resilient`, three scenarios (the default chaos list, the named `core_link_cut`, and an inline `gate_capacity_squeeze` that blacks out five zones in turn), seed 7, six checkpoints, a worst-SLA floor of 80 %, a 2-point SLA regression budget, a 10 % p95 regression budget, and a *deadline scale* of 0.28.

Deadline scale. The bundled catalogue runs at roughly a third of its stated deadlines, so SLA would stay at 100 % even with most of the fabric down. Multiplying every deadline by 0.28 turns the gate into a margin test: "does the policy still meet a deadline of 28 % of the stated budget?"

Determinism and integration. Fault schedules are fixed, planning is a dry run, and the seed fixes the randomness of the bandit and RL components, so the same commit and seed give a byte-identical report; the test suite asserts this. A GitHub Actions workflow runs the gate on every pull request that touches `dt/`, `sim/`, `planner/`, `nodes/`, `jobs/` or `ci/`, writes a Markdown summary to the job page and uploads the JSON report.

6.2.9 Blast-Radius Finder

Purpose. Chaos replay answers "what happens if I inject the faults I thought of?". The blast-radius finder inverts the question: it searches the fault space for the *minimal* combinations of faults that push a job past its deadline.

Fault space. Five fault kinds are modelled: `zone_blackout` (every node with a zone label value), `node_down`, `thermal_derate` (default 0.6), `link_down` and `link_degrade`. Node-level faults are restricted to the nodes that the unfaulted plan actually uses, because killing a node that holds

no work cannot break the job.

Search. The search is breadth-first over the size of the fault set, so any set reported is minimal by construction. First every single fault is evaluated. Faults that break the job alone are reported and excluded from combinations; the survivors are ranked by the latency they cause, and only the worst `branch` of them are combined into pairs, then triples, skipping any combination that contains an already-known breaking set. If n is the size of the fault space and b the branching limit, the number of placements is about $1 + n + \binom{b}{2}$ at depth two and adds $\binom{b}{3}$ at depth three. The API bounds the depth to 1–3 (default 2), the branch to 2–20 (default 10) and the number of reported sets to 1–50 (default 15), and runs the search on a throw-away `DTState` so that the live dashboard never flickers.

Output. The report gives the baseline latency and headroom, the nodes used, the size of the search (fault space, combinations, placements, duration), the minimal breaking sets, the five worst surviving single faults, the fault that appears in most breaking sets, and a one-line verdict such as "survives every fault combination searched".

6.2.10 What-If Planner

Purpose. "Would two more GPU nodes fix our deadline misses, or do we need a faster link?" is a procurement question that the twin can answer without touching the real fabric. `dt/whatif.py` forks the state twice from disk, applies a proposed change to one fork, replays the same job catalogue through both and diffs the results.

Changes. A change set may contain `add_nodes` (full descriptors), `clone_nodes` ("give me N more of this node"), `remove_nodes`, `patch_nodes` (deep-merge a partial descriptor), `links` (upsert) and `remove_links`. Nothing is persisted: every change is applied with `persist=False`, so `nodes/` and `topology.yaml` are untouched.

Contention. By default the jobs are planned with reservations committed as they go, so job two sees the capacity job one consumed. Without contention every job independently picks the single best node and extra capacity looks worthless, which is exactly the question a what-if is asked. Faults (for example a zone blackout) can be applied to both forks to ask whether a purchase holds up *during* an outage.

Report. For each side: per-job latency, deadline, whether it was met, cost, carbon and nodes used, plus the number of deadlines met, SLA percentage, mean and p95 latency, catalogue cost and carbon, and the fabric size. The deltas and a verdict sentence ("2 more job(s) meet their deadline and p95 improves 27 %") are added.

6.2.11 Monte-Carlo Orchestrator

Purpose. `sim/montecarlo.py` estimates the distribution of job completion time and the SLO hit rate of a catalogue under random perturbations. It is a standalone, simplified model: it loads node descriptors and jobs directly, perturbs them in each trial, places each stage with a built-in greedy rule (minimum compute plus transfer), and estimates the makespan.

Perturbations. In each trial every node fails with probability $\frac{1}{2}\big(0.03 + 0.01 \cdot \text{crashes} + 0.1 \cdot \text{derate} + 0.05 \cdot [\text{trust} < 0.75]\big)$ and otherwise receives a Gaussian thermal derate; every link fails with probability $\frac{1}{2}\big(0.02 + 0.2 \cdot \text{loss}/100\big)$ and otherwise has its speed multiplied by a factor in $[0.5, 1]$, its RTT by $[0.8, 1.5]$ and its jitter by $[1, 2.5]$.

Workloads. A catalogue can be loaded from a YAML file or generated: five stage profiles (`cpu`, `gpu`, `io`, `vision`, `edge`) define ranges for data size, cores, memory, VRAM and allowed formats; a generated job has 2–4 stages, random edges, a deadline drawn from three regimes,

redundancy 1–3 and a priority.

Output. A CSV with one row per trial (latency and SLO flag) and a console summary of mean, p95 and p99 latency and SLO hit rate. The default seed is 123. The orchestrator uses its own cost formulas, which are a simplified version of the cost model, so its absolute numbers are not comparable with the planners' predictions.

6.2.12 Interoperability, Bundles and MQTT

Exporters. `dt/exporters.py` serialises the twin as Azure Digital Twins Definition Language models (identifiers such as `dtmi:fabric:node:<name>;1`, with properties and telemetry for each node and link) and as Kubernetes custom resources (kinds `NodeTwin` and `LinkTwin`, apiVersion `fabric/v1alpha1`, together with matching `CustomResourceDefinition` objects for the group `fabric.dev`).

Cluster importer. `tools/import_cluster.py` builds a twin from a Kubernetes node list (`kubectlgetnodes-ojson`, or a saved file). It maps the node name, architecture (amd64, arm64, riscv64), CPU and memory capacity (parsing Kubernetes quantities such as `250m` and `16Gi`), GPU capacity and labels, the zone label, the instance type, the control-plane role and the `Ready` condition (a not-ready node becomes a down node). Kubernetes says nothing about power, thermals, storage wear, topology or trust, so those fields are written with explicit defaults (for example 2.5 GHz, 65 W, reliability 0.95, trust 0.8) that are listed in the importer's summary: an imported fabric gives believable *placement*, but its energy, carbon and risk figures are only as good as the defaults that the user replaces.

Incident bundles. `tools/bundle.py` packages a run into one ZIP with a manifest (bundle version, Git commit, timestamp, node and link counts), the node descriptors, the topology, the job catalogue, the faults in force, and, when taken from the live API, the snapshot, recent events and recent plans. Import unpacks to a named directory, refuses to overwrite a non-empty one without `--force`, flags a version mismatch rather than failing, and rejects any archive entry that would escape the target directory.

MQTT sidecar. `sim/mqtt_bridge.py` is an optional process that subscribes to a broker and forwards telemetry to `POST/observe`. Topics follow `fabric/<node>/telemetry` for a node and `fabric/<a>__<b>/telemetry` for a link; the payload is either {`"changes": {...}`} (apply) or {`"action": "revert", "fields": [...]`} (revert). The parsing is a pure function that returns `None` for malformed input and never raises, and the `paho-mqtt` library is imported only when a connection is made, so the module and its tests need neither a broker nor the library.

6.2.13 API, Events and Dashboard

API. `dt/api.py` is a Flask application with the 29 routes of Table 5.4. A `before_request` hook enforces the API key on the ten gated endpoints when `FABRIC_API_KEY` is set, using a constant-time comparison; the variable is read on every request so that tests can change it. `/plan` resolves the strategy name, runs the planner, adds the SLO penalty, the federation summary, the predictive overview, and cost and carbon, registers a committed plan with the self-healer, stores a slimmed copy (without the point-in-time fabric snapshots, about 70 KB each) in a 200-entry ring buffer, and emits `fabric.plan.committed`. `/plan_batch` reuses the same handler for each job.

Events. State changes are wrapped as CloudEvents 1.0 with type prefixes `fabric.node.*`, `fabric.link.*`, `fabric.reservation.*`, `fabric.plan.committed`, `fabric.job.*`, `fabric.chaos.*`, `fabric.selfheal.*` and `fabric.guardian.*`, source `fabric.dt.state`. The bus keeps the last 512 events (configurable) and fans out to subscriber queues without blocking: if a slow

client's queue is full, the event is dropped for that client. `/stream` serves the bus as Server-Sent Events (a `retry` hint, a comment on connection, a keep-alive comment every 15 s, and one unnamed `data:` frame per event so that a client's `onmessage` handler receives every type); `/events` returns the recent buffer.

Dashboard. The React application (Section 6.3) subscribes to `/stream` and refetches only the queries that an event type affects, with a 30-second safety-net poll.

Table 6.7: Environment variables that configure the twin

Variable	Default	Meaning
`FABRIC_API_KEY`	unset	Enables `X-API-Key` on mutating routes
`FABRIC_SELF_HEAL_INTERVAL` / `_RELIABILITY` / `_AVAIL` / `_STABILITY`	2.5 s / 0.75 / 90 s / 2	Self-healing poll, reliability and availability thresholds, stability window
`FABRIC_GUARDIAN_INTERVAL` / `_UTIL` / `_TTL`	5 s / 0.92 / 300 s	Guardian poll, utilisation threshold, reservation lifetime
`FABRIC_COST_WEIGHT`, `FABRIC_CARBON_WEIGHT`	25, 25	Weights of the cost-aware and carbon-aware greedy planners
`FABRIC_ENABLE_NOISE`, `FABRIC_NOISE_*`	0	Optional noise injector and its intervals
`FABRIC_DT_EVENT_BUFFER`, `FABRIC_SSE_KEEPALIVE`	512, 15 s	Event buffer size; SSE keep-alive
`FABRIC_JOBS_ROOT`, `FABRIC_RL_STATE`, `FABRIC_BANDIT_STATE`	jobs, sim/…	Job directory; persisted Q-table and bandit state

6.3 User Interface

The dashboard is a single-page application written in TypeScript with React 19, built with Vite and styled with Tailwind CSS 4. It is served as static files and communicates with the Flask API only over REST and Server-Sent Events. The active tab is mirrored in the URL hash (`\#nodes`, `\#docs/api`, …), so that a reload or the browser's back button returns to the same screen, and each tab sets its own page title. Server data is cached and refreshed with TanStack Query; D3 draws the topology graph. Figures 6.3 and 6.4 show the structure of the interface as schematics; screen captures of the running application can be inserted at the marked points of each subsection.

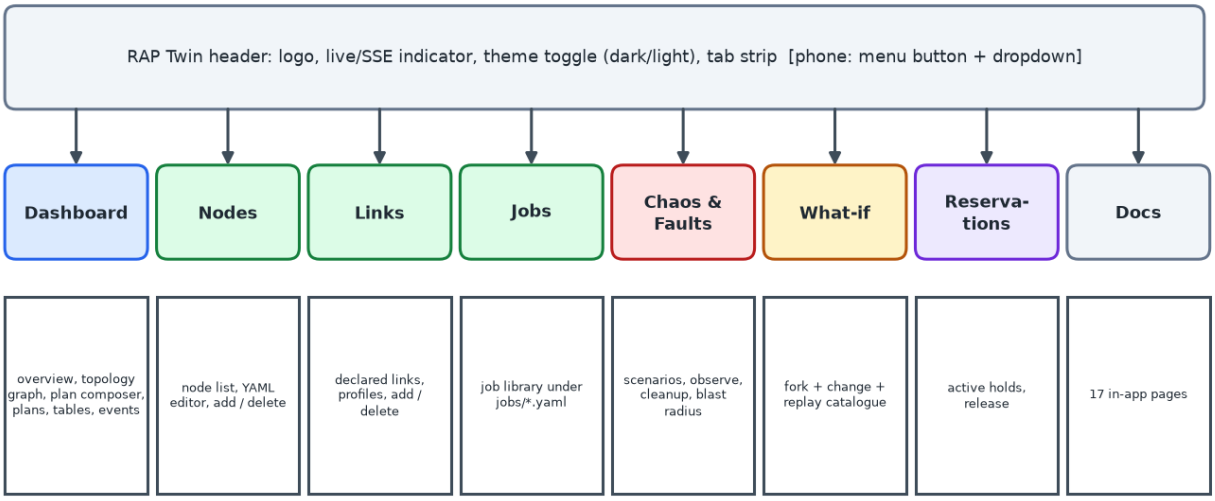


Figure 6.3: Dashboard navigation: eight tabs and their contents

6.3.1 Cross-Cutting Behaviour

Live updates. A hook subscribes to `/stream` and, for each event, refetches only the queries that the event type can have changed: plan events refresh the plan list and the snapshot; chaos and override events refresh the chaos status and the snapshot; link additions and removals refresh the topology; node, link and reservation events refresh the snapshot; self-heal and guardian events refresh the snapshot and the plans. Bursts (a chaos scenario can emit hundreds of events per second) are coalesced with a 120 ms flush, and a 30-second safety-net poll heals any missed event. A header indicator shows whether the stream is connecting, live, or offline (in which case the application falls back to polling), and a Live/Paused button lets the user freeze the display.

Theme and layout. The interface offers a dark and a light theme with a toggle in the header; colours are defined as design tokens so that every panel follows the chosen theme. On a phone the tab strip is replaced by a menu button and a dropdown that closes on outside click, the Escape key or navigation; on tablets and desktops a scrollable tab strip with edge fades and arrow buttons is used.

Errors and guidance. If the API cannot be reached, a banner states the problem and the command that starts the server. Destructive actions (clear history, release all reservations) ask for confirmation. Because a dry-run plan reserves nothing, the plan composer shows a hint that CPU usage on the Overview and the topology graph will not change until "Dry run" is unchecked.

6.3.2 Dashboard Tab: Overview Screen

The Overview card summarises the fabric in twelve key figures: node, link and federation counts, nodes down, active reservations, the share of CPU in use, the highest federation load, and – for the selected plan – its federation spread, fallback coverage, cost, CO_2 and the time of the last snapshot (`Overview.tsx`). Data come from `GET/snapshot`, refreshed by events. Figure 6.4 shows how the tab is composed.

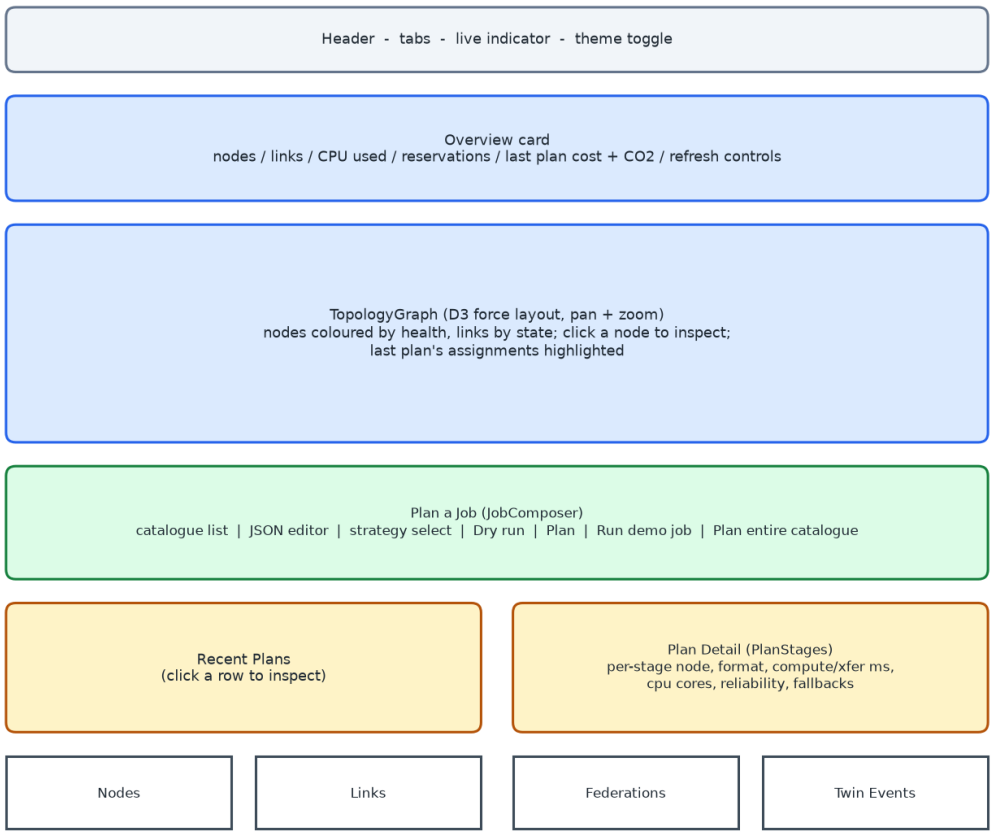


Figure 6.4: Composition of the Dashboard tab (schematic)

6.3.3 Topology and Federations Screens

The topology graph (`TopologyGraph.tsx`) draws the fabric with a D3 force simulation. A hundred-node fabric would overflow a fixed canvas, so the graph supports pan and zoom, shows per-node labels only above a zoom threshold (hover names a node at any zoom), and lets the user open a node's details. Nodes are coloured by state, a node that carries a stage of the selected plan receives a highlighted outline, and a legend explains the dashed same-zone grouping and the zone-backbone links. This makes the effect of a strategy visible at a glance: on the reference fabric a greedy plan collapses onto one node, whereas a resilient plan spreads across federations. The federation tables (`FederationsPanel.tsx`) list each federation's nodes, free and total CPU, memory and VRAM, down and hot nodes, reservations, average trust and loss and load factor; the companion table shows cross-federation link health aggregated from the topology and from chaos overrides.

6.3.4 Job Composer Screen

The composer (`JobComposer.tsx`) is where a placement is requested. On the left is the job library (`GET/jobs`); selecting an entry loads it as JSON into an editor on the right, together with a summary of its stages, deadline and redundancy. The user chooses one of ten strategies, decides whether to run as a dry run (checked by default) and presses *Plan*. Three further buttons run the built-in demonstration job, plan the *entire catalogue* in one batch call, and clear the editor. The outcome

is announced beneath the editor ("placed in 929.6 ms", quoting the plan's predicted latency, or "INFEASIBLE" with a reason), and the resulting plan appears in the plan list.

6.3.5 Recent Plans and Plan Detail Screens

Recent Plans (`RecentPlans.tsx`) lists the plans held in the API's 200-entry ring buffer; clicking a row selects it. Plan Detail (`PlanStages.tsx`) renders the selected plan as a chain of stage cards joined by arrows. Each card shows the stage identifier, the chosen node, the execution format, the compute and transfer time, the CPU cores requested and, when known, the cores available on that node, the node's reliability and availability window, its federation, up to three fallbacks and the reservation identifier. A red card with a reason marks an infeasible stage. Beneath the chain a summary strip lists latency, energy, risk, cost (with the egress component), CO_2, average reliability, federation spread, fallback coverage, SLO penalty and, for the Markov planner, the value of the solved MDP.

6.3.6 Nodes and Links Screens

The *Nodes* tab (`NodeManager.tsx`, `NodeEditor.tsx`) lists every node with a search over name, class and label. Selecting or creating a node opens an editor with sections for identity (name, class, architecture, role), compute (cores, base frequency, micro-architecture, RAM, GPU model and VRAM), network and health (fabric, speed, base latency, reliability) and labels, together with a raw JSON view. Saving writes `nodes/<name>.yaml` and applies it to the live twin immediately; deleting removes the file and any reservations held on the node. The *Links* tab (`LinkManager.tsx`) lists the declared links and lets the user add or delete one: endpoints may be node names or site names, a profile can be chosen to fill in the metrics (which can still be overridden), and scope, QoS class and subnet can be set. Declared links are written to the `links:` section of `topology.yaml`.

6.3.7 Jobs Screen

The *Jobs* tab (`JobManager.tsx`) maintains the library under `jobs/*.yaml`. A job is edited either as JSON or through a form: identifier, deadline, redundancy, and an ordered list of stages that can be added, moved up or down and removed, each with an identifier, a type, a data size, and CPU, memory and VRAM requests. Names are validated before saving so that a job cannot be written outside the jobs directory.

6.3.8 Chaos and Faults Screens

The *Chaos & Faults* tab (`ChaosPanel.tsx`, `ObservationPanel.tsx`, `BlastRadiusPanel.tsx`) brings the resilience tools together. *Chaos scenarios* lists the scenarios of `topology.yaml`, takes a speed factor, starts and stops a run inside the twin, and shows a progress bar and the engine log. *Health & cleanup* shows the number of nodes down, nodes derated and links down, and offers three actions: reset all faults, clear the plan history, and download an incident bundle (a plain link, so the browser streams the ZIP directly from `/bundle`). *Inject Observation* posts a hand-written node or link change to `/observe`, the same path that the chaos engine and the MQTT sidecar use. *Blast radius* lets the user choose a job, a strategy, a maximum set size and an optional deadline, runs the search, and shows the verdict, the baseline headroom, the minimal breaking sets and the worst surviving faults.

6.3.9 What-If and Reservations Screens

The *What-if* tab (`WhatIfPanel.tsx`) asks a capacity question. The user picks a mode – add copies of a node, remove a node, upgrade a node's core count, or write a custom JSON change – a strategy,

and optionally a zone to black out in both forks. The result is a before-and-after table of deadlines met, p95 and mean latency, catalogue cost, catalogue CO_2 and fabric size, with a verdict sentence. The *Reservations* tab (`ReservationsPanel.tsx`) lists the capacity held by committed plans, with a filter, and lets the user release one reservation, the filtered set, or all of them.

6.3.10 Documentation Screen

The *Docs* tab is a small documentation site inside the application. It has 17 pages in four groups: *Getting started* (Introduction, Quickstart), *Concepts* (Architecture, Digital-twin state, Node descriptors, Topology and links, Jobs), *Guides* (Dashboard, Planning, Chaos and resilience, Analysis tools, Telemetry ingestion) and *Reference* (API reference, Configuration, Measured results, Deployment and security, Troubleshooting and FAQ). The content is static, so it renders even when the API is down, and it has a search box and an in-page outline.

6.3.11 Screen-to-Interface Mapping

Table 6.8 summarises which API routes feed each screen.

Table 6.8: Dashboard tabs and the interfaces they use

Tab	**Panels (component files)**	**Routes used**
Dashboard	Overview, TopologyGraph, JobComposer, RecentPlans, PlanStages, NodesTable, LinksTable, Federations, EventsFeed	`/snapshot`, `/plans`, `/jobs`, `/plan`, `/plan_batch`, `/events`, `/stream`
Nodes	NodeManager, NodeEditor	`/nodes/<name>`, `/add_node`, `/snapshot`
Links	LinkManager	`/topology`, `/links`
Jobs	JobManager	`/jobs`
Chaos & Faults	ChaosPanel, Health & cleanup, ObservationPanel, BlastRadiusPanel, EventsFeed	`/chaos`, `/chaos/start`, `/chaos/stop`, `/overrides/reset`, `/plans`, `/bundle`, `/observe`, `/blast_radius`
What-if	WhatIfPanel	`/whatif`
Reservations	ReservationsPanel	`/snapshot`, `/release`
Docs	Docs and 17 page components	none (static)

6.4 Data Design

6.4.1 Data Structure

RAP twin does not use a relational or NoSQL database management system. Persistent configuration is stored as YAML and JSON files that are human-readable and version-controlled, and the runtime state lives in the in-memory `DTState` object together with a JSON overrides file. Table 6.9 lists the principal data stores.

Table 6.9: Principal data stores

Store	Content	Written by
`nodes/*.yaml` (100 files)	One node descriptor each	Generator (`sim/gen_nodes.py`), cluster importer, dashboard
`sim/topology.yaml`	Defaults (network, routing, security, pricing, energy), regions and sites, subnets, link profiles, links, placement policies, data locality, trust overrides, chaos list and scenarios	Hand-edited; `/links` writes only the `links:` block
`sim/overrides.json`	Faults in force: `\{"nodes":\{\ldots\ },"links":\{\ldots\}\}`	Chaos CLI; the watcher reads it
`jobs/*.yaml`	Job library	Hand-edited; Jobs tab
`ci/gate.yaml`, `ci/baseline.json`	Gate thresholds; committed reference report	Hand-edited; `tools/ci_gate`
`sim/rl_state.json`, `sim/bandit_state.json`	Persisted Q-table and bandit statistics	RL and bandit policies of the API process
`reports/*.json`	Benchmark and gate reports	Benchmark scripts
In memory	Dyn state, reservations, event buffer (512), plan history (200), predictor series	`DTState`, API

Node descriptor

A node descriptor is a YAML document whose top-level blocks are consumed by different parts of the system (Table 6.10). An abridged real descriptor of a gaming-class workstation is shown below.

Table 6.10: Node descriptor blocks and their consumers

Block	**Key fields**	**Used by**
identity	`name`, `class`, `arch`, `role`, `os`	Name validation, federation, import/export
`cpu`, `memory`	`cores`, `base_ghz`, `turbo_ghz`, `tdp_w`; `ram_gb`	Capacities (`caps`), compute time, energy
`gpu`, `accelerators`	`vram_gb`, `accel_score`; `npu_tops`	Accelerator multiplier, VRAM feasibility
`power`, `battery`	`tdp_total_w`, battery level and drain	Energy, reliability, availability window
`storage`, `health`	`tbw_pct_used`; `last_week_crashes`, `thermal_derate`, `reliability`, `mtbf_hours`	Risk score, predictor
`network`	`fabric`, `speed_gbps`, `base_latency_ms`, `loss_pct`	Link estimation when no link is declared
`formats_supported`	native, wasm, cuda, npu, fpga, asic	Format feasibility, accelerator multiplier
`labels`	`zone`, `rack`, `trust`, `wasm`, `owner`	Federations, trust term of the risk score, zone pricing

```
name: grig-001
class: gaming_rig
arch: amd64
cpu:    {cores: 16, base_ghz: 3.14, turbo_ghz: 4.67, tdp_w: 78.84}
memory: {ram_gb: 16, ddr_gen: ddr5}
gpu:    {type: none, vram_gb: 0, accel_score: 0.0}
network: {fabric: ethernet, speed_gbps: 1, base_latency_ms: 0.32,
          jitter_ms: 0.09, loss_pct: 0.03}
formats_supported: [native, wasm]
labels: {zone: edge, rack: B1, trust: '0.68', wasm: enabled}
health: {last_week_crashes: 1, thermal_derate: 0.1}
```

Listing 6.1: Abridged node descriptor (nodes/grig-001.yaml)

The reference fabric of 100 nodes is deliberately heterogeneous (Table 6.11); it contains everything from 4-core single-board computers to a 128-core HPC node and an 80 GB data-centre GPU.

Table 6.11: Statistics of the reference fabric (`nodes/`, `sim/topology.yaml`)

Property	Value
Nodes by class	gaming rig 24, single-board computer 24, workstation 17, server 16, laptop 9, phone 7, HPC 3
Zones (federations)	rackB 21, edge 18, home 16, cloudlet 15, rackC 12, rackA 10, lab 8
Capacity	2 022 CPU cores and 10 508 GB of RAM in total; 51 nodes with a GPU
Execution formats	native and WebAssembly on all 100; CUDA on 51; NPU on 10; FPGA on 4; ASIC on 1
Topology	5 site-level links (campus core and city mesh) and 2 node-level links; 6 link profiles from 1 Gbps WAN to 400 Gbps InfiniBand; 4 sites in 2 regions
Pricing and energy	INR; 2.0 per core-hour, 30 per GPU-hour, 6 per NPU-hour, 0.8 per GB-hour, 1.0 per GB egress; 8 per kWh; 720 g CO_2/kWh
Job catalogue	11 jobs, 36 stages, 3–4 stages per job, deadlines from 3 800 to 12 000 ms

Job descriptor

A job is a short YAML document: an identifier, a deadline in milliseconds, an optional redundancy, and an ordered list of stages. Each stage carries a type, a data size, resource requests and the execution formats it allows or forbids.

```
- id: job-vision-small
  deadline_ms: 4500
  stages:
    - id: s1_ingest
      type: io
      size_mb: 40
      resources: {cpu_cores: 1, mem_gb: 1}
      allowed_formats: [native, wasm]
    - id: s3_cnn
      type: cv
      size_mb: 120
      resources: {cpu_cores: 4, mem_gb: 4, gpu_vram_gb: 2}
      allowed_formats: [cuda, native]
```

Listing 6.2: A job descriptor (jobs/jobs_10.yaml, abridged)

Plan result

The planner returns a JSON object whose main fields are shown below for the greedy planner on the job above (values from a dry run on the reference fabric; the API adds the predictive overview and federation summary, which are omitted here).

```
{ "job_id": "job-vision-small",
  "assignments": {"s1_ingest": "hpc-080", "s2_preproc": "hpc-080",
                  "s3_cnn": "hpc-080"},
  "latency_ms": 929.567, "energy_kj": 0.01145, "risk": 0.0949,
```

```
"cost_total": 0.004326, "currency": "INR", "co2_g": 0.0024,
"infeasible": false,
"per_stage": [ ..., { "id": "s3_cnn", "node": "hpc-080",
    "format": "cuda", "compute_ms": 312.672, "xfer_ms": 0.0,
    "cpu_cores_requested": 4.0, "cpu_cores_available": 128.0,
    "cost": 0.003587, "co2_g": 0.0008 } ] }
```

Listing 6.3: Abridged plan result for job-vision-small (greedy, dry run)

6.4.2 Database Description

There is deliberately no database server in this project. Figure 6.5 shows the logical entities and how they relate: a topology declares links and scenario events; a link joins two nodes (or sites); every node has exactly one runtime overlay, which holds zero or more reservations; a job has several stages; a plan assigns each stage of a job to a node and, when committed, creates reservations; a chaos event targets a node, a link or a group of nodes. These relationships are enforced in memory by `DTState` and the API validators rather than by foreign keys. This keeps the sandbox dependency-free and trivially portable, at the cost of not yet supporting multi-process or multi-host deployments of the simulator itself – a trade-off made explicitly (Section 5.4).

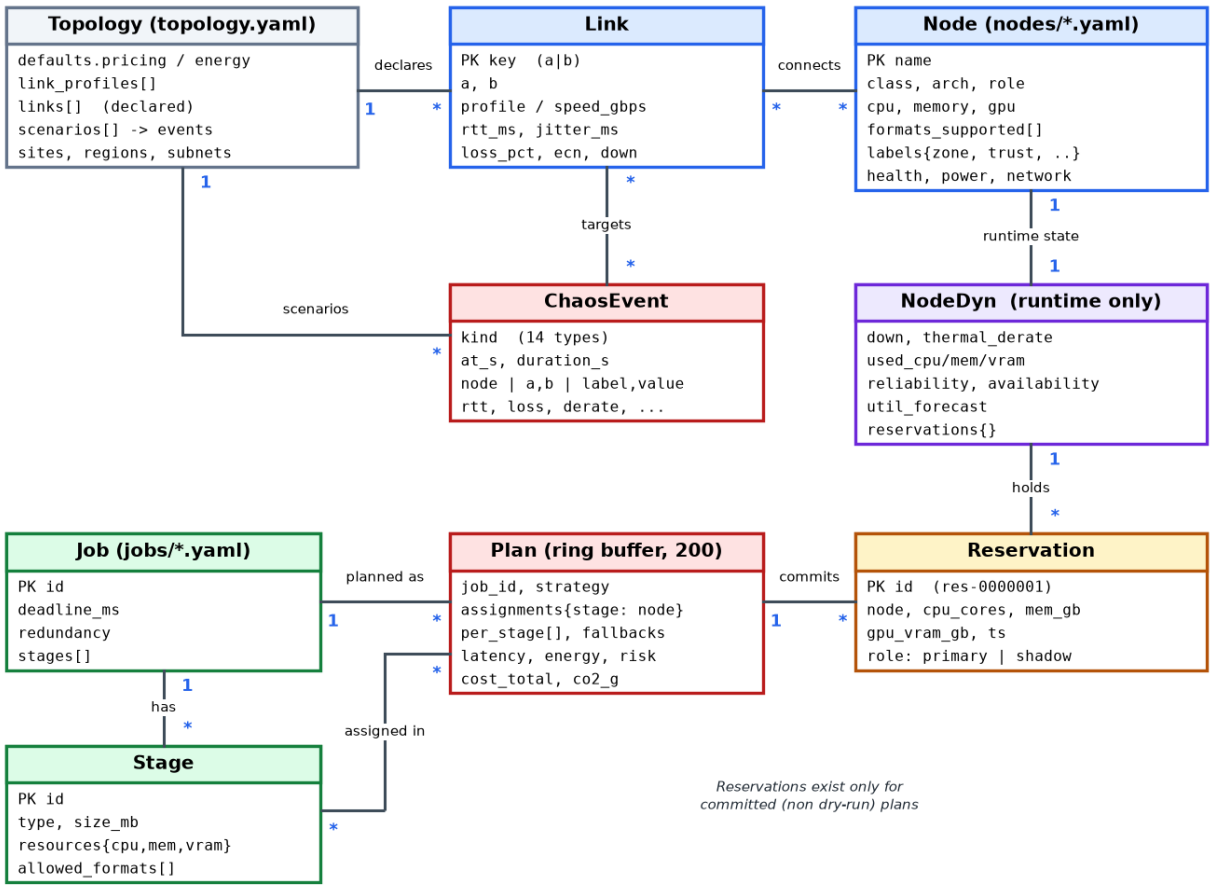


Figure 6.5: Logical entity-relationship diagram of the twin's data

6.5 UML Diagrams

This section presents the behaviour and structure of the system with standard UML notation: a use-case diagram, a class diagram of the twin's core, two sequence diagrams for the most important interactions, and a state diagram for the two lifecycles that matter to resilience. The entity-relationship diagram is given with the data design (Figure 6.5) and the planner algorithms are drawn as flowcharts in Section 6.7.

6.5.1 Use-Case Diagram

Figure 6.6 shows the interactions between the actors and the system. The primary actor is the fabric operator or researcher, who loads topologies, submits jobs for dry-run or committed planning, injects telemetry and chaos, runs Monte-Carlo sweeps, evaluates what-if changes and exports the twin (use cases UC1–UC5 and UC8–UC10 of Table 5.2). The supporting actors are the self-healing controller and the resource guardian (UC6 and UC7, triggered by the system), the chaos engine, and an external control plane that consumes the DTDL and CRD exports. The CI system is a further consumer of the gate.

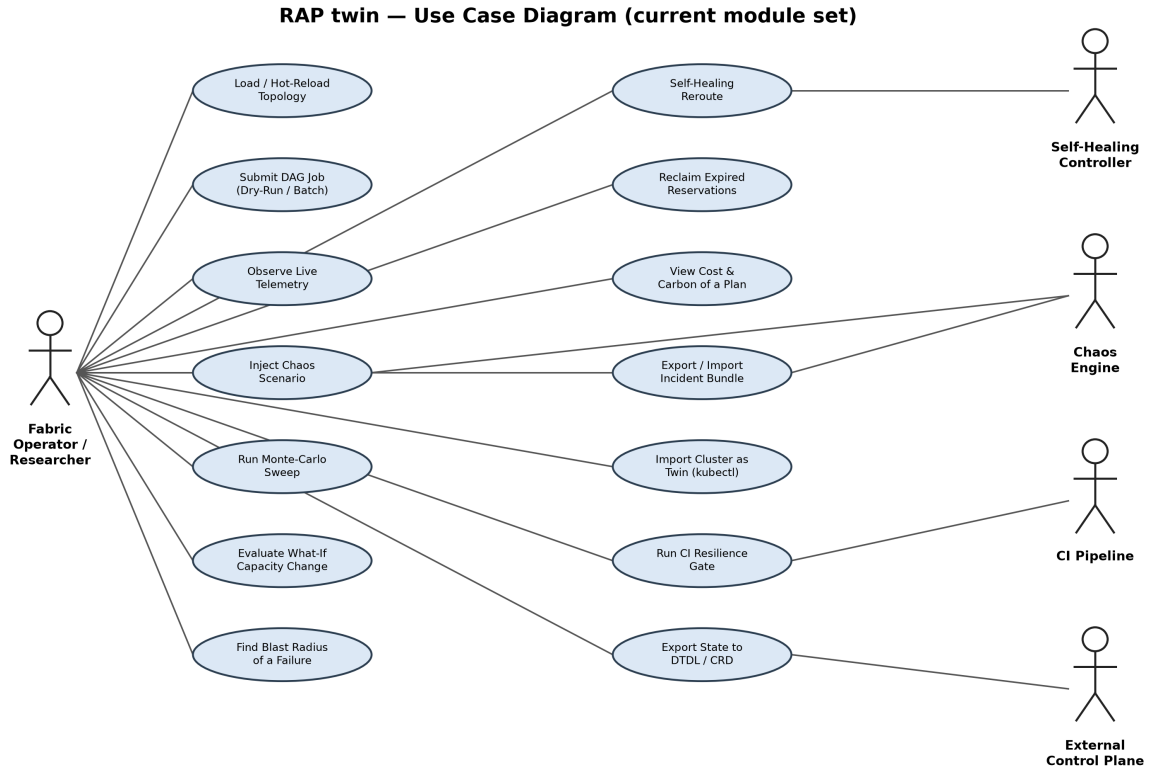


Figure 6.6: Use Case Diagram

6.5.2 Class Diagram

Figure 6.7 shows the classes of the digital-twin core and their associations. `DTState` is the hub: the three controllers (self-healing, guardian, chaos runner) mutate it through reservations and observations; it owns a `PredictiveAnalyzer`, from which it reads forecasts, and an `EventBus`, to

which it emits CloudEvents. Planners read it through `nodes_for_planner()` and reserve capacity through `reserve()`. All planners price stages with the shared `CostModel`; `Economics` supplies the money and carbon rates used by the cost-aware greedy planners and by `annotate_plan`. The `BanditPolicy` and `RLPolicy` are optional collaborators of the greedy and Markov planners respectively. The planner interface is structural rather than inherited: any object with `plan_job(job,dry_run)` can be used, which is what makes adding a strategy cheap.

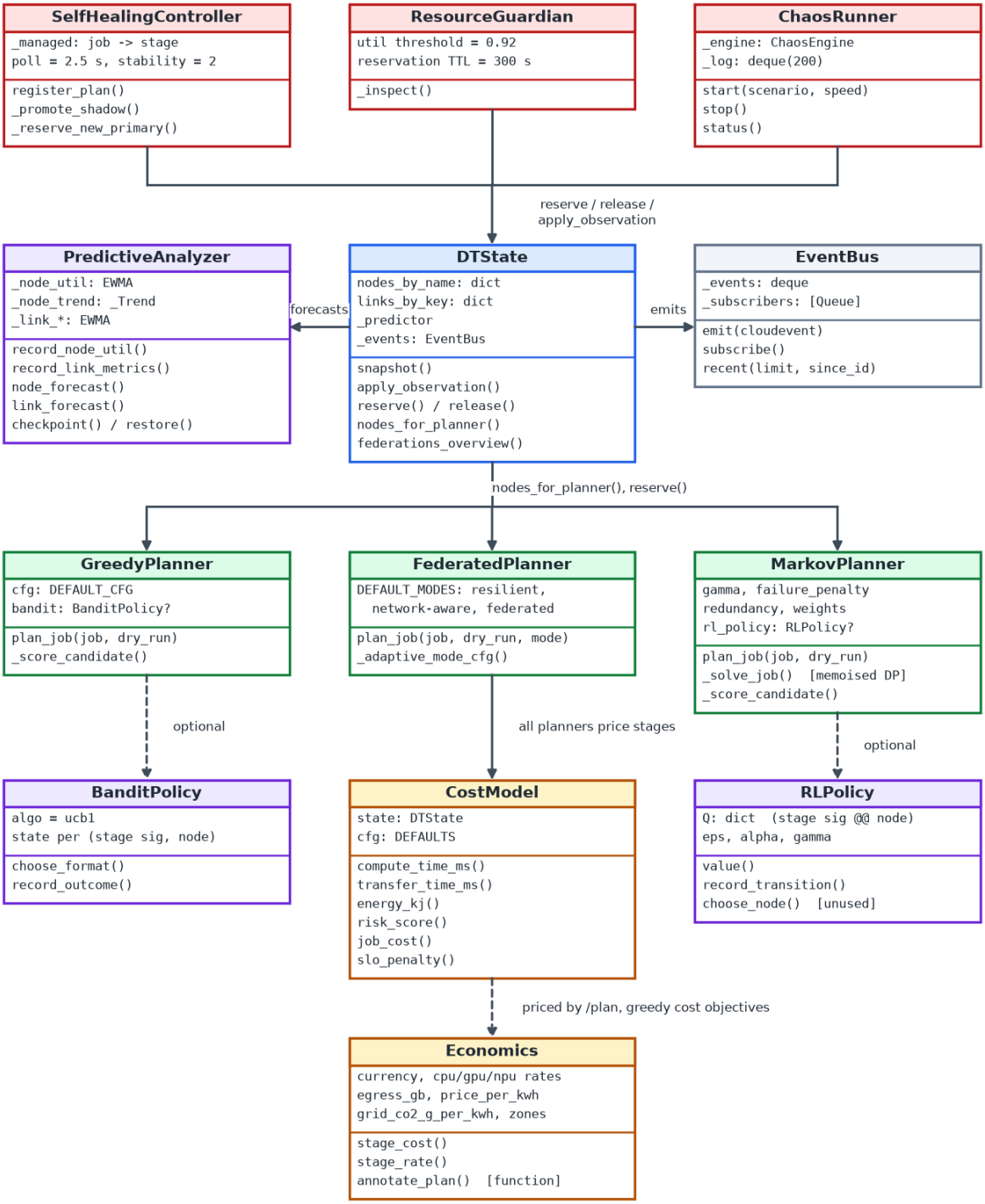


Figure 6.7: Class diagram of the digital-twin core (`dt/`)

6.5.3 Sequence Diagrams

Planning a job. Figure 6.8 follows a `POST/plan` request. The API selects the planner and calls `plan_job`. The planner obtains a consistent copy of all nodes with their forecasts, and for every stage and candidate node asks the cost model for compute time, transfer time, energy and risk, scores the candidates and picks the best (with fallbacks, for the resilient and Markov planners). If the request is not a dry run, the planner reserves capacity on the chosen nodes. The API then prices the plan, registers a committed plan with the self-healing controller, stores it in the history and emits `fabric.plan.committed`; the dashboard, which is subscribed to the SSE stream, refetches the plan list and the snapshot.

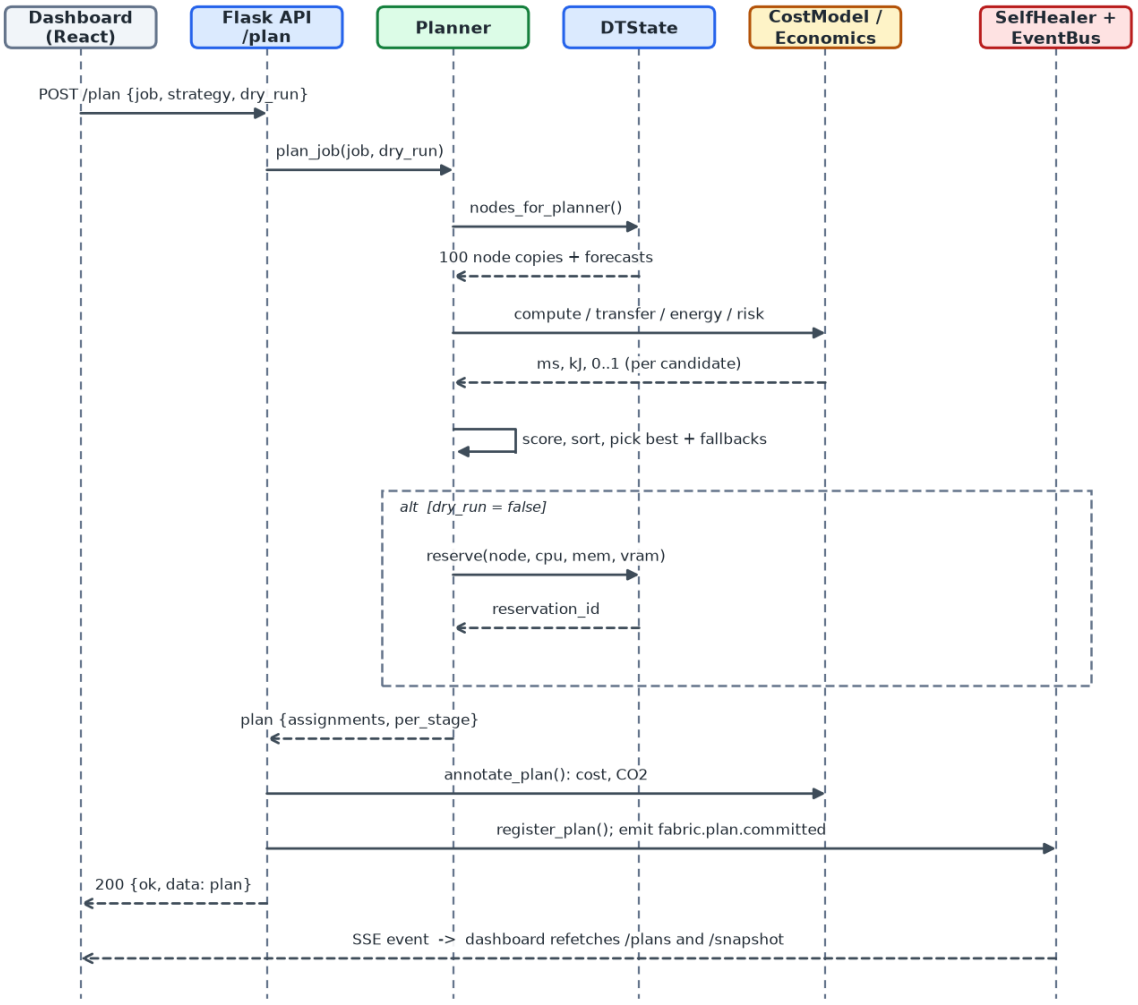


Figure 6.8: Sequence diagram: planning a job with `POST /plan`

Fault and self-healing. Figure 6.9 follows a node failure. The chaos runner applies a `node_kill` observation to the state engine, which updates the node's forecast through the predictor and emits events. The self-healing controller, polling every 2.5 s, reads the primary node of each

committed stage; after two consecutive degraded polls it releases the old primary and promotes a shadow reservation (or reserves a fallback) and emits a `fabric.selfheal.*` event, to which the dashboard reacts.

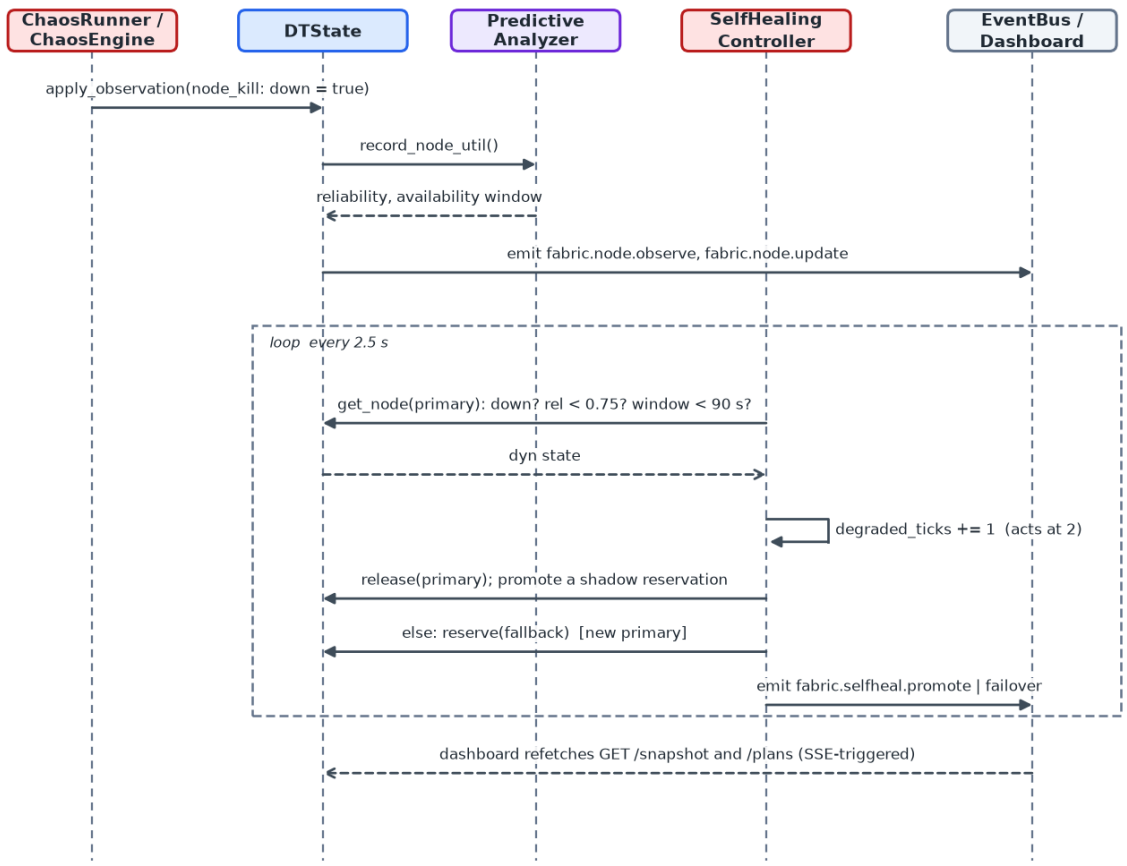


Figure 6.9: Sequence diagram: a node fault and the self-healing reaction

6.5.4 State Diagrams

Figure 6.10 gives the lifecycle of a reservation and of a node's health. A reservation is requested; it is either rejected (no capacity, or the node is down) or becomes active as a primary or a shadow; a shadow is promoted when its primary degrades; and any active reservation ends when it is released, either explicitly, by the self-healer when it replaces a primary, or by the guardian when it exceeds its lifetime on a nearly full node. A node is healthy until a derate, a low battery, a short MTBF or an injected fault drops its reliability below 0.75 or its failure window below 90 s, at which point it is degraded; a kill or a blackout takes it down; and a recovery or a revert returns it to the healthy state.

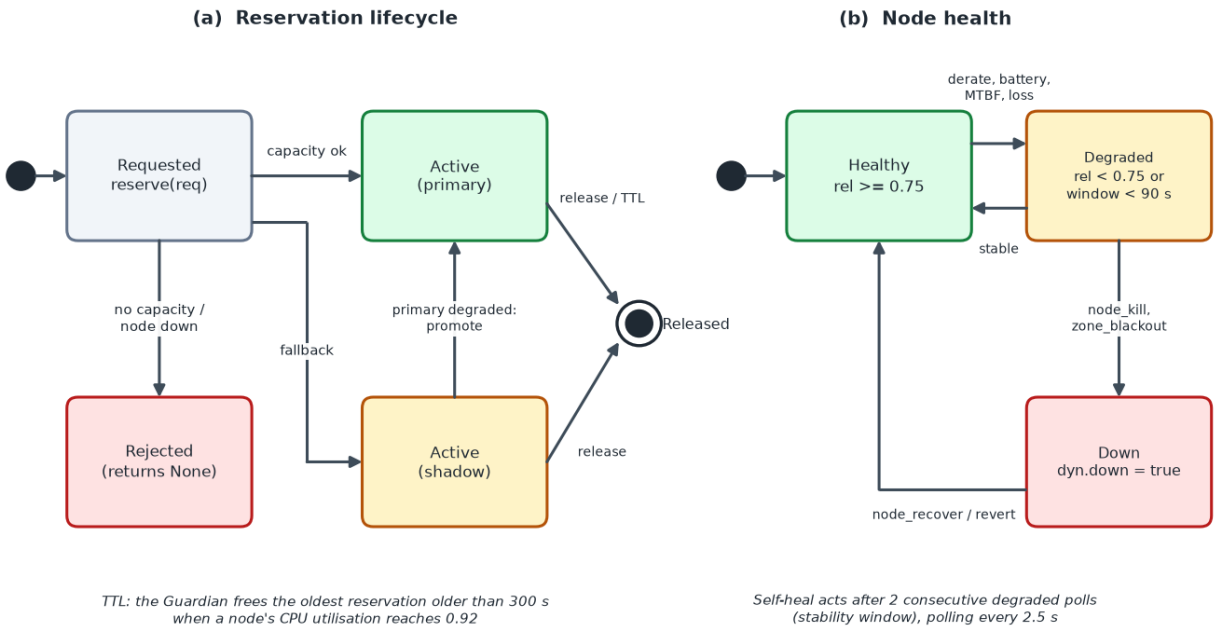


Figure 6.10: State diagrams: (a) reservation lifecycle, (b) node health

6.6 Tools and Technology Used

Table 6.12 lists the technologies of the project, their role, and the reason each was chosen.

Table 6.12: Technologies and their role

Area	**Technology**	**Role and rationale**
Language	Python 3	Planners, cost model, chaos engine and tools; readable enough to serve as the reference for the algorithms
Web framework	Flask ≥3.0, gunicorn	Small, explicit routing; threaded workers allow long-lived SSE connections
Data formats	PyYAML, jsonschema	Human-editable descriptors; JSON Schema validators for nodes, jobs, topology
Analytics	NumPy, Pandas, Matplotlib, rich	Benchmark aggregation, plots and readable terminal output of the command-line tools
Containers	Docker SDK ≥7.0	Optional containerised emulation of the fabric with architecture-specific images and GPU requests
Messaging	paho-mqtt ≥2.1 (optional)	Event-driven telemetry ingestion without a heavyweight broker stack
Front end	React 19, TypeScript 6, Vite 8	Component model for the dashboard; static type checking of the API contract; fast builds
Styling and charts	Tailwind CSS 4, D3 7	Design-token theming (dark and light); force-directed topology graph with pan and zoom
Data fetching	TanStack Query 5	Cache, background refetch and mutation handling driven by SSE events
Quality	pytest, black, isort, ruff, oxlint	166 automated tests; formatting and linting of the Python and TypeScript code
Deployment	Caddy, systemd, rsync	Automatic TLS, static hosting and reverse proxy; service supervision; one-command deployment script
CI/CD	GitHub Actions	Resilience-gate workflow on pull requests; CodeQL static analysis workflow
Interoperability	Azure DTDL, Kubernetes CRDs, CloudEvents 1.0	Open formats for twin export, cluster import and event envelopes
Documentation	LaTeX, Markdown, in-app Docs tab	This report, repository READMEs and the interactive documentation

6.7 Algorithm

This section describes the placement algorithms of RAP twin. Earlier drafts of this report described only the resilient planner and gave its score as a product of three factors; a review of the source code showed that this was wrong – the score is a *sum of millisecond-valued terms* – and the section

has been rewritten from the code (`dt/policy/*.py`). Each algorithm is described by its objective, its parameters, its procedure and its complexity.

6.7.1 Problem Formulation and Notation

A job is a sequence of stages $s_1, \ldots, s_m$. Let $\mathcal{N}$ be the set of nodes ($|\mathcal{N}| = 100$ on the reference fabric). A stage s is *feasible* on a node n if n is not down, its free CPU, memory and VRAM are at least the stage's requests, and it supports at least one of the stage's allowed formats; the set of feasible nodes is $\mathcal{C}(s)$. A planner chooses an assignment $a : \{s_i\} \to \mathcal{N}$ with $a(s_i) \in \mathcal{C}(s_i)$. The cost model (Section 6.2.3) predicts, for the pair (s, n) and the node p that ran the previous stage, the quantities in Table 6.13.

Table 6.13: Notation used in the algorithm descriptions

Symbol	Meaning
$t_c(s, n)$	predicted compute time of stage s on node n, in ms
$t_x(p, n, s)$	predicted transfer time of the stage's data from node p to node n in ms (0 if $p = n$ or s is the first stage)
$E(s, n)$	predicted energy in kJ
$\rho(s, n) \in [0, 1]$	risk score, including the loss of the link $p \to n$
$r_n,\ A_n$	forecast reliability in $[0, 1]$ and availability window in seconds o node n
$f(n)$	federation of node n (zone label)
$L = \sum_i \big(t_c(s_i, a(s_i)) + t_x(a(s_{i-1}), a(s_i), s_i)\big)$	predicted job latency, in ms

6.7.2 Greedy Family: Greedy, Cheapest-Energy, Cheapest-Cost, Greenest, Bandit

Idea. The greedy family places the stages one at a time. For each stage it scores every feasible node and takes the lowest score; it never revisits an earlier decision. The four named strategies differ only in the weights of one score.

Score. For stage s, candidate n, previous node p:

$$\begin{aligned} \text{score}(n) = {} & t_c + t_x + w_\rho\,\rho + w_E\,E + w_\$\,M + w_g\,G \\ & + w_r\,(1 - r_n) + w_c\,\max\Big(0,\ 1 - \tfrac{A_n}{120}\Big)\,[A_n < 120] \\ & - b_{\text{loc}}\,[n = p] - b_{\text{stick}}\,[n = \text{previous assignment}(s)], \end{aligned} \tag{6.1}$$

where M is the placement's cost per hour plus cross-site egress, G its carbon rate in $\mathrm{g\,CO_2}$ per hour, and $[\cdot]$ is 1 when the condition holds. The weights are given in Table 6.14; the last two lines of the score reward keeping consecutive stages on one node (so transfers are avoided) and keeping a stage where a previous plan placed it (stability across re-plans).

Table 6.14: Parameters of the greedy family (`DEFAULT_CFG` and the factory overrides)

Parameter	Value	Meaning
w_ρ (risk)	10	ms of penalty per unit of risk
w_E (energy)	0; 0.1 for Cheapest-Energy	ms per kJ
$w_\$$ (cost)	0; 25 for Cheapest-Cost	ms per currency unit per hour; egress is multiplied by 1000 before weighting
w_g (carbon)	0; 25 for Greenest	ms per g CO_2 per hour
w_r (reliability)	1200	ms penalty at zero reliability
w_c (churn)	250	penalty when the availability window is shorter than 120 s
b_{loc}, b_{stick}	0.5 (factory), 0.75	locality and stickiness bonuses in ms
redundancy	1 (job may raise it)	the next redundancy − 1 candidates are listed as fallbacks

Why rates, not absolute cost. A stage that runs for a few hundred milliseconds costs almost nothing on any node, so an absolute-cost term cannot discriminate between candidates at any weight. The cost and carbon terms are therefore *per-hour rates* of the placement (rental of the requested cores and memory plus accelerator surcharge plus the electricity and carbon implied by the node's power draw). A consequence, discussed in Section 7.2.2, is that a rate is only a proxy: a low-power but slow node has a low carbon *rate* yet may emit as much per job as a fast node.

Algorithm 1: Greedy family: stage-by-stage placement

Input: job $J = (s_1, \ldots, s_m)$, twin state, weights w, `dry_run`
Output: assignment a, per-stage predictions, reservations
$p \leftarrow \perp$;
foreach *stage* s_i *in order* **do**
  $\mathcal{V} \leftarrow$ `nodes_for_planner()` ; `// fresh copy: sees earlier reservations`
  $\mathcal{C} \leftarrow \{n \in \mathcal{V} : \text{feasible}(s_i, n)\}$;
  if $\mathcal{C} = \emptyset$ **then** mark s_i infeasible; $p \leftarrow \perp$; **continue**;
  foreach $n \in \mathcal{C}$ **do**
    compute t_c, t_x, E, ρ, M, G and score(n) by Eq. (6.1);
  end
  $n^\star \leftarrow \arg\min_{n \in \mathcal{C}} \text{score}(n)$; fallbacks $\leftarrow$ the next redundancy − 1 nodes;
  if ***not*** `dry_run` **then**
    id $\leftarrow$ `reserve`$(n^\star, s_i)$; **if** $id = \perp$ **then** mark infeasible; **continue**;
  end
  $a(s_i) \leftarrow n^\star$; $p \leftarrow n^\star$;
end
return a, `job_cost`(J, a);

Complexity. Each stage costs one copy of the node table and the scoring of at most $|\mathcal{N}|$ candidates, so planning a job is $O(m\,|\mathcal{N}|)$ cost-model evaluations. Copying the node table dominates in practice (Section 7.2.1).

Bandit variant. The `bandit` strategy lets a UCB1 policy choose the execution format used

to evaluate each candidate. For a context (stage signature, node) the arms are the formats that the stage allows and the node supports (minus forbidden formats; if the stage names none, the node's formats). Each arm a keeps a pull count n_a and a running mean reward $\bar{x}_a$, with $N = \sum_a n_a$, and the policy chooses

$$a^\star = \arg\max_a \left(\bar{x}_a + \sqrt{\tfrac{2\ln N}{n_a}}\right), \qquad \text{unpulled arms have score } +\infty,$$

and after the stage is placed it records the reward $1 - \min(1,\, t_c/2000)$. The stage signature buckets the stage's type, size and resource requests so that similar stages share statistics. The state can be persisted to disk; the factory builds the policy without a path so that throw-away planners (the CI gate, the what-if forks) never write learned state.

6.7.3 Resilient (Federated) Planner for Fault-Tolerant Placement

Idea. The greedy family optimises one placement at a time and keeps no spare. The resilient planner (`dt/policy/resilient.py`) treats every placement as a bet against failure: it penalises concentrating a job in one federation, penalises federations that are already loaded, degraded or hot, penalises unreliable links and nodes, and records one or more *fallback* nodes per stage; when the plan is committed it also reserves capacity on those fallbacks (*shadow reservations*) so that the self-healing controller can promote one instantly.

Score. For stage s and candidate n in federation $f = f(n)$, the score is a sum of terms, all in milliseconds, *lower is better*:

$$\begin{aligned}\mathrm{score}(n) = {} & \underbrace{t_c + t_x}_{\text{latency}} + \underbrace{w_{\text{load}}\,\bar{u}_f(s)}_{\text{load}} + \underbrace{w_{\text{spread}}\,k_f}_{\text{spread}} + \underbrace{w_{\text{net}}\big([\text{link down}] + \min(1, \lambda/10)\big)}_{\text{network}} \\ & + \underbrace{w_{\text{res}}\,(d_f + h_f)}_{\text{resilience}} + \underbrace{w_{\text{risk}}\,\rho}_{\text{risk}} + \underbrace{w_{\text{rel}}\,(1 - r_n)}_{\text{reliability}} \\ & + \underbrace{w_{\text{av}}\,\tfrac{\max(0,\,H - \min(A_n, H))}{H}}_{\text{availability}} - \underbrace{b_{\text{prev}}\,[n = p]}_{\text{locality}}.\end{aligned} \tag{6.2}$$

Here $\bar{u}_f(s)$ is the mean projected utilisation of federation f over CPU, memory and VRAM after placing s, k_f is the number of stages of this job already placed in f, λ is the loss percentage of the link $p \to n$, d_f and h_f are the fractions of nodes of f that are down and hot, and H is the availability horizon. Feasibility is a hard filter: infeasible nodes never receive a score.

Table 6.15: Presets of the resilient planner (`DEFAULT_MODES`)

Parameter	**resilient**	**network-aware**	**federated**
redundancy (primary + fallbacks)	2	1	3
w_{risk}	220	200	210
w_{load}	380	260	360
w_{spread}	210	140	260
w_{net}	240	300	230
w_{res}	250	190	240
b_{prev}	15	12	10
w_{rel}	360	320	340
w_{av} (ms)	260	220	240
horizon H (s)	240	180	300

Adaptive tuning. Before planning, the weights of the chosen preset are adjusted from the current state of the fabric: if the mean link latency exceeds 20 ms, w_{net} is multiplied by 1.25 and w_{spread} by 1.1; if the mean link loss exceeds 1.5 %, w_{res} is multiplied by 1.2; if the mean node reliability is below 0.85, w_{rel} becomes $1.2\,w_{\text{rel}} + 25$ and w_{res} is multiplied by 1.1; if the shortest availability window is below the horizon, w_{av} is multiplied by 1.15. The redundancy is the larger of the preset's and the job's own.

Fallbacks and shadows. After sorting the candidates, the best becomes the primary. The planner then takes the next redundancy $-$ 1 candidates as fallbacks, *preferring nodes in other federations*, so that a zone-wide outage cannot remove both. When the plan is committed, the primary is reserved with role `primary` and each fallback with role `shadow`. After each stage the planner subtracts the stage's resources from its working copy of the node and of its federation, so later stages of the same job see the reduced capacity (this also happens in dry-run mode). The plan reports two summary metrics: *federation spread* (distinct federations used divided by the number of stages) and *fallback coverage* (share of stages with at least one fallback).

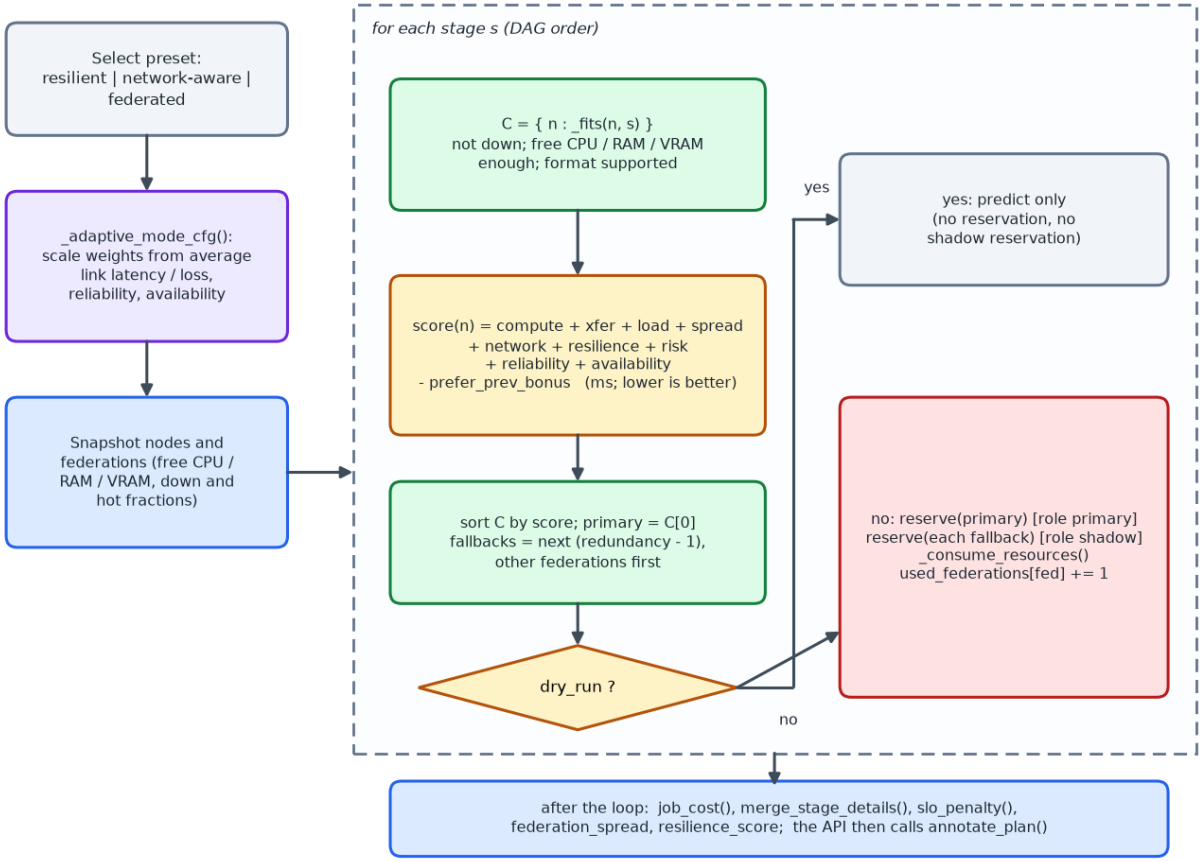


Figure 6.11: Flowchart of the resilient planner

Algorithm 2: Resilient (Federated) planner

Input: job J, twin state, preset, `dry_run`
Output: assignment a, fallbacks and shadow reservations per stage
$\theta \leftarrow$ adapt(preset, state); take a snapshot of nodes and federations; $p \leftarrow \perp$; $k \leftarrow 0$ for every federation;
foreach *stage* s_i **do**
  $\mathcal{C} \leftarrow \{n : \text{feasible}(s_i, n)\}$; **if** $\mathcal{C} = \emptyset$ **then** mark infeasible; $p \leftarrow \perp$; **continue**;
  foreach $n \in \mathcal{C}$ **do**
    compute score(n) by Eq. (6.2);
  end
  sort $\mathcal{C}$ by score; $n^\star \leftarrow \mathcal{C}[1]$;
  $F \leftarrow$ first $(\theta.\text{redundancy} - 1)$ nodes of $\mathcal{C}[2..]$ in a federation $\neq f(n^\star)$, then the rest;
  if ***not*** `dry_run` **then**
    reserve $n^\star$ (primary); **foreach** $n \in F$ **do**
      reserve n (shadow)
    end
  end
  subtract the stage's resources from the working copies of $n^\star$ and of $f(n^\star)$;
  $k_{f(n^\star)} \leftarrow k_{f(n^\star)} + 1$; $p \leftarrow n^\star$;
end
return *a,* `job_cost`*, federation spread, fallback coverage*;

Complexity. Scoring is $O(m\,|\mathcal{N}|)$ and sorting adds $O(m\,|\mathcal{N}|\log|\mathcal{N}|)$. Unlike the greedy planner it takes one snapshot per job rather than one per stage, which makes it cheaper in practice.

6.7.4 RL-Markov Planner: Markov Decision Process

Idea. The RL-Markov planner (`dt/policy/mdp.py`) treats a job as a finite-horizon Markov decision process and solves it exactly by dynamic programming, so that a decision at stage i accounts for its effect on the stages that follow (for example, avoiding a node from which every onward transfer is slow).

Formulation. The state is (i, p) – the index of the next stage and the node that ran the previous one. The action is the choice of a node $n \in \mathcal{C}(s_i)$. With probability $1 - \rho_n$ the stage succeeds and the process moves to $(i+1, n)$; with probability ρ_n (the risk score) it fails and incurs the failure penalty $F = 10$. The Bellman recursion, with discount $\gamma = 0.92$, is

$$V(i,p) = \min_{n \in \mathcal{C}(s_i)} \Big[\, c_i(n) \;+\; \gamma\big((1-\rho_n)\,V(i+1,n) + \rho_n\,F\big)\Big], \qquad V(m+1,\cdot) = 0, \tag{6.3}$$

and $V(i,p) = F$ if no node is feasible. The recursion is memoised on (i,p), so each state is solved once.

Stage cost: weighted Tchebycheff. Instead of one ad-hoc scalar, the immediate cost $c_i(n)$ aggregates nine criteria. For each criterion k the values over the candidates are normalised to $z_k(n) = \frac{x_k(n) - x_k^{\min}}{x_k^{\max} - x_k^{\min}} \in [0,1]$, and

$$c_i(n) = \max_k\; w_k\, z_k(n) \;+\; \rho_{\text{aug}} \sum_k \big(w_k\, z_k(n)\big)^2, \qquad \rho_{\text{aug}} = 0.12. \tag{6.4}$$

The maximum term guarantees that the worst-performing criterion is never ignored; the small quadratic term breaks ties among candidates that share the same worst criterion (an *augmented* Tchebycheff scalarisation).

Table 6.16: Criteria and weights of the Markov planner

Criterion	**Weight**	**Definition**
latency	0.34	$t_c + t_x$
energy	0.14	$E(s, n)$ in kJ
risk	0.16	$\rho(s, n)$
load	0.10	mean projected utilisation of the node over CPU, memory and VRAM
network	0.08	$0.7 \min(1, t_x/500) + 0.3 \min(1, \lambda/5)$
federation	0.06	load factor of the node's federation
RL	0.04	penalty derived from the learned Q-value (below)
reliability gap	0.04	$1 - r_n$ (0.5 if unknown)
availability gap	0.04	$\max(0, H - \min(A_n, H))/H$ with $H = 240\,\text{s}$ (0.5 if unknown)

Reinforcement-learning component. The planner can be paired with a tabular Q-learning policy (`dt/policy/rl_stub.py`) whose key is a stage signature (type and bucketed size, CPU, memory, VRAM) together with a node. After each plan – including dry runs; only persistence to disk is skipped for dry runs – the planner records a transition for every stage and applies the temporal-difference update

$$Q(\sigma, n) \leftarrow Q(\sigma, n) + \alpha\Big(r + \gamma \max_{n'} Q(\sigma', n') - Q(\sigma, n)\Big), \quad r = 1 - \min\big(1, \tfrac{t_c}{2000}\big),$$

with $\alpha = 0.3$, $\gamma = 0.92$. The learned value enters the objective as the criterion "RL": the penalty is 0.5 when $Q \leq 0$ and $0.5 - 0.5 \tanh(Q/10)$ otherwise. The policy also contains an ε-greedy node selector ($\varepsilon = 0.15$) and a SARSA variant, but the planner never calls the selector; the Q-values act only through the penalty. With an untrained table all candidates have the same penalty, so the RL criterion has no effect until experience accumulates.

Fallbacks. After solving a stage, the planner lists the next redundancy − 1 candidates (redundancy 3) by total cost as fallbacks.

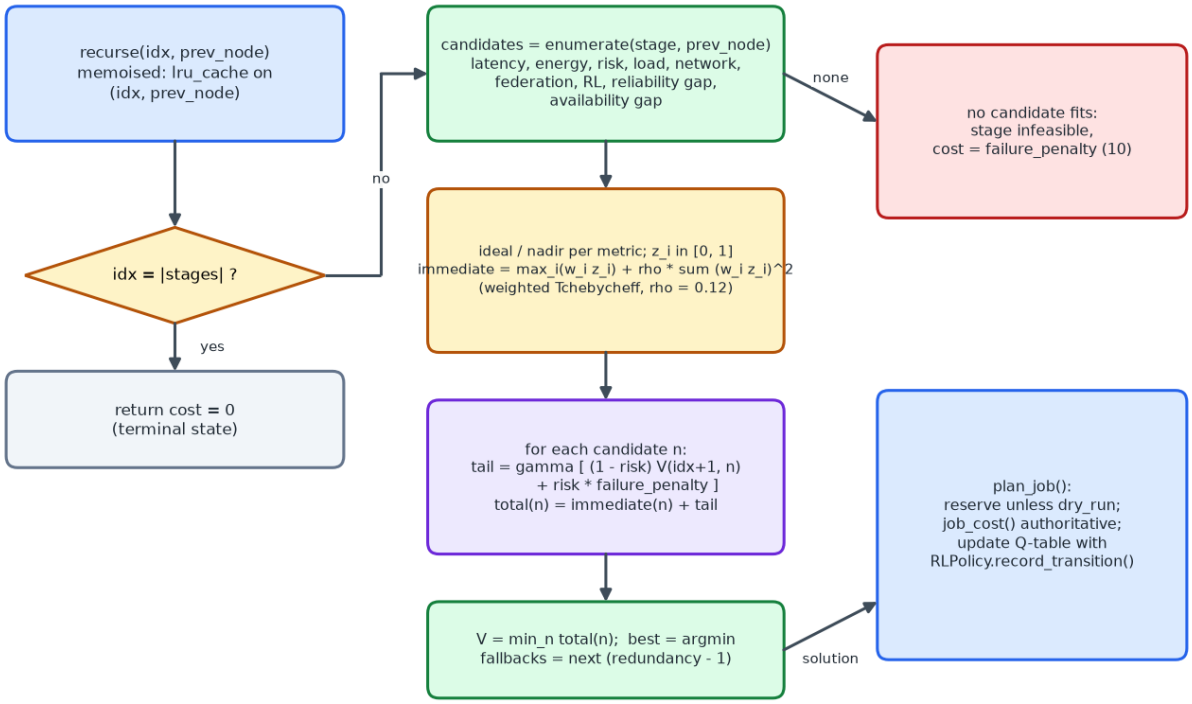


Figure 6.12: Flowchart of the Markov planner's recursion

Algorithm 3: RL-Markov planner: memoised finite-horizon dynamic programme

Input: job J, twin snapshot, weights w, γ, F, ρ_{aug}
Output: assignment a minimising the expected discounted cost $V(1, \perp)$
Function `Solve`(i, p)**:**
  if $i > m$ **then return** $(\emptyset, 0)$;
  if $memo[i, p]$ *exists* **then return** $memo[i, p]$;
  $\mathcal{C} \leftarrow$ enumerate feasible candidates of s_i given p with all nine criteria;
  if $\mathcal{C} = \emptyset$ **then return** $(\mathit{infeasible}, F)$;
  normalise each criterion over $\mathcal{C}$ (ideal and nadir);
  foreach $n \in \mathcal{C}$ **do**
    $(a', v') \leftarrow$ `Solve`$(i + 1, n)$;
    $\text{total}(n) \leftarrow c_i(n) + \gamma\left((1 - \rho_n)\, v' + \rho_n F\right)$;
  end
  $n^\star \leftarrow \arg\min_n \text{total}(n)$; $\text{memo}[i, p] \leftarrow (\{s_i \mapsto n^\star\} \cup a'_{n^\star},\ \text{total}(n^\star))$;
  return $memo[i, p]$;
$(a, V) \leftarrow$ `Solve`$(1, \perp)$; reserve unless `dry_run`; update the Q-table;

Complexity. There are at most $m\,(|\mathcal{N}| + 1)$ states, and each enumerates up to $|\mathcal{N}|$ candidates, so the planner performs $O(m\,|\mathcal{N}|^2)$ cost-model evaluations (about 35 000 for a 3.5-stage job on 100 nodes), which is why it is two orders of magnitude slower than the heuristics (Section 7.2.1).
Capacity caveat. The dynamic programme evaluates every stage against the *unmodified* snap-

shot; it does not subtract the resources that earlier stages of the same job will consume. Several stages can therefore be stacked on one node that cannot hold them all, which surfaces as a failed reservation when the plan is committed (Section 7.5).

6.7.5 Cost, Energy and Carbon Objectives

The cost-aware and carbon-aware strategies reuse the greedy score of Eq. (6.1) with $w_{\$}$ or w_g non-zero. Using the rate functions of Section 6.2.4, the rental rate of a placement is $c\,r_{\text{cpu}} + m\,r_{\text{mem}}$ plus an accelerator surcharge when the stage runs in a CUDA or ASIC format or requests VRAM (or an NPU surcharge in NPU format); the power rate is $\hat{P} = E/t_c$ (kW), so that the electricity rate is $\hat{P}\,p_{\text{kWh}}$ and the carbon rate is $\hat{P}\,g$. Cross-site egress, $\frac{\text{size}}{1024}\,r_{\text{egress}}$ when consecutive stages run in different sites, is a one-off charge that is multiplied by 1000 so that it is comparable with an hourly rate.

6.7.6 Predictive Reliability Update

Every observation that changes a node (a reservation, a release, a fault, a telemetry sample) triggers an update of that node's forecast: utilisation is smoothed with $\alpha = 0.35$, the trend over a window of 12 samples gives the 30-second look-ahead, and the reliability is recomputed from the descriptor value, the mean time between failures and the battery as in Section 6.2.2. The forecasts are read by the cost model (a utilisation forecast stretches compute time by up to 35 %, and a reliability shortfall adds up to 6 s) and by the planners' reliability and availability terms.

6.7.7 Self-Healing Decision Rule

The self-healing controller (Section 6.2.7) applies a simple rule per committed stage. Writing D for the predicate "the primary node is missing, down, has reliability below 0.75 or a predicted failure window below 90 s":

Algorithm 4: Self-healing: one polling tick for one managed stage

Input: managed stage g (primary, shadows, fallbacks, redundancy)
if $D(g.primary)$ **then**
  g.ticks $\leftarrow g$.ticks $+ 1$;
else
  g.ticks $\leftarrow 0$; top up shadows to redundancy $- 1$;
  return;
end
if $g.ticks < 2$ **then return** ; `// stability window`
;
if *a live shadow exists* **then**
  release the old primary; the oldest live shadow becomes the primary; emit `promote`;
  return;
end
foreach *fallback node n in random order* **do**
  id $\leftarrow$ reserve(n);
  if $id = \perp$ **then continue**;
  if *release(old primary) succeeds* **then**
    n becomes the primary; emit `failover`;
    return;
  else
    release(n, id);
  end
end

6.7.8 Minimal Fault-Set Search (Blast Radius)

Given a job J, a planner, a deadline D and a fault space Φ (zone blackouts, node failures, derates and link failures), the search finds the minimal subsets $S \subseteq \Phi$ such that planning J under S exceeds D or is infeasible. It exploits monotonicity (a superset of a breaking set also breaks the job) to prune.

Algorithm 5: Breadth-first search for minimal breaking fault sets

Input: job J, fault space Φ, depth limit d, branching limit b, set limit M
Output: minimal breaking sets $\mathcal{B}$
$\mathcal{B} \leftarrow \emptyset$;
foreach $\phi \in \Phi$ **do**
  evaluate($\{\phi\}$); **if** *broken* **then** add $\{\phi\}$ to $\mathcal{B}$;
end
$\Sigma \leftarrow$ surviving single faults sorted by the latency they cause (descending); keep the first b;
for $k = 2$ **to** d **do**
  foreach $S \in \binom{\Sigma}{k}$ **do**
    if $|\mathcal{B}| \geq M$ **then return** $\mathcal{B}$;
    if S *contains a set already in* $\mathcal{B}$ **then continue**;
    if *evaluate*(S) *is broken* **then** add S to $\mathcal{B}$;
  end
end
return $\mathcal{B}$;

Each evaluation applies the faults to a state object, takes a checkpoint of the predictor, plans the job, and then reverts the faults and restores the checkpoint, so that the $\approx 1 + |\Phi| + \binom{b}{2}$ evaluations are independent.

6.7.9 Comparison of the Strategies

Table 6.17 contrasts the algorithms along the dimensions that matter when choosing one. The measured planning times are those of Section 7.2.1.

Table 6.17: Comparison of the planning strategies

Property	**Greedy family**	**Bandit**	**Resilient**	**RL-Markov**
Decision unit	one stage at a time	one stage at a time	one stage at a time	whole job (DP)
Objective	latency + weighted terms	as greedy	latency + resilience terms (ms)	Tchebycheff over nine criteria
Fallbacks	optional list	none	yes, with shadow reservations	yes (2)
Learning	none	UCB1 over formats	none	Q-table (weak effect)
Complexity	$O(m\|\mathcal{N}\|)$	$O(m\|\mathcal{N}\|)$	$O(m\|\mathcal{N}\| \log \|\mathcal{N}\|)$	$O(m\|\mathcal{N}\|^2)$
Mean planning time per job	38–43 ms	43 ms	28 ms	1 618 ms

CHAPTER 7
RESULTS AND DISCUSSION

7.1 Experimental Setup

7.1.1 Dataset

Benchmarks use the project's own fabric (100 nodes, Table 6.11) and job catalogue (`jobs/jobs_10.yaml`: 11 jobs, 36 stages). Two groups of measurements are reported. The *benchmark table* (Table 7.1) was produced by `tools/benchmark_srs_table.py` at commit `2e7962d`, seed 7, on the first 6 jobs: `tools/policy_benchmark.py` gives mean job completion time (JCT) and energy under normal operation, and `tools/ci_gate.py` gives p95 JCT and SLA hit rate under the `core_link_cut` scenario at `deadline_scale=0.28`. These are two different operating conditions, not one run, and the output file says so. The *additional experiments* of Section 7.2 were run for this report with `experiments/run_experiments.py` on a laptop (Intel Core i5-1240P, 16 threads, 15 GB RAM, Linux, Python 3.14.7, 17 October 2026) over all 11 jobs. "Latency" everywhere means the *model-predicted* job latency, not a measured execution.

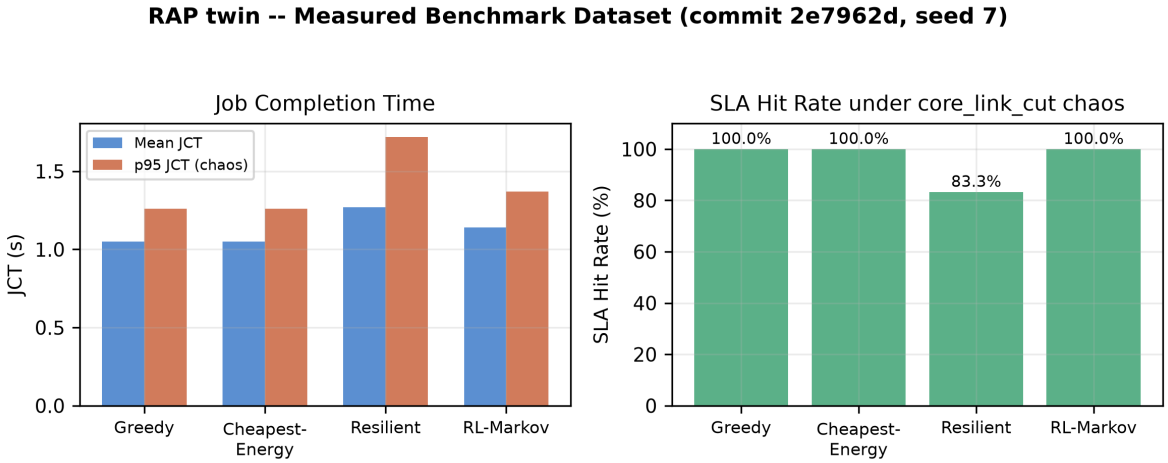


Figure 7.1: RAP twin – Measured Benchmark Dataset (commit `2e7962d`, seed 7)

7.1.2 Performance Parameters

Table 7.1: Measured Benchmark Table (commit `2e7962d`, seed 7, job_limit 6)

Strategy	Mean JCT (s)	p95 JCT (s)	Energy (J)	SLA Hit Rate
Greedy	1.05	1.26	13.0	100.0%
Cheapest-Energy	1.05	1.26	13.0	100.0%
Resilient	1.27	1.72	13.0	83.3%
RL-Markov	1.14	1.37	16.2	100.0%

Mean JCT and energy: normal operation. p95 JCT and SLA hit rate: `core_link_cut` at `deadline_scale=0.28`. Two different conditions.

7.2 Additional Experiments

7.2.1 Strategy Comparison in Dry-Run Mode

Table 7.2: Dry-run planning of the 11-job catalogue on the 100-node fabric (one pass; planning time over 20 repetitions)

Strategy	Plan time mean (ms)	Plan time p95 (ms)	Latency mean (ms)	Latency p95 (ms)	Energy (kJ)	Cost (Rs.)	CO_2 (mg)
Greedy	38.7	50.1	1 033	1 268	0.1406	0.0563	29.0
Cheapest-Energy	37.6	47.5	1 033	1 268	0.1406	0.0563	29.0
Cheapest-Cost	41.5	52.7	1 033	1 268	0.1406	0.0563	29.0
Greenest	41.4	52.5	1 033	1 268	0.1406	0.0563	29.0
Bandit	43.1	54.9	1 033	1 268	0.1406	0.0563	29.0
Resilient	27.7	37.2	1 229	1 516	0.1417	2.2284	28.6
RL-Markov	1 617.8	2 431.6	1 120	1 385	0.1782	0.0622	35.8

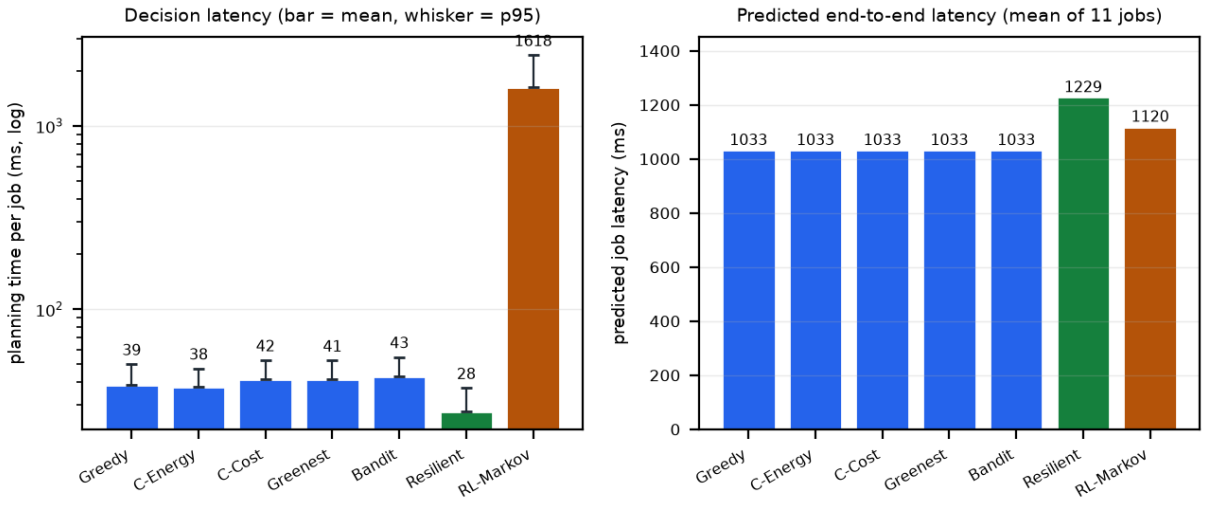


Figure 7.2: Planning time and predicted latency per strategy

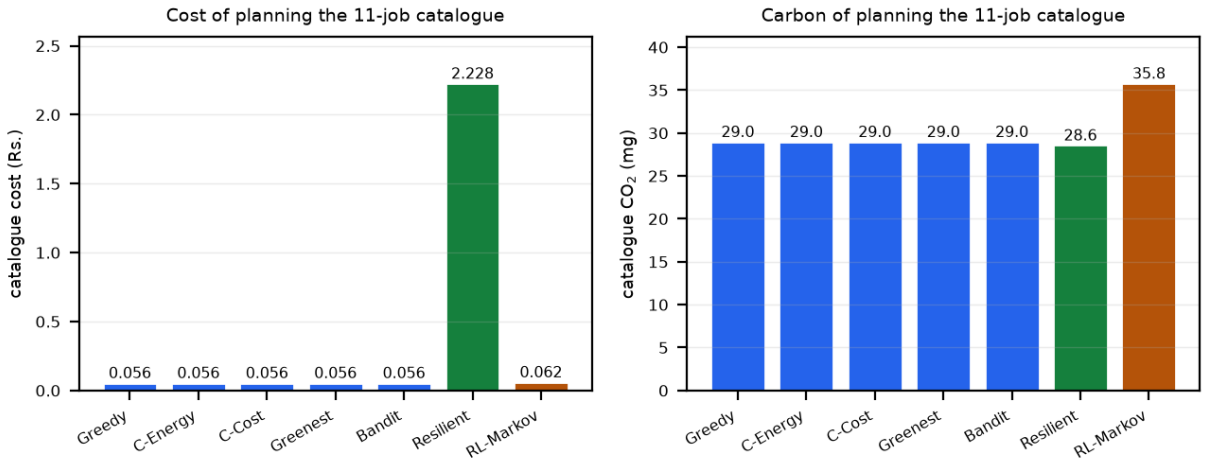


Figure 7.3: Catalogue cost and carbon per strategy (dry run)

Four observations follow from Table 7.2. (1) *Five strategies are indistinguishable.* Greedy, Cheapest-Energy, Cheapest-Cost, Greenest and Bandit placed all 36 stages on the same node, `hpc-080` (128 cores). In dry-run mode no job consumes capacity, so every job independently picks the fastest node, and its speed outweighs the small energy, cost and carbon terms. (2) *Resilient* predicts 19 % higher mean latency (1 229 vs 1 033 ms) and a cost about 40 times higher (Rs. 2.23 vs 0.056); 97 % of that is cross-zone egress (Rs. 2.17) because its spread penalty places a job in 2.9 zones on average, against 1.0 for the greedy family. (3) *RL-Markov* predicts 8 % higher latency but 57 % lower risk, 27 % more energy, and puts every stage on one server (`srv-057`). Its planning time is 1.6 s per job, about 42 times the greedy planner's. (4) *Planning time.* The greedy planner needs 38.7 ms per job, which exceeds the SRS target of 5 ms. A profile attributes about 60 % of this to copying the 100-node table once per stage in `nodes_for_planner()`; the resilient planner copies once per job and is faster (27.7 ms).

7.2.2 Committed Planning with Contention

Table 7.3: Sequential committed planning of the 11-job catalogue (each job sees the capacity reserved by earlier jobs)

Strategy	Infeas. jobs	Latency mean (ms)	Latency p95 (ms)	Cost (Rs.)	CO_2 (mg)	Nodes used	Reserv. held
Greedy	0	1 100	1 409	0.808	29.2	3	36
Cheapest-Energy	0	1 100	1 409	0.808	29.2	3	36
Cheapest-Cost	0	1 139	1 524	0.059	36.0	4	36
Greenest	0	1 122	1 475	1.042	29.2	4	36
Bandit	0	1 100	1 409	0.808	29.2	3	36
Resilient	0	1 278	1 738	2.659	30.0	9	72
RL-Markov	3	1 181	1 460	0.362	30.9	4	33

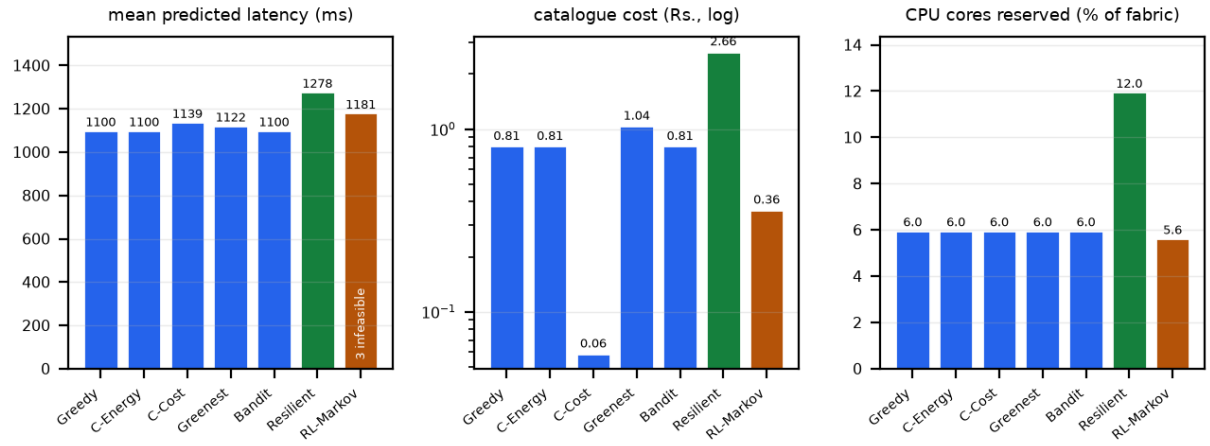


Figure 7.4: Committed sequential planning: latency, cost and reserved CPU

When reservations are committed, strategies diverge. Cheapest-Cost lowers the catalogue cost from Rs. 0.808 to 0.059 (93 %) for 3.5 % more latency, because the greedy plan spills across zones and pays Rs. 0.75 egress while Cheapest-Cost stays within two zones. Greenest did *not* reduce carbon (29.2 mg for both) and raised cost: its objective is a carbon *rate* per hour, and a slower low-power node has a lower rate but emits as much per job. Resilient doubles the reservations (72 vs 36, shadows) and CPU held (12 % vs 6 %) and pays Rs. 2.60 egress. RL-Markov left 3 of 11 jobs infeasible (`reservation_failed`) because its dynamic programme stacks stages on one node without accounting for capacity (Section 7.5); its latency covers only the 8 feasible jobs.

7.2.3 Chaos-as-CI Gate

Table 7.4: Resilience gate results (committed baseline, seed 7, 6 jobs, deadline scale 0.28)

Scenario	Strategy	Events	Worst SLA	Mean SLA	Worst p95 (ms)	Peak cost (Rs.)
`default`	greedy	10	100.0%	100.0%	1 260	0.033
`default`	resilient	10	83.3%	83.3%	1 721	1.586
`core_link_cut`	greedy	16	100.0%	100.0%	1 260	0.033
`core_link_cut`	resilient	16	83.3%	83.3%	1 721	1.586
`gate_capacity_squeeze`	greedy	10	100.0%	100.0%	1 458	0.619
`gate_capacity_squeeze`	resilient	10	83.3%	85.7%	1 732	1.586

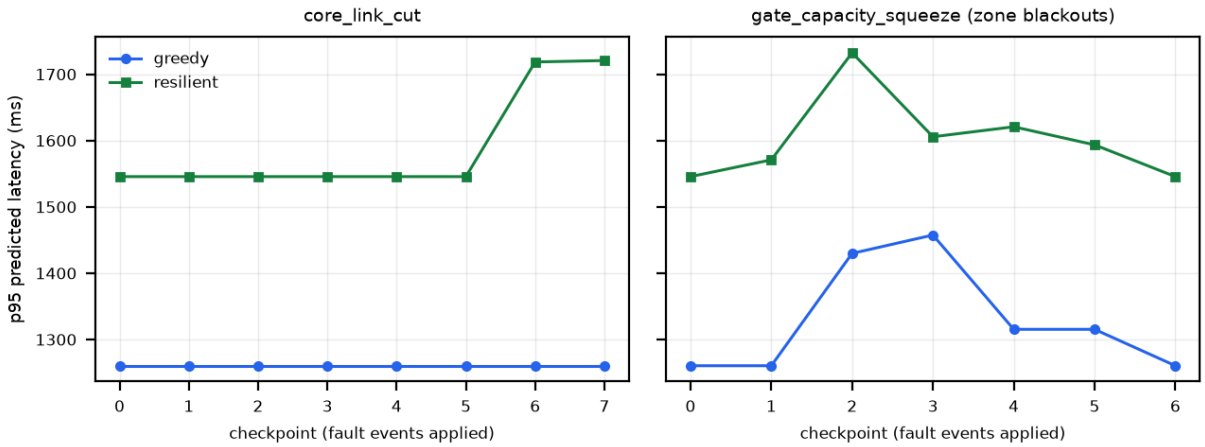


Figure 7.5: p95 predicted latency at each gate checkpoint

Re-running the gate against its committed baseline passed (exit code 0, 14.7 s here; 12.4 s in the baseline). The zone-blackout scenario raises Greedy's p95 from 1 260 to 1 458 ms (+16 %) and Resilient's from 1 546 to 1 732 ms (+12 %). Resilient's 83.3 % SLA (5 of 6 jobs) is already present at the fault-free baseline sample: it is caused by the tight deadline scale and the planner's higher latency, not by a fault. The named `core_link_cut` scenario left Greedy's plans unchanged at every checkpoint, because its events address site-level link keys and nodes that do not exist in the fabric (Section 7.5); only the zone events of the default list move the numbers.

7.2.4 Monte-Carlo Sweep

`sim/montecarlo.py` with 1 000 trials (seed 123, 26.7 s) gave a catalogue total of 516 ms mean, 618 ms p95 and 632 ms p99, and an SLO hit rate of 100 % (all trials had finite latency). Because this tool uses its own simplified cost formulas (no reliability penalty), its absolute values are about twenty times lower than the planners' predictions and must not be compared with them.

7.2.5 Blast-Radius and What-If Examples

The blast-radius search (Greedy, depth 2, branch 10) examined 16 candidate faults in 62 placements and reported that both `job-vision-large` (3.6 s) and `job-cv-no-gpu` (2.6 s) survive every combination searched. Table 7.5 shows three what-if analyses: even removing the three largest HPC nodes leaves all 11 deadlines met, and adding two servers changes nothing. This confirms that the reference fabric is heavily over-provisioned for the catalogue, so capacity tools report "no measurable difference".

Table 7.5: What-if analyses of the 11-job catalogue (Greedy, committed planning in both forks)

Change	Fabric	Deadlines met	p95 (ms)	Cost (Rs.)
Add 2 copies of `srv-015`	100 → 102 nodes	11/11 → 11/11	1 409 → 1 409	0.808 → 0.808
Remove `hpc-080`	100 → 99 nodes	11/11 → 11/11	1 409 → 1 466	0.808 → 1.276
Remove 3 HPC nodes	100 → 97 nodes	11/11 → 11/11	1 409 → 1 433	0.808 → 0.526

7.3 Efficiency Issues

- **Resilient trades latency and money for fallback coverage.** Its p95 is 36 % above Greedy's in the benchmark table (1.72 vs 1.26 s) and its SLA hit rate under the tight deadline is lower (83.3 % vs 100 %). Fallback coverage never pays off here because the benchmark's faults do not hit its placements.
- **The RL-Markov cost anomaly is not reproduced in dry-run.** The worklog recorded RL-Markov at about 75 times Greedy's cost (Rs. 0.894 vs 0.012) on one job. In the present dry-run measurement with a fresh Q-table it cost only 10 % more; the large egress bill appeared instead with Resilient (about 40 times). The anomaly therefore depends on the job and on learned state, and remains an open question.
- **Predictive-state leak, fixed.** `PredictiveAnalyzer` kept an elevated projected derate after a fault was reverted, biasing later trials; `checkpoint()/restore()` now wraps every blast-radius trial. `POST/overrides/reset` still does not clear predictive history.

7.4 Software Testing

7.4.1 Test Cases and Test Results

The suite has 166 pytest tests in 13 files; all pass (166 passed in 8.6 s). Tests use Flask's test client, temporary directories and fresh `DTState` objects, so they never touch the repository's data. Table 7.6 lists the test cases by group with the expected result; every group passed.

Table 7.6: Test cases by group (166 tests, all passed)

File / group	No.	Test cases	Expected result
`test_api.py`	3	Plan returns rich per-stage metrics (incl. CPU request); batch reuses plan logic; batch propagates errors	Fields present; errors returned
`test_api_auth.py`: key logic	6	No key set: all routes open; missing / wrong / correct key on a mutating route; read-only and evaluation routes open; SSE not gated	401 or success as specified
`test_api_auth.py`: gated routes	10	Each of the 10 gated routes without a key	401
`test_benchmark_*.py`	10	Unit conversion ms→s, kJ→J, p95, SLA %; missing p95 renders a dash; no fabricated DDQN baseline; plots written in both formats	Correct table and files
`test_blast_radius.py`	13	Apply/revert leave no trace; zone blackout hits all zone nodes; candidate space; independent trials; finds the single breaking fault; survival reported; minimal sets; common fault; truncation; fabric restored; report text; job lookup	Minimal, repeatable results

File / group	No.	Test cases	Expected result
test_bundle.py	10	Whole fabric bundled; live evidence optional; missing sources skipped; round trip; refuses non-empty target; rejects non-bundle and path traversal; version mismatch flagged; CLI	Safe export/import
test_ci_gate.py	19	Passing run; SLA floor; p95, cost, CO_2 ceilings; baseline regression and p95 budget; matching only same pair; deadline scale; job limit; inline scenario; percentile; reports written; failure when bar unreachable; determinism; fabric untouched; unknown scenario message; shipped config matches baseline	Correct verdicts; identical reruns
test_economics.py	11	Topology prices read; unpriced topology; hand-calculated stage cost; GPU only when accelerated; egress only across sites; zone overrides; rate independent of duration; annotation for any planner; carbon objective; infeasible not billed; site by labels	Exact arithmetic
test_import_cluster.py	24	Kubernetes quantity parsing (8); name sanitising; capacity and topology; GPU and VRAM label; control plane role; not-ready node; ARM; unknown arch; defaults; single node; duplicate names; bad payload; descriptors load into twin; overwrite rules; CLI dry run and bad JSON	Valid descriptors
test_launch_fabric.py	7	GPU fields for GPU and non-GPU nodes; real descriptors; device request only for GPU nodes; no side effects	Correct specs
test_management.py	15	Path-traversal names rejected; node removal and watcher; link upsert/remove/validation; reset overrides; node, link, job CRUD; validation; clear plans; chaos run and reset	Persisted and validated
test_mqtt_bridge.py	20	Node/link apply and revert parsing; default action; 10 malformed inputs never raise; posting; failures swallowed; no paho at construction; subscription	Pure parsing, no crash
test_whatif.py	18	Clone, patch, link change without persisting; 8 invalid change sets; capacity helps under contention; useless without contention; removal worse; faults on both sides; nothing persisted; cost and carbon; no jobs error	Correct diffs

Beyond unit tests, the gate is verified to be deterministic and the public deployment was checked end to end (every route answering, a real plan returning cost and CO_2, what-if and blast

radius returning reports).

7.5 Known Limitations and Threats to Validity

The following defects and limits were found by reviewing and running the code for this report. They are described, not fixed, because fixing them changes benchmark numbers and requires re-baselining the CI gate.

1. **Link p95 forecast.** `link_forecast()` uses the accumulator `trend.sum_xy` as the slope; all seven registered links report p95 $\approx 7.2 \times 10^{12}$ ms, so a transfer over any registered link between real nodes is priced as effectively infinite. The shipped links are keyed by site names or absent nodes (`ws-001`, `srv-003`, `sbc-010`), so ordinary plans do not hit it.
2. **Trend look-ahead inert.** With epoch-millisecond timestamps the least-squares slope loses precision (measured -10^{-6} against 0.02 with index timestamps), so the 30-second utilisation look-ahead has no effect.
3. **Reliability feedback.** The adjusted reliability is stored and fed back as the base on the next update; for a node with MTBF and uptime it decays geometrically (0.95, 0.71, 0.53, ..., 0.17 after six updates). No shipped node defines these fields.
4. **WebAssembly penalty has no effect.** The multiplier 1/1.35 is clamped by `max(1,a)`; WASM-only and native-only stages take the same time.
5. **RL selector unused.** `RLPolicy.choose_node()` is never called; Q-values act only through the RL criterion.
6. **MDP capacity stacking.** The dynamic programme ignores capacity consumed by earlier stages of the same job (3 of 11 jobs infeasible when committed); partial reservations of a failed job are not rolled back.
7. **Shipped chaos scenarios.** `core_link_cut` targets site-level link keys and non-existent nodes and does not change plans; `ws-002` in `wifi_brownout` is also absent.
8. **Overlapping chaos events.** A revert resets a field to its default, not to the value of another still-active event.
9. **Guardian and self-healer.** The guardian may release a primary's reservation without informing the self-healer.
10. **Link profiles ignored.** A link's `profile:` is stored but never expanded into metrics.
11. **Schemas.** `schemas/job.schema.yaml` does not parse (line 179), 25 of 100 node files fail the node schema, and `dt/validators.py` is not used by the API.
12. **FR-05.** No estimation-deviation (Δf) tracking exists; forecasts are never scored against outcomes.
13. **Docker launcher.** It reads a `resources` block that descriptors lack, so every container gets 1 CPU and 1 GB; link-rate shaping finds no node-level links.
14. **Monte-Carlo.** The `--dt` mode posts a payload the current `/plan` does not accept and silently falls back to the built-in planner; `makemontecarlo` writes CSV to a `.json` file.
15. **Planning speed.** 38.7 ms per job (target 5 ms).

16. **Validity of results.** Latencies are model predictions, not executions; the benchmark table is one seed and mixes two conditions; the fabric is over-provisioned; dry-run planning ignores contention; the dashboard's API key ships in the public JavaScript bundle and deters only casual misuse; measurements ran on a different machine and Python version from those in the SRS.

7.6 Summary

7.6.1 Result Analysis and Discussion

Three conclusions follow. First, no strategy wins on every metric: Resilient buys redundancy with latency, reservations and egress cost; RL-Markov is the slowest planner and can over-stack nodes; the greedy family is fast but, on this fabric, indistinguishable until capacity is contended. Second, rate-based cost and carbon objectives are proxies, and Greenest did not lower carbon. Third, the tooling is itself a result: the deterministic gate, the 166 tests and the reproducible scripts made these trade-offs and the defects above measurable.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 Conclusions

RAP twin delivers a working, tested and deployed digital-twin planning sandbox. Against the seven objectives of Chapter 3: (1) seven strategies run behind one interface; (2) the state engine merges descriptors and telemetry and reloads within about 0.5 s; (3) self-healing and the guardian act after a two-poll stability window; (4) the chaos engine (14 event kinds) drives a deterministic CI gate; (5) Monte-Carlo sweeps run seeded; (6) what-if and blast-radius analyses never touch live state; (7) Kubernetes import and DTDL/CRD export exist. All 166 tests pass. The project also reports its own limits: five strategies coincide on an over-provisioned fabric, Resilient costs far more in egress, the Greenest objective did not cut carbon, and several implementation defects (Section 7.5) qualify the numbers.

8.2 Future Work

1. **Algorithm recommendation system** (mentor suggestion): given a job's deadline, size, resource mix and the fabric's state, dry-run all strategies and rank them by predicted latency, cost, carbon and risk, explaining the trade-off in the dashboard.
2. **Fix the defects** of Section 7.5 (link p95, trend timestamps, capacity accounting in the MDP planner, shipped chaos scenarios, link profiles, schemas), then re-baseline the CI gate.
3. **Validate the twin against real containers** using the Docker launcher and report prediction error.
4. **Investigate the RL-Markov and Resilient cost behaviour** and add a tighter sample fabric so trade-offs are visible without tuning.
5. **Real reinforcement learning** with an episode loop, and a score of forecasts against outcomes (Δf, FR-05).
6. **Further UI refinement and documentation**, and live integration with a 5G/6G test-bed (FR-30).

8.3 Application

RAP twin applies to pre-production capacity planning (what-if), resilience testing of scheduling changes (CI gate), cost- and carbon-aware scheduling, incident reproduction with bundles, and teaching of scheduling and chaos engineering.

Annexure A

Program Outcome and Program Specific Outcome Mapping

A.1 Program Outcome (PO) Mapping

PO	Description	Level	Justification
PO1	Engineering knowledge: apply mathematics, science, and engineering fundamentals to complex engineering problems.	3	RAP twin applies mathematical modelling (MDP, cost/carbon optimization) and distributed-systems engineering principles to build a digital twin of heterogeneous fabrics.
PO2	Problem analysis: identify, formulate, and analyze complex engineering problems.	3	Analyzed complex resource-allocation bottlenecks, network variability, and planner trade-offs (Chapter 7) to identify optimal scheduling behaviour.
PO3	Design/development of solutions: design components or processes meeting specified needs.	3	Designed a modular platform including a thread-safe state machine, a pluggable multi-strategy planning engine, and a CI resilience gate.
PO4	Conduct investigations of complex problems using research-based methods.	3	Extensive literature review (Chapter 2) and measured benchmarking experiments were conducted, including an honestly-reported, unresolved cost anomaly.
PO5	Modern tool usage: apply appropriate modern engineering and IT tools.	3	RAP twin utilizes modern tools – Python, Flask, React, Docker, MQTT, YAML/JSON Schema, and CI/CD via GitHub Actions.
PO6	The engineer and the world: analyze societal, environmental, and sustainability impacts.	2	Addresses the need for cost- and carbon-aware computing via the Economics module and the Greenest/Cheapest-Cost strategies, reducing the carbon footprint of distributed workloads.
PO7	Ethics: apply ethical principles and professional responsibilities.	3	Ensures data integrity in telemetry ingestion, API-key-gated access to mutating routes, and honest reporting of unresolved issues rather than suppressing them.
PO8	Individual and team work: function effectively individually and in teams.	3	Developed collaboratively across three clearly defined roles (Section 4.4.1) under mentorship.
PO9	Communication skills: comprehend and write effective reports and documentation.	3	Maintained a Software Requirements Specification, an engineering worklog, a published paper draft, and this final report for the RAP twin project.

PO	Description	Level	Justification
PO10	Project management and finance: apply project management and financial principles.	2	The project involved planning for compute resource optimization and per-plan cost/carbon accounting, though it did not encompass full-scale engineering financial management.
PO11	Life-long learning: recognize the need for continuous learning amid technological change.	3	Built a scalable architecture that allows continuous addition of new planner strategies and interoperability targets as technology evolves.

A.2 Program Specific Outcome (PSO) Mapping

PSO	Description	Level	Justification
PSO1	Interpret data, use software tools to conduct experiments, and apply AI/ML techniques to multi-disciplinary problems.	3	Interprets real-time telemetry, uses software tools for chaos experiments and the CI resilience gate, and applies RL-Markov and MDP techniques to infrastructure-optimization problems.
PSO2	Apply standard practices, strategies, and models of Data Science to discover knowledge.	3	Applies predictive analytics and MDP-based models to discover optimal placement patterns, and statistically validates them through Monte-Carlo benchmarking.

Annexure B
References

[1] Q. Qi, F. Tao, T. Zuo and A. Y. Nee, "A review of digital twins in smart industries," *IEEE Access*, vol. 7, pp. 14232–14252, Jan. 2019.

[2] A. Fuller, Z. Fan, C. Day and C. Barlow, "Digital Twin: Enabling Technologies, Challenges and Open Research," *IEEE Access*, vol. 8, pp. 108952–108971, June 2020.

[3] F. Geyer and S. Bondorf, "Proximity-based service discovery for distributed digital twin systems," *IEEE Access*, vol. 9, pp. 12456–12470, 2021.

[4] Z. Lv, D. Chen, R. Lou, and H. Song, "Digital Twins: A Survey on Enabling Technologies, Challenges, Trends, and Future Prospects," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 1, pp. 54–87, 2022.

[5] M. Grieves and J. Vickers, "Digital twins: Recent advances and future directions in engineering fields," *Nature Reviews Methods Primers*, vol. 2, no. 1, pp. 1–18, Dec. 2022.

[6] J. Wang, L. Ye and R. X. Gao, "A simulation-based digital twin model for data-driven decision optimization," *Journal of Manufacturing Systems*, vol. 62, pp. 245–258, Jan. 2022.

[7] S. S. Kim, Y. J. Lee and M. G. Lee, "Digital twin model with machine learning for real-time monitoring and predictive maintenance," *International Journal of Precision Engineering and Manufacturing*, vol. 23, no. 5, pp. 565–578, May 2022.

[8] C. Xu, Z. Tang, H. Yu, P. Zeng and L. Kong, "Digital Twin-Driven Collaborative Scheduling for Heterogeneous Task and Edge-End Resource via Multi-Agent Deep Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 10, pp. 3056–3069, Oct. 2023.

[9] Y. Dai, J. Zhao, J. Zhang, Y. Zhang and T. Jiang, "Federated Deep Reinforcement Learning for Task Offloading in Digital Twin Edge Networks," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 3, pp. 2849–2861, May-June 2024.

[10] A. M. V. V. Sai, C. Wang, Z. Cai and Y. Li, "Navigating the Digital Twin Network landscape: A survey on architecture, applications, privacy and security," *High-Confidence Computing*, vol. 4, no. 4, p. 100269, Dec. 2024.

[11] S. M. Raza, R. Minerva, N. Crespi, M. Alvi, M. Herath and H. Dutta, "A comprehensive survey of Network Digital Twin architecture, capabilities, challenges, and requirements for Edge-Cloud Continuum," *Computer Communications*, vol. 236, pp. 165–192, Jan. 2025.

[12] H. Tran-Dang and D.-S. Kim, "Digital Twin-empowered intelligent computation offloading for edge computing in the era of 5G and beyond: A state-of-the-art survey," *ICT Express*, vol. 11, no. 2, pp. 167–180, Apr. 2025.

[13] E. Esen, A. Akbulut and C. Catal, "Chaos experiments in microservice architectures: A systematic literature review," *Computer Standards & Interfaces*, vol. 97, p. 104116, 2026.

Annexure C
Plagiarism Report

To be attached after the institute's plagiarism check.

Annexure D
Paper Published (if any)

Not applicable at the time of submission.

Annexure E
Sponsorship detail (if any)

None.